

Dedication

To my dear parents,

*for their unconditional love, sacrifices, prayers, and
constant support.*

To my family,

*for their encouragement, patience, and confidence
throughout this journey.*

To my teachers and mentors,

for their guidance, inspiration, and valuable advice.

I dedicate this work with deep gratitude and respect.

AIT-LAHCEN REDOUAN

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor for the guidance, availability, constructive feedback, and encouragement provided throughout this final-year project. The advice received helped me refine the scope of the work, strengthen the technical choices, and maintain a disciplined engineering approach from design to implementation.

I would also like to thank the academic staff whose teaching and methodological support shaped the foundations of this project. The knowledge acquired during my Master's program directly contributed to my understanding of data engineering, distributed systems, databases, software development, and the design of reliable information platforms.

My appreciation also goes to the professional environment that made this work possible and gave it practical relevance. Developing this project within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES allowed me to understand real data engineering constraints and to position this work as a distributed customer data platform intended to support trustworthy analytics, structured governance, and future AI-oriented developments.

I am equally grateful to my colleagues and classmates for their discussions, exchanges, and mutual support during this academic journey. Their presence made the periods of experimentation, debugging, and refinement both more productive and more meaningful.

I extend my deepest thanks to my family for their patience, encouragement, and constant confidence throughout the long phases of research, implementation, testing, and writing. Their support remained an essential source of motivation from the beginning of the project to its completion.

Finally, I would like to thank the members of the jury for the time and attention devoted to evaluating this work. I hope that this report faithfully reflects the seriousness, effort, and commitment invested in the design of a modern, distributed, and operational data engineering platform.

Abstract

Modern organizations generate large volumes of customer-related data across heterogeneous and fragmented operational environments. Campaign records, behavioral traces, commercial transactions, and profile attributes are produced in different formats, at different rates, and under inconsistent semantic assumptions. When these sources remain isolated, customer information becomes difficult to validate, reconcile, replay, and transform into dependable analytical assets. This fragmentation slows decision-making, prevents a unified customer view, and deprives future intelligent services of a sufficiently reliable data foundation.

This final-year project addresses that problem through the design and implementation of AdOptimizer CDP, a distributed customer-data lakehouse platform developed within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES. The completed platform combines replayable Kafka ingestion, HDFS bronze storage, Spark distributed processing, Iceberg and Parquet tables governed through Hive Metastore, and analytical access through Trino. dbt-spark structures the transformation layers, Great Expectations records validation evidence, Airflow orchestrates the workflows, Prometheus and Grafana provide observability, and layered security controls protect service identity, access, communication, and persisted runtime data.

Deployed across a local control plane and three Linux virtual machines, the platform was validated through end-to-end execution scenarios covering ingestion, distributed persistence, transformation, cross-engine analytical access, data-quality checks, orchestration, monitoring, and secure service interaction. The resulting architecture strengthens reproducibility, scalability, traceability, operational reliability, and extensibility. It transforms heterogeneous raw datasets into governed and reusable data products while establishing the engineering foundation for trusted analytics, future Customer 360 capabilities, and subsequent machine-learning developments.

Keywords: Big Data, Data Engineering, Distributed Systems, Data Lakehouse, Customer Data Platform, Data Governance, Data Quality, Observability, Security, Customer 360.

Résumé

Les organisations modernes génèrent d'importants volumes de données clients dans des environnements opérationnels hétérogènes et fragmentés. Les données de campagne, les traces comportementales, les transactions commerciales et les attributs de profil sont produits dans des formats différents, à des cadences variables et selon des hypothèses sémantiques non homogènes. Lorsque ces sources demeurent isolées, les informations clients deviennent difficiles à valider, à réconcilier, à rejouer et à transformer en actifs analytiques fiables. Cette fragmentation ralentit la prise de décision, empêche une vision client unifiée et prive les futurs services intelligents d'un socle de données suffisamment robuste.

Ce projet de fin d'études répond à cette problématique par la conception et la mise en œuvre de AdOptimizer CDP, une plateforme lakehouse distribuée consacrée aux données clients et développée dans le cadre de l'initiative plus large AdOptimizer AI au sein de SMART AUTOMATION TECHNOLOGIES. La plateforme mise en œuvre combine une ingestion rejouable avec Kafka, un stockage bronze dans HDFS, des traitements distribués avec Spark, des tables Iceberg et Parquet gouvernées par Hive Metastore, ainsi qu'un accès analytique via Trino. dbt-spark structure les couches de transformation, Great Expectations conserve les preuves de validation, Airflow orchestre les workflows, Prometheus et Grafana assurent l'observabilité, tandis que des contrôles de sécurité en couches protègent l'identité des services, les accès, les communications et les données persistées.

Déployée entre un plan de contrôle local et trois machines virtuelles Linux, la plateforme a été validée au moyen de scénarios d'exécution de bout en bout couvrant l'ingestion, la persistance distribuée, la transformation, l'accès analytique inter-moteurs, les contrôles de qualité, l'orchestration, la supervision et les interactions sécurisées entre services. L'architecture obtenue renforce la reproductibilité, la scalabilité, la traçabilité, la fiabilité opérationnelle et l'extensibilité. Elle transforme des jeux de données brutes hétérogènes en produits de données gouvernés et réutilisables, tout en établissant le socle d'ingénierie nécessaire à des analyses fiables, à de futures capacités Customer 360 et à des développements ultérieurs en apprentissage automatique.

Mots-clés : Big Data, Ingénierie des données, Systèmes distribués, Data Lakehouse, Plateforme de données client, Gouvernance des données, Qualité des données, Observabilité, Sécurité, Customer 360.

Contents

List of Figures

List of Tables

List of Acronyms and Abbreviations

Abbr.	Meaning
ACID	Atomicity, Consistency, Isolation, and Durability
AI	Artificial Intelligence
BASE	Basically Available, Soft State, and Eventual Consistency
BI	Business Intelligence
CAP	Consistency, Availability, and Partition Tolerance
CDP	Customer Data Platform
CPU	Central Processing Unit
DAG	Directed Acyclic Graph
dbt	Data Build Tool
DFS	Distributed File System
ETL	Extract, Transform, Load
GPU	Graphics Processing Unit
HDFS	Hadoop Distributed File System
HTML	Hypertext Markup Language
HTTPS	Hypertext Transfer Protocol Secure
IAM	Identity and Access Management
IP	Internet Protocol
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation

Abbr.	Meaning
JSONL	JSON Lines
KDC	Key Distribution Center
LTS	Long-Term Support
LUKS	Linux Unified Key Setup
ML	Machine Learning
ODBC	Open Database Connectivity
PFE	Projet de Fin d'Études
PKI	Public Key Infrastructure
QEMU	Quick Emulator
RAM	Random-Access Memory
SIEM	Security Information and Event Management
SPNEGO	Simple and Protected Generic Security Service Application Program Interface Negotiation Mechanism
SQL	Structured Query Language
SSH	Secure Shell
TLS	Transport Layer Security
UI	User Interface
VM	Virtual Machine
YARN	Yet Another Resource Negotiator

Chapter I

General Project Context

I.1 Introduction

This chapter establishes the organizational, business, and academic context of AdOptimizer CDP. It presents the host company and the broader strategic initiative, defines the customer-data problem addressed by the project, explains the choice of a distributed lakehouse approach, and states the implemented objectives, scope, boundaries, and contribution of the final-year project.

I.2 Host Organization and Strategic Environment

I.2.1 Company Context

AdOptimizer CDP was assigned and supported by SMART AUTOMATION TECHNOLOGIES as the professional framework of this final-year project. The company operates in a technology-oriented environment centered on digital innovation, automation, and intelligent systems. Within this context, customer data is treated as a strategic technical asset that must be ingested, governed, processed, exposed, validated, monitored, and secured through a coherent engineering platform.



Figure I.1: Visual identity of SMART AUTOMATION TECHNOLOGIES

The broader strategic direction behind the work is AdOptimizer AI, the company’s long-term vision for an intelligent advertising and marketing ecosystem supporting customer understanding, advanced analytics, and data-driven optimization. Because such capabilities require reliable and governed data, SMART AUTOMATION TECHNOLOGIES positioned the PFE around the distributed customer-data foundation needed by that wider initiative.

I.2.2 Institutional Values and Project Alignment

The visual identity of SMART AUTOMATION TECHNOLOGIES highlights five institutional values: **Innovation**, **Team Spirit**, **Integrity**, **Excellence**, and **Commitment**.



Figure I.2: Core values of SMART AUTOMATION TECHNOLOGIES

These values are reflected in the project through the adoption of a modern lakehouse architecture, the construction of a shared foundation for engineers and analysts, the emphasis on trusted and reproducible data, the integration of complementary services into one end-to-end platform, and the operational effort required to stabilize a distributed implementation.

I.2.3 Position of the Project Within AdOptimizer AI

Within the broader AdOptimizer AI vision, AdOptimizer CDP is the **distributed customer-data lakehouse platform** that provides the initiative’s Data Engineering and Big Data foundation. Its role is to prepare governed, reusable, and trustworthy data products rather than to deliver the final intelligent marketing functions directly.

Future services may include customer segmentation, conversion and lifetime-value analysis, churn prediction, personalization, campaign optimization, persona generation, and recommendation-oriented decision support. These downstream capabilities remain outside

the implemented PFE scope. The completed contribution is the distributed, observable, quality-governed, and secured data platform on which such services can later depend.

I.3 Project Definition and Scope

I.3.1 Problem Statement

Customer-related data is fragmented across operational contexts. In this project, the fragmentation is represented by campaign-response and profile data, digital behavioral and purchase-intent data, and retail transaction histories. When these sources remain isolated, customer understanding stays incomplete, reprocessing becomes difficult, analytical trust weakens, and governed datasets for downstream consumers are harder to produce.

The engineering problem is therefore broader than data storage. It requires a platform that can ingest heterogeneous sources reproducibly, preserve replayable raw artifacts, process them into governed lakehouse tables, expose trusted analytical access, validate data quality, coordinate execution, observe distributed behavior, and secure core infrastructure flows.

The project addresses the following engineering question:

How can a distributed customer-data platform reproducibly ingest heterogeneous datasets, preserve replayable raw data in distributed storage, construct governed lakehouse tables, expose trusted analytical access, automate transformation and quality controls, orchestrate and monitor end-to-end execution, and secure core service interactions across a multi-node environment?

I.3.2 Motivation for a Distributed Lakehouse Approach

A distributed lakehouse architecture was selected because the project combines heterogeneous ingestion, replayable raw persistence, distributed processing, metadata governance, SQL access, controlled transformations, and data-quality verification. The lakehouse model preserves the flexibility of raw distributed storage while providing managed analytical tables that can be shared across processing and query engines.

The distributed deployment also makes service responsibilities and node collaboration explicit. Storage, computation, querying, orchestration, validation, observability, and security remain separate concerns but operate through defined interfaces. This architecture therefore addresses the engineering problem as an integrated data platform rather than as a collection of isolated scripts.

I.3.3 Project Identity and Academic Positioning

The implemented system is best characterized as a **secure distributed customer-data engineering platform**. Its academic positioning combines several complementary dimensions:

- a Data Engineering platform for reproducible customer-data pipelines;
- a Big Data architecture using distributed storage, processing, and querying;
- a lakehouse implementation joining replayable raw data with governed analytical tables;
- a secure multi-node deployment with orchestration, quality, and observability controls;
- a technical foundation for future Customer 360 and AI-oriented services.

This identity reflects the system that was implemented and validated; it does not position the project as a finished machine-learning product, a business-intelligence dashboard, or a simple ETL script collection.

I.3.4 Project Objectives

The project was structured around the following objectives:

1. organize heterogeneous customer datasets and establish replayable ingestion through Kafka and an HDFS bronze layer;
2. process raw data with Spark into Iceberg tables backed by Parquet and governed through Hive Metastore;
3. expose analytical access through Trino, structure transformations with dbt-spark, and validate data quality with Great Expectations;
4. orchestrate the complete execution sequence through Airflow and centralize operational visibility through Prometheus, Grafana, and pipeline evidence;
5. deploy cooperating services across a local control plane and three Linux virtual machines;
6. protect the distributed runtime through baseline controls for service identity, browser access, transport, remote execution, and persisted data.

I.3.5 Implemented Scope and Boundaries

The implemented scope covers the active Client 1 bundle composed of the *Bank Marketing*, *Online Shoppers Purchasing Intention*, and *Online Retail II* datasets. Together, these sources represent campaign and profile context, digital journey and purchase-intent behavior, and transactional retail activity. This combination provides a credible multi-source basis for future Customer 360 analysis while remaining compatible with the available infrastructure.

The project does not implement downstream machine-learning modules such as dynamic segmentation, churn or lifetime-value prediction, persona generation, and recommendation engines. It delivers the distributed and secured data foundation required for those capabilities, thereby keeping the report focused on the completed engineering contribution.

I.3.6 Implemented Outcomes and Academic Contribution

The completed outcome is a reproducible end-to-end platform in which heterogeneous customer datasets are ingested, persisted, processed, transformed, queried, validated, orchestrated, monitored, and secured. It provides replayable bronze artifacts in distributed storage, governed analytical tables accessible through SQL, persisted quality evidence, repeatable workflow execution, and observable collaboration across multiple nodes.

The academic contribution lies in integrating event transport, distributed storage and computation, lakehouse table management, metadata governance, analytical access, transformation control, quality assurance, orchestration, observability, deployment, and security hardening into one consistent customer-data platform. Its originality is architectural and engineering-oriented: the work demonstrates how these concerns can be combined and operated coherently within a constrained multi-node environment.

I.4 Conclusion

This chapter established the organizational, business, and academic framing of AdOptimizer CDP. It clarified the role of SMART AUTOMATION TECHNOLOGIES, distinguished the project from the broader AdOptimizer AI vision, defined the engineering problem, and stated the implemented objectives, scope, boundaries, and contribution. Chapter II now develops the theoretical foundations that justify the distributed lakehouse architecture and its principal design choices.

Chapter II

State of the Art and Theoretical Background

II.1 Introduction

This chapter establishes the theoretical foundation required to explain the architecture of AdOptimizer CDP rigorously. The implemented platform is not merely a combination of tools; it reflects architectural choices shaped by established constraints in distributed data engineering. The discussion therefore begins with general concepts and trade-offs before moving toward analytical architecture, governance, observability, and security.

The discussion draws on the broader literature on distributed and data-intensive systems, including the design logic emphasized by Martin Kleppmann in *Designing Data-Intensive Applications*, especially around reliability, scalability, maintainability, partitioning, replication, and consistency trade-offs [?]. Its purpose is not to survey the entire field, but to retain the concepts that directly explain the design decisions made in the project.

II.2 Foundations of Modern Distributed Data Systems

II.2.1 From Centralized Systems to Distributed Data Platforms

Traditional information systems were largely organized around centralized databases and a relatively small number of application servers. This model remains effective when data volume is moderate, operational contexts are stable, and the number of consumers remains limited. However, once organizations begin collecting heterogeneous data from multiple

channels, centralized architectures become difficult to sustain. Storage pressure increases, computational workloads become more varied, and the system becomes more sensitive to single-node bottlenecks.

The centralized model is simple to deploy, but it concentrates ingestion, storage, processing, metadata, and access on the same execution point. As workloads grow, this concentration creates bottlenecks, single points of failure, and clear scalability limits.

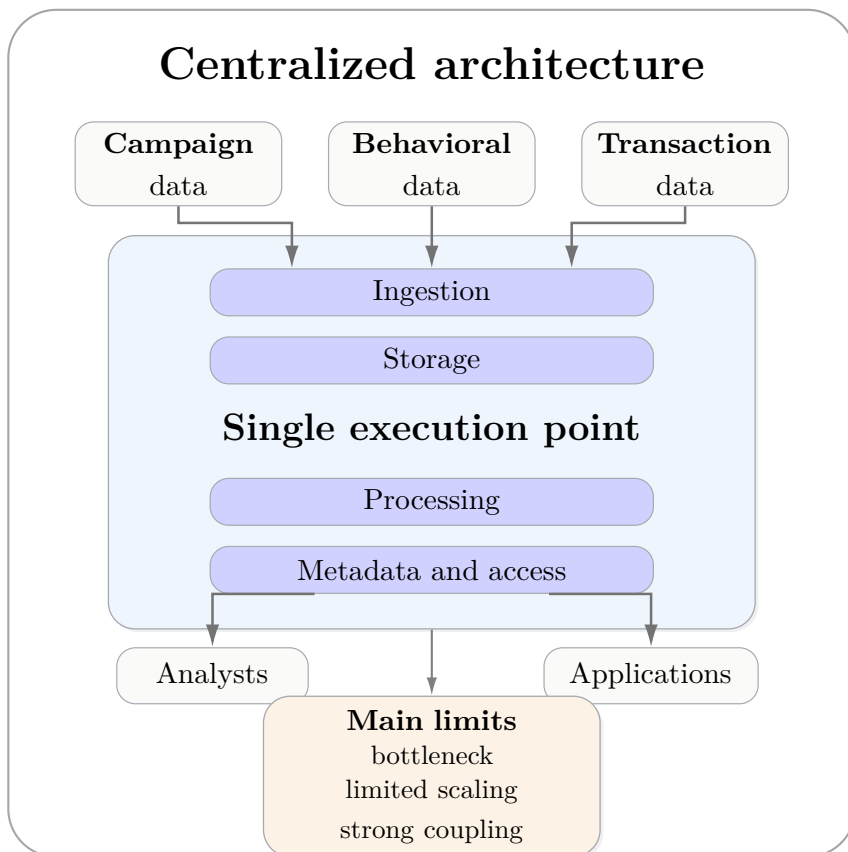


Figure II.1: Centralized execution model and its main architectural limits.

Figure II.1 highlights why centralized designs become fragile under heterogeneous analytical growth: all responsibilities compete on the same machine boundary, and operational evolution is constrained by the limits of that boundary.

Distributed data platforms emerged as a response to these limits. Their role is not only to increase capacity, but also to separate responsibilities such as ingestion, storage, compute, metadata, and analytical access. This decomposition allows each concern to evolve with the technical model best suited to it [?].

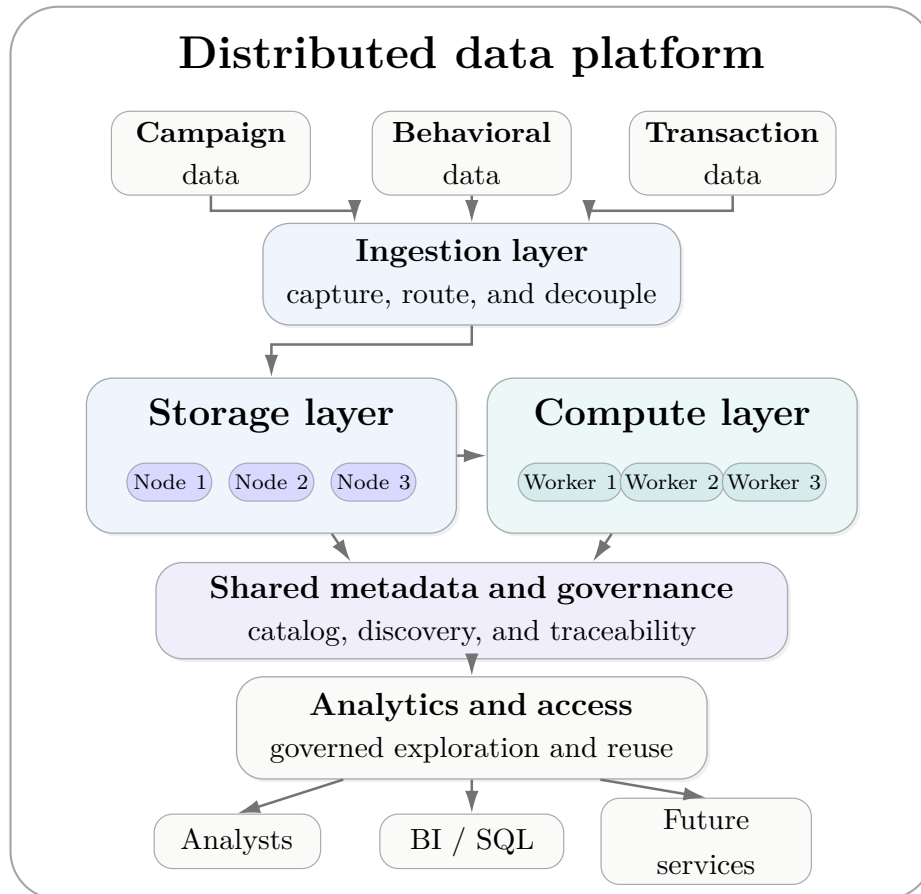


Figure II.2: Layered organization of a distributed data platform.

By contrast, the distributed platform separates responsibilities according to operational role. This supports modular growth, parallel execution, better resilience, and clearer alignment between storage and processing patterns. Figure II.2 therefore prepares the discussion on horizontal scaling, reliability, and maintainability.

Criterion	Centralized architecture	Distributed data platform
Node organization	Core responsibilities remain concentrated on one primary execution point.	Responsibilities are distributed across several coordinated layers or nodes.
Scaling model	Capacity growth depends mainly on strengthening one machine.	Capacity growth is achieved by adding or specializing nodes and services.
Failure exposure	One overloaded or failed node can degrade the whole platform quickly.	Failure impact is more localized when replication and layered separation are used.
Data-processing logic	Ingestion, storage, processing, and access remain tightly coupled.	Ingestion, storage, compute, metadata, and access can evolve with different rhythms.
Governance evolution	Metadata and traceability are harder to extend cleanly as workloads diversify.	Governance can be attached explicitly to catalog, storage, and analytical layers.
Long-term suitability	More suitable for moderate and stable workloads.	More suitable for heterogeneous, growing, and multi-consumer analytical workloads.

Table II.1: Comparison between centralized and distributed architectural logic

II.2.2 Vertical Scaling vs Horizontal Scaling

One of the first decisions in large-scale systems concerns how growth should be absorbed. Vertical scaling strengthens a machine by increasing CPU, memory, or storage capacity. It is often simpler in the short term because the system does not need to be partitioned immediately. Nevertheless, it has two major limits: physical growth is finite, and dependence on a small number of powerful machines weakens resilience.

Horizontal scaling increases capacity by adding nodes. It is more complex because it requires workload distribution, coordination, and failure management. However, it offers better long-term elasticity and better alignment with modern data engineering workloads. For a distributed customer-data platform, horizontal scaling is therefore a structural architectural principle rather than a simple performance technique [?].

A simplified capacity model is:

$$C_{\text{total}}(n) \approx n \times C_{\text{node}} \times \eta(n), \quad 0 < \eta(n) \leq 1$$

where n is the number of participating nodes, C_{node} is the average capacity of one node, and $\eta(n)$ is an efficiency factor accounting for coordination, serialization, network transfer, and workload imbalance. Horizontal capacity therefore rarely grows perfectly linearly, but

additional nodes can still increase total capacity when the workload is partitionable.

II.2.3 Reliability, Scalability, and Maintainability

The literature on distributed systems frequently emphasizes three central qualities: reliability, scalability, and maintainability [?]. Reliability refers to the system’s ability to continue providing correct service despite component failures, abnormal workloads, or operational disturbances. Scalability refers to the platform’s capacity to grow without disproportionate degradation or redesign. Maintainability refers to the ability to understand, operate, extend, and debug the system over time.

These qualities are fundamental for a platform such as AdOptimizer CDP. An unreliable platform cannot support trusted analytical reuse. A non-scalable platform cannot absorb new datasets or future use cases. A non-maintainable platform becomes fragile and difficult to operate. These qualities are therefore direct evaluation criteria for the engineering relevance of the project.

II.2.4 Data-Intensive Systems as a Distinct Engineering Problem

Data-intensive systems differ from ordinary software systems because data movement, storage, replayability, and consistency become first-class engineering concerns. In such systems, correctness is not defined only by the logic of business functions. It also depends on whether data is durably persisted, whether transformations are reproducible, whether schemas remain interpretable over time, and whether the system can explain the origin and lifecycle of the information it exposes.

This distinction is important for the positioning of the present work. AdOptimizer CDP is not simply an application with a database behind it. It is a distributed data engineering platform whose value lies in organizing the lifecycle of heterogeneous customer-related data in a reliable, observable, and governable way.

II.3 Core Theoretical Trade-offs in Distributed Data Engineering

II.3.1 The CAP Theorem and Consistency Trade-offs

The CAP theorem formalizes a specific constraint: when a network partition prevents nodes from communicating, a distributed system cannot guarantee both linearizable consistency and availability for every request [?]. Partition tolerance is not a freely selectable option

once communication failures are possible; the practical decision is how each service behaves during such failures.

Different layers of a data platform may therefore adopt different consistency and availability policies. Event transport, distributed storage, metadata services, and analytical table layers are not expected to behave identically. CAP does not prescribe one platform-wide choice; it explains why guarantees must be evaluated per service and per operation.

II.3.2 ACID vs BASE Models

ACID describes transactional guarantees: atomicity, consistency, isolation, and durability. These properties are important when a group of operations must be committed or rejected as one unit and concurrent updates must preserve defined invariants. However, not every stage of a distributed data platform requires the same transactional scope or isolation level.

BASE, commonly summarized as Basically Available, Soft state, and Eventual consistency, is a design philosophy that accepts temporary divergence while replicas converge. ACID and BASE are not symmetric or mutually exclusive alternatives: one characterizes transaction guarantees, while the other describes a broader availability-oriented operating model. A modern data platform may combine eventual convergence in one service with ACID transactions in another, according to the responsibilities of each layer [?].

II.3.3 Data Partitioning and Replication

Partitioning divides data into smaller units so that storage and computation can be distributed across multiple nodes. It is one of the foundational mechanisms behind modern large-scale platforms. Without partitioning, a single node becomes a natural bottleneck for read throughput, write throughput, fault recovery, and transformation execution.

Replication complements partitioning by improving resilience and availability. When several copies of data or logs exist, the system can continue operating even if one node is unavailable. Yet replication also increases coordination and storage cost, which is why it must be treated as an architectural decision rather than a simple implementation detail.

The storage cost of replication can be summarized as:

$$S_{\text{effective}} \approx r \times S_{\text{raw}}$$

where S_{raw} is the original data volume and r is the replication factor. This relation shows why replication improves resilience while also increasing storage requirements.

II.3.4 Fault Tolerance and Service Resilience

In distributed systems, failure must be considered normal rather than exceptional. Nodes may disappear, ports may become temporarily unavailable, certificates may expire, services may start in the wrong order, and storage volumes may create cascading readiness issues. A resilient platform therefore needs explicit strategies for retries, health checks, replication, recovery, and replay.

This principle is especially relevant in data engineering because failures affect not only uptime but also trust in data products. A silent ingestion failure may create missing data, while a metadata failure may make tables unreadable. Fault tolerance is therefore directly tied to platform credibility and reproducibility.

II.3.5 Batch Processing vs Stream Processing

Batch processing is based on finite datasets processed periodically or on demand. It is suitable for historical backfills, curated analytical transformations, and reproducible large-scale recomputation. Stream processing is based on continuous flows of records or events that are handled incrementally as they arrive. It is useful when ingestion or propagation must remain closer to real-time behavior.

These two paradigms should not be treated as mutually exclusive. In modern platforms, they often coexist: streaming captures raw events continuously, while downstream computation structures them into trusted analytical tables through batch or micro-batch logic [?, ?].

At a very general level, processing intensity may be represented by:

$$\text{Throughput} = \frac{\text{Number of records processed}}{\text{Processing time}}$$

This expression is useful because it links theoretical processing models to an operational question: how much data can the platform absorb and transform within a given interval.

II.3.6 Schema-on-Write vs Schema-on-Read

Schema-on-write means that data must be shaped according to a predefined structure before it is stored in an authoritative analytical repository. This approach improves control and downstream semantic stability, but it may reduce ingestion flexibility. Schema-on-read, by contrast, stores raw or semi-structured data first and interprets structure later when the data is processed or queried.

This trade-off is central to modern lakehouse thinking. Raw layers benefit from schema-

on-read because they preserve source fidelity and replayability. Curated layers benefit from stronger schema discipline because users require stable analytical entities. A layered platform therefore combines both strategies rather than choosing only one.

II.4 Big Data Foundations

II.4.1 The Five Vs of Big Data

Big Data was classically characterized through volume, velocity, and variety [?]. Contemporary discussions commonly extend this model with veracity and value. Volume refers to the quantity of data under management; velocity to the rate at which it is generated and moved; variety to differences in format, structure, and semantics; veracity to reliability and quality; and value to its usefulness for decisions and downstream services.

These dimensions show that data-engineering complexity does not depend on volume alone. In AdOptimizer CDP, variety, veracity, replayability, and reuse pressure are as important as raw scale, while distribution supports role separation, horizontal evolution, resilience, and explicit multi-node execution.

II.4.2 Heterogeneous Data Sources and Data Variety

Customer understanding rarely comes from a single operational system. It may involve campaign response datasets, online behavioral signals, e-commerce or retail transactions, session-level intent indicators, and profile-related attributes. Each source differs in format, cadence, completeness, and downstream meaning. This heterogeneity makes integration difficult because technical normalization is not sufficient; semantic reconciliation is also required.

As a result, variety becomes both a storage problem and a modeling problem. The platform must ingest different raw forms, persist them durably, and later organize them into analytical layers without erasing provenance.

II.4.3 Distributed Storage Principles

Distributed storage spreads persisted data across multiple nodes instead of concentrating it on a single machine. Its goals include capacity growth, resilience, replication, parallel read/write support, and better alignment with distributed compute. An important consequence of distributed storage is that it separates durable raw persistence from the compute jobs that later transform the data.

This separation has strong methodological value. When raw data is durably preserved, pipelines become replayable and easier to audit. Transformations can be rerun without repeatedly depending on the original upstream source.

II.4.4 Distributed Compute Principles

Distributed compute allows processing tasks to be decomposed and executed across several nodes. Instead of one machine reading and transforming an entire dataset, work is split according to partitions, files, or execution stages. This can reduce total execution time and align compute more closely with distributed storage [?].

However, distributed compute is beneficial only when the workload justifies it. Scheduling, serialization, dependency management, network transfer, and cluster readiness introduce overhead. Parallel execution is therefore most useful when the work can be partitioned effectively and its computational or data-movement cost exceeds the coordination overhead.

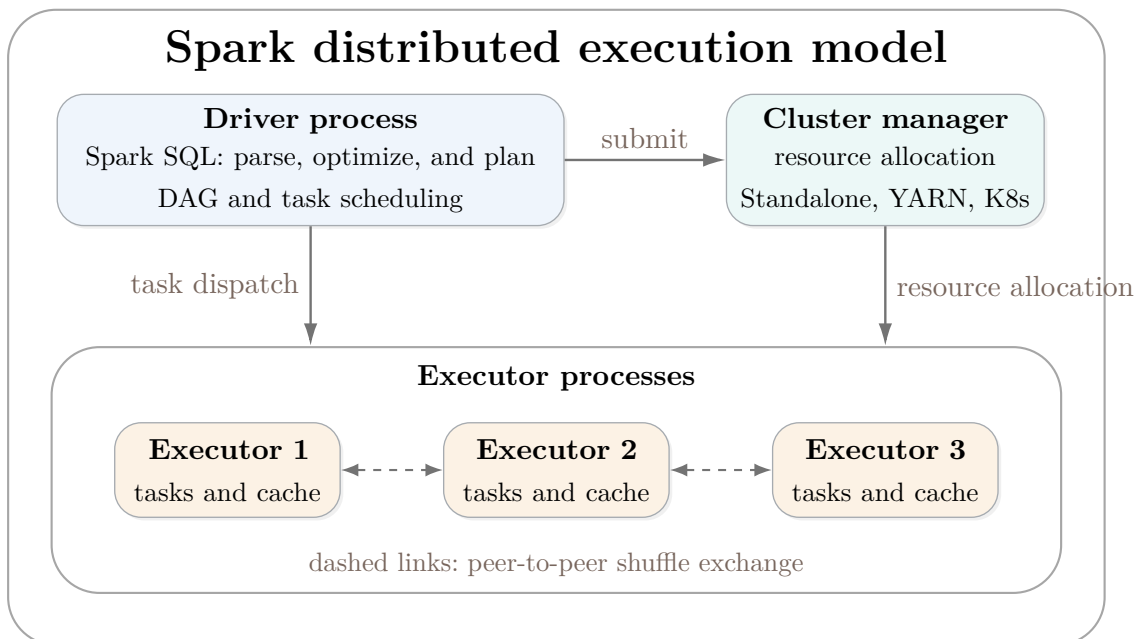


Figure II.3: Spark planning, resource allocation, task execution, and shuffle flow.

Figure II.3 illustrates the operational logic behind this principle: a driver coordinates execution, a cluster manager allocates resources, executors process partitions in parallel, and the SQL engine materializes the physical plan that structures distributed analytical work.

II.5 Evolution of Analytical Data Architectures

II.5.1 Traditional Data Warehouse Architecture

The traditional data warehouse is organized around structured, curated, and strongly modeled analytical data. It is highly effective for stable reporting, dimensional modeling, and SQL-based decision support. Its main strength is semantic control.

Its limitation appears when heterogeneous or semi-structured data must be ingested rapidly and preserved before full normalization. In such contexts, the warehouse-only model can become too rigid because it expects analytical structure too early in the lifecycle.

II.5.2 Data Lake Architecture

The data lake emerged as a response to the rigidity of traditional warehouses. It enables large quantities of raw and semi-structured data to be stored before full modeling. This increases ingestion flexibility and supports exploratory reuse across different downstream consumers.

However, data lakes also introduced a risk: if metadata, naming discipline, and validation remain weak, the platform can degrade into an opaque storage zone with low analytical trust. The lake model solves flexibility problems but can create governance problems.

II.5.3 Data Lakehouse Architecture

The data lakehouse seeks to combine the flexibility of the lake with the transactional and governance discipline associated with warehouses. It preserves distributed and replayable storage logic while adding stronger table semantics, better schema evolution control, and improved analytical exposure [?].

This model is particularly well adapted to projects such as AdOptimizer CDP. It allows raw datasets to be preserved in distributed storage while curated datasets are exposed through governed tables suitable for transformation frameworks and SQL query engines.

II.5.4 Lambda Architecture

Lambda architecture organizes processing through two paths: a batch path for full historical recomputation and a speed layer for low-latency updates. Its objective is to reconcile correctness and freshness. The model was influential because it formalized the difficulty of satisfying both needs through a single pipeline.

Nevertheless, Lambda architecture is often criticized for operational duplication. Two processing logics must be maintained, monitored, and reconciled. Its academic relevance remains high because it helps explain why later architectures searched for simpler ways to preserve freshness without multiplying engineering burden.

II.5.5 Kappa Architecture

Kappa architecture reduces the conceptual duplication of Lambda by emphasizing a stream-centered approach in which the event log becomes the main source of truth. Reprocessing is achieved through replay rather than through a separate batch computation layer [?].

This model is attractive when event logs are rich and replay can serve as the main recomputation mechanism. However, many analytical platforms still require clearly governed historical layers and structured table abstractions. Kappa is therefore instructive, but not automatically sufficient for every customer-data use case.

II.5.6 Comparative Analysis of These Architectures

The historical movement from warehouses to lakes, from lakes to lakehouses, and from isolated reporting toward event-aware architectures reflects a broader shift in priorities. Early architectures privileged control and stable relational semantics. Later architectures privileged ingestion flexibility and scale. Contemporary platforms increasingly seek a hybrid answer that preserves raw fidelity, enables distributed processing, and still produces governed analytical products.

For AdOptimizer CDP, the lakehouse model is the most suitable. It supports raw bronze persistence, distributed storage, structured analytical tables, metadata-driven reuse, and downstream SQL exploration. The selected architecture is therefore a reasoned synthesis of the state of the art.

The implemented project does not reproduce either Lambda or Kappa architecture in a pure textbook form. It does not maintain a separate low-latency speed layer alongside a batch layer, as a strict Lambda design would require. Nor does it treat a streaming computation path as the sole mechanism for producing analytical state, as a strict Kappa design would suggest. Instead, it combines replayable event transport with durable raw history and batch-oriented refinement into governed analytical tables. The resulting model is therefore best described as an event-aware, batch-oriented lakehouse rather than as a complete Lambda or Kappa implementation [?, ?, ?].

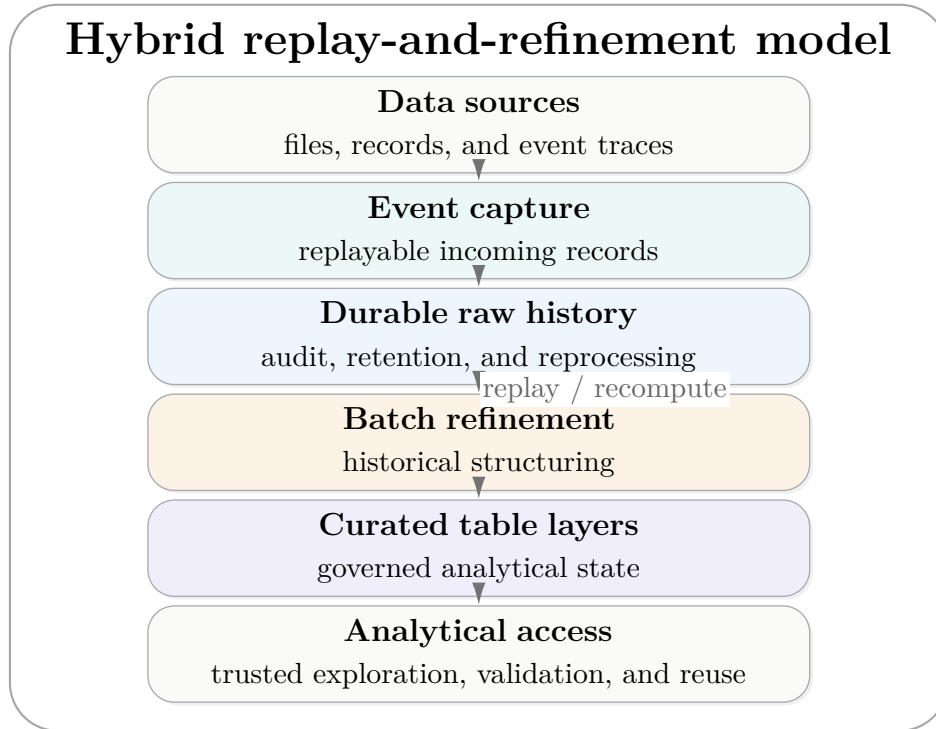


Figure II.4: Hybrid flow from replayable raw history to governed analytical tables.

II.6 Foundational Concepts for Governance and Trust in Data Platforms

II.6.1 Metadata, Cataloging, and Data Discovery

Metadata gives meaning to stored data by describing structure, origin, semantics, and technical location. Without metadata, even accessible data can remain practically unusable. Cataloging extends this principle by making datasets discoverable and interpretable by different actors [?].

In a distributed customer-data platform, metadata is especially important because the architecture spans several services and layers. Governance cannot depend on informal memory alone. Schema registration, cataloging, and dataset discovery must therefore be explicit platform capabilities.

II.6.2 Data Lineage and Traceability

Lineage refers to the ability to trace how data moved from source to output, which transformations were applied, and which downstream assets depend on which upstream artifacts. Traceability is essential for reproducibility, debugging, and auditability.

Lineage distinguishes an integrated data engineering platform from a collection of disconnected scripts. When dependencies and transformations can be reconstructed, the platform becomes more explainable, auditable, and maintainable.

II.6.3 Data Quality Dimensions

Data quality is multidimensional. Common dimensions include completeness, consistency, validity, accuracy, timeliness, and uniqueness. These dimensions are especially important in customer-data contexts because downstream segmentation, profiling, or future predictive analytics become weak when source data is inconsistent or poorly governed.

Modern platforms increasingly formalize quality as an explicit layer of the architecture rather than as a purely manual cleaning activity. Checks, expectations, thresholds, and validation artifacts transform quality into a monitored engineering property.

For example, a simple completeness metric can be written as:

$$\text{Completeness} = \frac{\text{Observed non-missing required values}}{\text{Expected required values}}$$

This ratio intentionally measures presence rather than correctness: validity and accuracy require separate rules. Even a basic completeness measure is useful because it converts a qualitative concern into a control signal that can be monitored over time.

II.6.4 Observability in Distributed Data Systems

Observability is the capacity to infer the internal state of a system through emitted evidence such as metrics, logs, traces, health checks, and runtime indicators. In distributed environments, observability is crucial because failures often emerge from the interaction between components rather than from one isolated service [?].

For data engineering, observability is tightly linked to platform trust. If the system cannot show whether ingestion succeeded, whether storage is healthy, whether services are reachable, or whether transformations produced expected results, operators cannot diagnose failures or establish confidence in the resulting data products.

II.7 Security Foundations for Distributed Data Platforms

II.7.1 Authentication in Distributed Systems

Authentication establishes who a user or service is. In distributed environments, it becomes critical because communication crosses process, host, and network boundaries. Storage services, metadata layers, and query endpoints must be able to distinguish legitimate principals from unauthorized ones [?].

Without strong authentication, a distributed platform depends on implicit trust between components. Authentication is therefore a structural foundation of secure distributed data engineering.

II.7.2 Authorization and Access Control

Once identity is established, the system must determine which actions are permitted. Authorization controls access to services, datasets, administrative operations, and query interfaces. It operationalizes the principle of least privilege by restricting each actor to the minimum access necessary for its role.

In data platforms, authorization cannot be limited to end-user interfaces alone. It also applies to service-to-service calls, operational scripts, metadata access, and internal gateways.

II.7.3 Encryption in Transit and Encryption at Rest

Encryption in transit protects data while it moves between nodes, services, or users. It reduces exposure to interception and helps preserve confidentiality across the network. Encryption at rest protects persisted data on storage media and reduces exposure if storage devices or mounted runtime paths are accessed outside intended controls [?].

These mechanisms address different threat surfaces and are complementary. In a multi-service lakehouse platform, transport protection reduces network exposure, while at-rest encryption protects persisted runtime volumes when storage is accessed outside the intended execution path.

II.7.4 Secure Service Exposure in Multi-Node Environments

Distributed platforms expose numerous service endpoints: orchestration UIs, query interfaces, storage consoles, monitoring dashboards, and service APIs. If these are exposed without proxying, credential checks, and transport protection, the system accumulates a wide attack surface [?, ?].

Secure exposure therefore requires deliberate control of entry points, certificates, internal-versus-external interfaces, and service-access boundaries.

II.8 Positioning of the Present Work

II.8.1 Alignment with Distributed Lakehouse Principles

AdOptimizer CDP aligns with the theoretical principles developed in this chapter through explicit separation of ingestion, durable raw persistence, distributed computation, table management, metadata, analytical access, transformation, validation, orchestration, and observability. Replayable source history supports recovery and reproducibility, while governed analytical tables introduce schema discipline and shared semantics above the raw layer.

The architecture also applies distribution selectively rather than treating it as an end in itself. Storage, processing, and querying use coordinated multi-node execution, whereas control-plane responsibilities remain centralized where centralized coordination improves operational clarity. This balance reflects the principle that distribution should follow service responsibilities, scalability needs, and failure boundaries.

II.8.2 Data Engineering Foundation, Not a Final AI Product

The theoretical analysis confirms that the project’s primary contribution lies in data lifecycle engineering. Reliability, replayability, metadata, lineage, quality controls, observability, and secure access are prerequisites for trustworthy downstream analytics. Predictive models cannot compensate for an unreliable or poorly governed data foundation.

Consequently, the absence of final segmentation, churn, lifetime-value, or recommendation models is a deliberate scope boundary rather than an incomplete implementation. AdOptimizer CDP provides the governed data products and operational platform on which such capabilities can later be developed.

II.8.3 Position Within AdOptimizer AI

Within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES, AdOptimizer CDP occupies the customer-data foundation layer. The broader initiative defines the future intelligent marketing direction, while the PFE implements the distributed lakehouse platform required to make that direction technically credible. This relationship preserves a clear distinction between the completed Data Engineering contribution and its future analytical and AI consumers.

II.9 Conclusion

This chapter established the theoretical and architectural background that justifies the design of AdOptimizer CDP. It introduced the evolution of distributed data systems, the principal engineering trade-offs that shape them, the foundations of Big Data platforms, the comparative logic of modern analytical architectures, and the principles of governance, observability, and security in distributed environments.

On this basis, the following chapters move from theory to technology selection, requirements, design, implementation, deployment, security, and validation. The platform is therefore presented not as a collection of tools, but as a coherent response to well-identified distributed data engineering challenges.

Chapter III

Technology Watch

III.1 Introduction

Chapter II established the conceptual foundations of distributed data platforms. This chapter moves from those foundations to the technological choices implemented in AdOptimizer CDP. Its purpose is not to reproduce product documentation or to present popular tools in isolation. It explains how the retained stack was evaluated against the project's actual constraints, which alternatives were considered, and why the selected technologies form a coherent platform.

The assessment is retrospective and implementation-based. The resulting platform is already deployed as a local control plane connected to a distributed data plane composed of three Linux virtual machines. Consequently, the selected technologies are evaluated according to their demonstrated contribution to ingestion, distributed storage, processing, lakehouse governance, analytical access, transformation, quality, orchestration, observability, deployment reproducibility, and security.

III.2 Project Constraints and Selection Criteria

The technology choices were guided by the implemented environment rather than by an abstract enterprise-scale target. The main constraints were the following:

- **Distributed execution:** the core storage, processing, ingestion, and query services had to demonstrate real cooperation across VM1, VM2, and VM3.
- **Limited infrastructure:** the deployment had to remain operable on three resource-constrained virtual machines without dedicated accelerators.

- **Interoperability:** processing, table management, metadata, and query technologies had to exchange data through compatible interfaces rather than through isolated proprietary paths.
- **Replayability and reproducibility:** ingestion and transformation stages had to support controlled re-execution, explicit dependencies, and repeatable deployment.
- **Operational evidence:** the platform had to expose observable service state, workflow history, validation results, and distributed-execution evidence.
- **Security integration:** identity, browser access, remote submission, transport, and runtime persistence had to be protected as part of the deployed architecture.
- **Academic clarity:** each technology had to have a precise architectural responsibility that could be explained and defended without hiding the system behind a managed platform.

These constraints led to five principal evaluation criteria: functional suitability, distributed compatibility, cross-tool integration, operational reproducibility, and feasibility within the available infrastructure. Community maturity and documentation quality were considered supporting factors, but they were not treated as substitutes for technical fit.

III.3 Comparative Assessment

The alternatives in Table III.1 were not rejected as universally inferior technologies. They were assessed against the narrower needs of the implemented AdOptimizer CDP platform. The decisive question was therefore not which technology is best in every context, but which combination best supports the required architecture with the least conceptual and operational divergence.

Layer	Alternatives considered	Retained choice	Decisive project rationale
Ingestion	RabbitMQ streams; Apache Pulsar	Apache Kafka	Partitioned topics, controlled replay, and a direct fit with the implemented three-broker ingestion design.
Bronze storage	Local/shared files; object storage	Hadoop HDFS	Explicit NameNode/DataNode behavior, block distribution, and replication provide visible proof of distributed raw persistence.
Table and metadata management	Delta Lake; Apache Hudi	Apache Iceberg, Hive Metastore, and Parquet	The combination gives Spark and Trino a shared open-table and catalog path while preserving efficient columnar files.
Distributed processing	Apache Flink; local Python processing	Apache Spark	The project is dominated by replayable batch loading and SQL-oriented transformations, matching Spark’s cluster execution and ecosystem integration.
Analytical query	Spark SQL alone; Hive-oriented querying	Trino	A separate coordinator/worker query plane avoids coupling interactive analytical access to transformation jobs.
Transformation and quality	Custom SQL and Python validation scripts	dbt-spark and Great Expectations	Declared model dependencies and persisted validation artifacts make transformation and quality controls explicit and auditable.
Orchestration	Shell scheduling; Prefect; Dagster	Apache Airflow	DAG dependencies, retries, scheduling, and run history match the implemented five-workflow operational sequence.
Observability and deployment	Service-specific logs; manual processes	Prometheus, Grafana, cAdvisor, and Docker	Central metrics and dashboards expose distributed behavior, while containerization keeps service packaging repeatable.
Security baseline	Application-only credentials; unprotected administrative endpoints	Kerberos, SPNEGO, TLS gateways, SSH, and LUKS	Layered controls protect service identity, browser access, remote execution, transport, and persisted runtime data.

Table III.1: Comparative assessment of the principal technology choices

III.3.1 Event Ingestion and Distributed Bronze Storage

Kafka was retained as the ingestion backbone because its partitioned-topic model maps directly to the project’s dataset-specific transport and three-broker topology [?]. RabbitMQ provides mature queue and stream abstractions, while Pulsar provides distributed messaging with multiple subscription models [?, ?]. Both are credible alternatives, but adopting either would have introduced a different operational model without improving the required Kafka-to-HDFS replay path already implemented in the platform.

HDFS was selected for the bronze landing zone because its NameNode/DataNode architecture and configurable block replication make distributed persistence directly observable across the three VMs [?]. Local or shared folders would not demonstrate distributed storage, while object storage would provide a different access abstraction and would weaken the intended Hadoop-centered deployment demonstration. HDFS therefore serves both an engineering purpose—replayable raw persistence—and an architectural purpose by proving that bronze data is physically managed by a distributed file system.

III.3.2 Lakehouse Tables, Metadata, and File Format

Apache Iceberg manages the structured raw and curated lakehouse tables, Hive Metastore centralizes their discoverable metadata, and Parquet provides the underlying columnar file representation [?, ?, ?]. This separation is important: Parquet defines how analytical data is stored in files, Iceberg adds table-level metadata and evolution capabilities, and Hive Metastore exposes shared catalog definitions to compatible engines.

Delta Lake and Apache Hudi were considered as alternative lakehouse table technologies [?, ?]. Iceberg was retained because the implemented project required a neutral table layer shared by Spark and Trino, with explicit schema and partition evolution rather than a platform-specific managed environment. The choice also aligns with the report’s objective of making the relationship between physical files, table metadata, and the external catalog academically visible.

III.3.3 Distributed Processing, Querying, and Transformation

Spark is the main distributed compute engine for bronze-to-raw loading and curated transformation execution. Its standalone master/worker model maps naturally to VM2 as the coordination node and VM1–VM3 as executor-bearing workers [?]. Apache Flink offers strong stateful processing for bounded and unbounded streams [?], but the implemented workloads are primarily controlled batch and SQL transformations executed through reproducible Airflow workflows. Spark therefore provides the required distribution without

introducing a second processing model that the project does not need.

Trino provides a separate distributed SQL access layer over the Iceberg and Hive-governed lakehouse [?]. This separation prevents analytical exploration from being represented merely as another transformation job and gives the deployment an explicit coordinator/worker query architecture. dbt-spark then organizes staging, intermediate, and analytics transformations as declared, dependency-aware models executed against the Spark environment [?]. Using dbt-spark rather than unstructured SQL scripts makes transformation order, model ownership, and generated execution artifacts easier to inspect and reproduce.

III.3.4 Quality, Orchestration, and Observability

Great Expectations turns data quality into an explicit validation layer by applying documented expectations and persisting validation results and Data Docs evidence [?]. Custom validation scripts could test individual rules, but they would require the project to recreate result storage, structured suites, and human-readable evidence. The retained tool therefore supports both operational checking and the report's need for verifiable quality artifacts.

Airflow orchestrates the implemented sequence of environment reset, lakehouse setup, service readiness, the integrated Kafka-HDFS-Spark raw-loading and raw-quality stage, and dbt-spark transformation [?]. Prefect and Dagster provide credible Python-oriented and asset-oriented orchestration models [?, ?]; however, Airflow's explicit DAG representation, scheduling, retries, task logs, and run history already align with the project's five-workflow execution contract.

Prometheus collects platform and exporter metrics, Grafana presents them as operational dashboards, and cAdvisor contributes container-level visibility from the VM nodes [?, ?, ?]. This arrangement is more defensible than relying on isolated service logs because it provides a centralized view of infrastructure health and pipeline behavior across the control and data planes.

III.3.5 Deployment Reproducibility and Security Support

Docker packages and isolates the services deployed on the control plane and VM nodes [?]. Containerization does not make the platform distributed by itself; rather, it makes the already distributed service topology repeatable by preserving explicit service definitions, configuration boundaries, and startup dependencies.

The completed platform also includes Kerberos-secured Hadoop identity, SPNEGO-protected HDFS browser access, TLS gateways, SSH-based Spark submission, and en-

encrypted runtime storage [?, ?, ?]. Their configuration and trust boundaries are analyzed in Chapter VII.

III.4 Synthesis of the Final Stack

Table III.2 consolidates the retained technologies by architectural responsibility and summarizes their concrete utility within the implemented platform.

Architectural layer	Retained technology	Utility in AdOptimizer CDP
Ingestion	Apache Kafka	Replayable dataset-specific ingestion through a three-broker cluster.
Bronze storage	HDFS	Distributed bronze persistence across the NameNode and DataNodes.
Table and metadata management	Iceberg, Parquet, and Hive Metastore	Governed tables, columnar files, and shared catalog metadata.
Distributed processing	Apache Spark	Distributed raw loading and curated transformation execution.
Analytical query	Trino	Coordinator/worker SQL access over the lakehouse tables.
Transformation	dbt-spark	Dependency-aware staging, intermediate, and analytics models.
Data quality	Great Expectations	Quality rules with stored validation results and Data Docs evidence.
Orchestration	Apache Airflow	Five-DAG orchestration, retries, logs, and run history.
Observability	Prometheus, Grafana, cAdvisor, and exporters	Central infrastructure, container, and pipeline monitoring.
Deployment	Docker	Repeatable service isolation, configuration, and deployment.
Security	Kerberos, SPNEGO, TLS, SSH, and LUKS	Identity, access, transport, remote-execution, and data-at-rest protection.

Table III.2: Final technology stack and utility within AdOptimizer CDP

III.5 Conclusion

This technology watch justified the retained AdOptimizer CDP stack against its architectural constraints. Kafka and HDFS provide replayable ingestion and storage; Spark, Iceberg, Parquet, and Hive Metastore form the lakehouse; Trino, dbt-spark, and Great Expectations query, transform, and validate it; and Airflow, observability, Docker, and the security controls operate and protect it. Together, these choices underpin the system design formalized in Chapter IV.

Chapter IV

Requirements Analysis and System Design

IV.1 Introduction

This chapter translates the business framing and technical choices established in the previous chapters into a structured design baseline for AdOptimizer CDP. It identifies what the platform was required to do, the constraints imposed by the deployment environment, the principal actors and consumers, and the way the implemented multi-layer architecture satisfies those needs. Whereas Chapter III justified the technology choices, this chapter formalizes the requirements and architectural organization of the completed solution. The design follows established data-intensive engineering principles: separation of concerns, explicit data-movement boundaries, and layered coordination between storage, compute, metadata, and access services [?, ?].

IV.2 Requirement Elicitation Context

The requirements of AdOptimizer CDP were derived from three complementary sources:

- the business objective of creating a distributed customer-data foundation under the broader AdOptimizer AI vision;
- the engineering need to ingest, store, process, validate, orchestrate, expose, and monitor heterogeneous customer-related datasets;
- the operational reality of a local control plane connected to three resource-constrained Linux virtual machines with integrated security hardening requirements.

The resulting requirements are therefore traceable to implemented capabilities, acceptance criteria, and the final distributed deployment model rather than being presented as abstract intentions.

IV.3 Stakeholders and System Actors

The platform exposes different interaction paths according to operational responsibility. Table IV.1 summarizes the principal roles considered in the design.

Actor or stakeholder	Primary interaction	Expected outcome
Platform administrator / operator	Deploys services, manages configuration and security material, verifies readiness, and investigates runtime failures.	A reproducible, observable, and protected distributed runtime.
Data engineer	Executes and maintains ingestion, processing, transformation, orchestration, and validation workflows.	Governed data products with traceable execution and quality evidence.
Data analyst	Uses authenticated SQL access to inspect governed raw and curated lakehouse tables.	Reliable analytical access without direct dependence on storage internals.
Future data-science and AI consumers	Consume curated customer-data products produced by the platform.	Reusable inputs for future Customer 360 and AI use cases; model development remains outside the PFE scope.

Table IV.1: Principal stakeholders and interaction responsibilities

IV.4 Functional Requirements

The functional scope of the platform is centered on the end-to-end lifecycle of customer-related data, from ingestion to governed analytical access. The most important functional requirements are summarized in Table IV.2 and Table IV.3.

The requirements were organized as a dependency-aware execution contract rather than as an independent list of features. FR-1 to FR-5 define the data foundation: source-aware ingestion, replayable Kafka transport, durable HDFS bronze persistence, distributed Spark loading, and shared metadata governance. FR-6 to FR-10 cover the controlled use and operation of that foundation through analytical querying, layered transformation, explicit quality validation, orchestration, and observability. This grouping reflects the implemented lifecycle because downstream capabilities are meaningful only after the preceding storage, processing, and catalog conditions have been satisfied.

Each acceptance criterion is also expressed through an observable technical result. The report therefore distinguishes between a service being reachable and a platform capability being completed: successful ingestion must leave routed records and bronze artifacts, raw loading must produce registered Iceberg tables, transformation must materialize declared models, and validation must retain inspectable evidence. This evidence-oriented formulation supports the traceability analysis in Appendix ?? and provides a consistent basis for the validation campaign presented in Chapter VIII.

ID	Functional requirement	Technical acceptance criterion
FR-1	Ingest heterogeneous customer-related datasets through a structured pipeline.	The same configured ingestion process accepts the three active Client 1 datasets and preserves their dataset-specific routing.
FR-2	Provide topic-based transport for reproducible raw data movement.	Kafka topics are created, used, and reset through controlled pipeline operations, with visible topic-based message flow.
FR-3	Persist replayable raw data in a distributed bronze landing zone.	HDFS stores bronze artifacts in a durable, distributed structure that can be re-read by downstream Spark processing stages.
FR-4	Load bronze data into governed raw lakehouse tables.	Spark reads the persisted bronze files and writes Iceberg raw tables backed by Parquet and registered through the shared catalog.
FR-5	Centralize analytical metadata for table discovery and reuse.	Hive Metastore exposes the table definitions required for downstream engines to identify and access the lakehouse objects.

Table IV.2: Primary functional requirements of the platform, part 1

ID	Functional requirement	Technical acceptance criterion
FR-6	Provide interactive SQL access for analytical exploration.	Trino executes catalog, schema, row-count, and representative analytical queries over governed lakehouse tables.
FR-7	Support layered transformations from raw to curated analytical outputs.	dbt-spark runs declared transformation models and preserves dependency-aware model execution.
FR-8	Apply lightweight data quality validation and persist validation evidence.	Great Expectations checks structural and semantic rules and records validation artifacts for the controlled datasets.
FR-9	Orchestrate the operational sequence of the data platform.	Airflow executes the five-workflow chain for environment reset, lakehouse setup, service readiness, integrated raw ingestion/loading and quality validation, and dbt transformation, with visible task state and run history.
FR-10	Expose infrastructure and pipeline observability signals.	Prometheus collects service, exporter, and cAdvisor metrics, while Grafana displays infrastructure-readiness and pipeline-oriented indicators.

Table IV.3: Primary functional requirements of the platform, part 2

IV.5 Non-Functional Requirements

Beyond functional behavior, the platform was designed around quality constraints related to scalability, modularity, traceability, security, operational inspectability, and resource feasibility. These requirements are summarized in Table IV.4. Their importance is consistent with the broader literature on reliable and maintainable distributed systems, in which reproducibility, observability, and controlled failure handling are architectural qualities rather than optional conveniences [?].

ID	Non-functional requirement	Design implication
NFR-1	Horizontal evolution of the principal data-plane roles.	Storage, compute, and query responsibilities are distributed so that capacity can be extended without concentrating all workloads on one node.
NFR-2	Modularity and separation of concerns.	Ingestion, storage, processing, metadata, validation, orchestration, monitoring, and security are implemented as distinct layers.
NFR-3	Reproducibility of ingestion and transformation flows.	The platform preserves replayable bronze data, versioned transformation logic, and explicit orchestration sequences.
NFR-4	Observability of service and pipeline behavior.	Health checks, metrics, dashboards, and orchestration logs must make failures visible and diagnosable.
NFR-5	Baseline security for access, movement, and persisted runtime data.	Authentication, protected service exposure, secure remote submission, and encrypted storage are integrated according to the deployed scope.
NFR-6	Operational inspectability and evidence retention.	Metrics, dashboards, task logs, run history, validation results, and execution artifacts make behavior and failures traceable.
NFR-7	Feasibility within constrained infrastructure.	Service placement, startup order, and workload execution must remain compatible with the available three-VM resource envelope.

Table IV.4: Main non-functional requirements of the platform

IV.6 System Architecture Overview

The global architecture consolidates the implemented AdOptimizer CDP platform into a single system-level view. It connects ingestion, distributed storage and processing, lakehouse governance, orchestration, validation, observability, and the secured multi-node deployment before the following sections examine each design dimension separately.

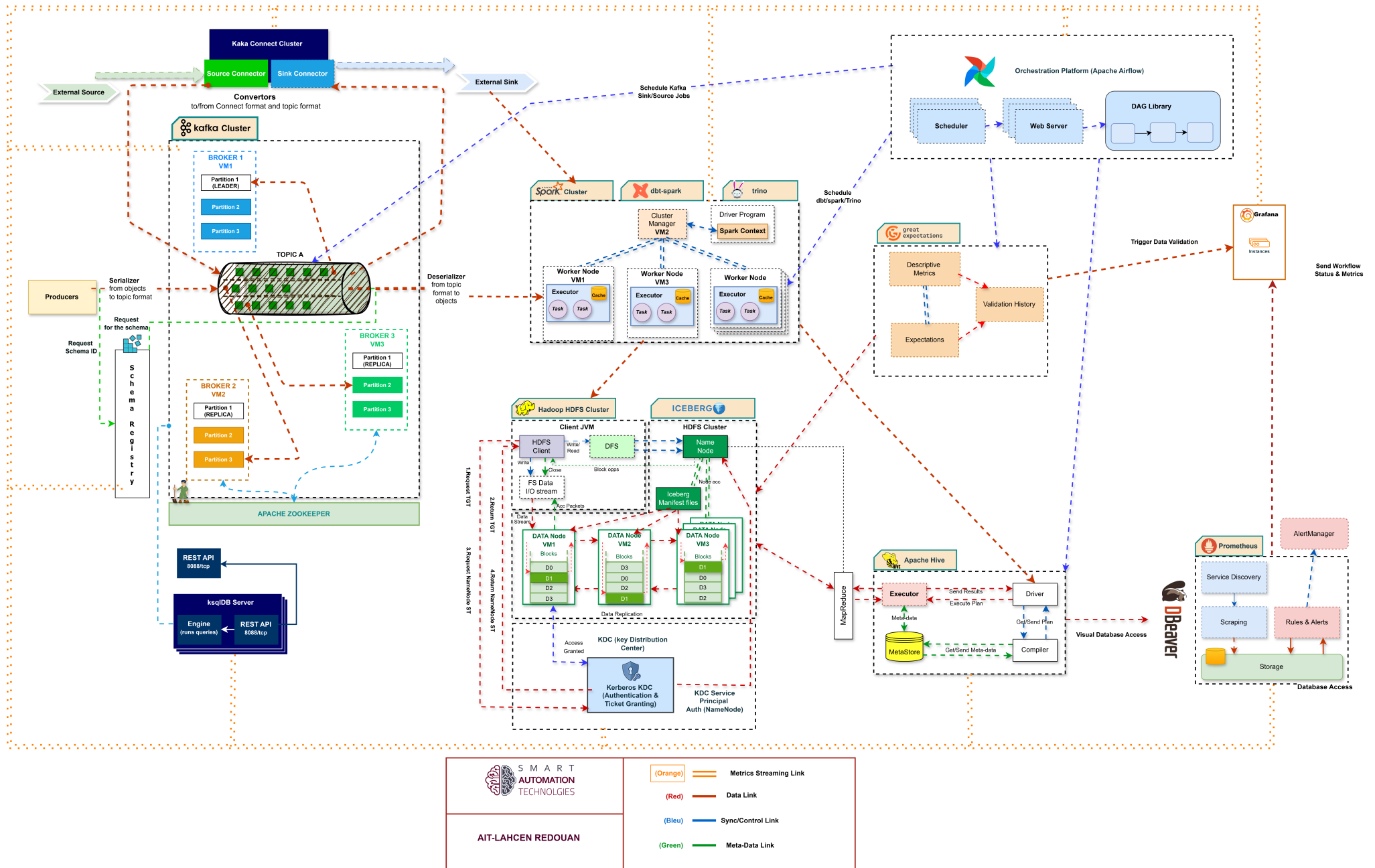


Figure IV.1: Global architecture of the implemented AdOptimizer CDP platform.

IV.7 Logical Layered Architecture

At the logical level, AdOptimizer CDP is organized as a layered customer-data platform rather than as a monolithic data pipeline. This organization is aligned with lakehouse thinking, where raw persistence, structured table layers, metadata governance, and analytical access are coordinated but not collapsed into one opaque runtime block [?]. Each layer has a precise responsibility in the end-to-end flow:

- **Ingestion layer:** receives and transports the raw input datasets through Kafka.
- **Bronze persistence layer:** stores replayable raw artifacts in HDFS.
- **Raw lakehouse layer:** structures the raw persisted inputs through Spark and Iceberg.
- **Metadata layer:** centralizes table definitions through Hive Metastore.
- **Raw-validation layer:** checks the source-specific raw lakehouse tables through Great Expectations and persists evidence.
- **Transformation layer:** applies modeled transformations and tests through dbt-spark.
- **Analytical access layer:** exposes governed raw and curated tables through Trino.
- **Cross-cutting orchestration and observability layer:** coordinates execution through Airflow and supervises the platform through Prometheus and Grafana.

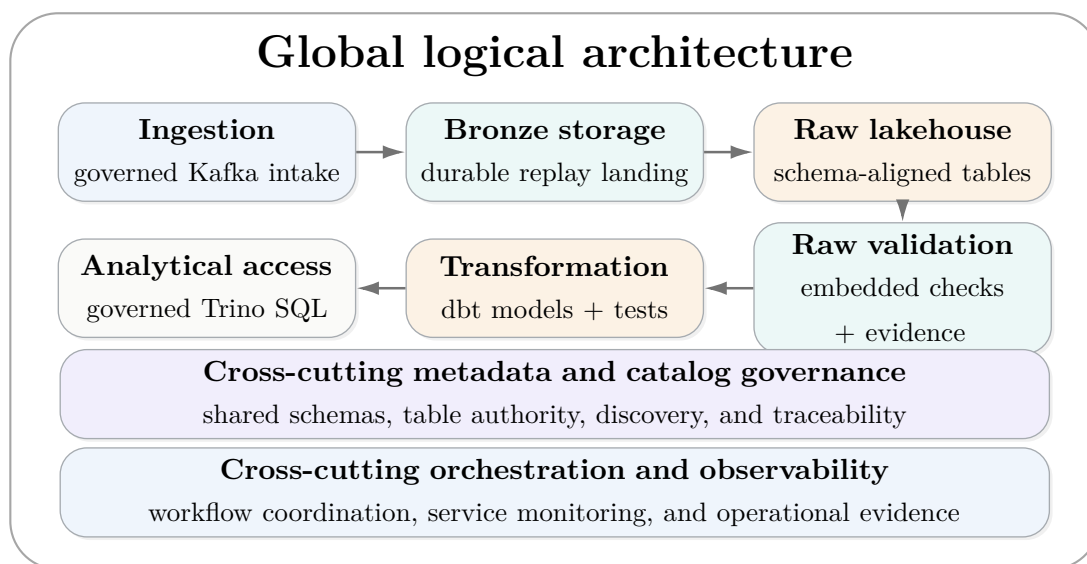


Figure IV.2: Logical data lifecycle and cross-cutting platform controls.

This logical decomposition is important because it explains the system as a coordinated architecture rather than as a sequence of unrelated tools. It also clarifies why the same dataset may pass through different representations before becoming a trusted analytical asset.

IV.8 Physical Multi-Node Architecture

At the physical level, the final design is distributed across one control plane and three data-plane virtual machines. Table IV.5 summarizes this allocation.

Node	IP / Type	VM specifications	Main services	Architectural role
Control plane	Local workstation	Not applicable, because this node is not part of the Linux VM fleet.	Airflow, Prometheus, Grafana, Kafka UI, metrics exporter, local TLS gateway, remote orchestration runtime	Supervises, orchestrates, monitors, and drives the distributed data plane.
VM1	10.10.252.11	Ubuntu 22.04.5 LTS; QEMU virtual CPU (2 sockets $\times$ 2 cores); 3.8 GiB RAM; 26 GB disk; no dedicated GPU.	Kafka broker 1, HDFS DataNode 1, Spark worker 1, Trino worker 1, cAdvisor	Worker node dedicated to distributed storage, processing, and query participation.
VM2	10.10.252.12	Ubuntu 22.04.5 LTS; QEMU virtual CPU (2 sockets $\times$ 2 cores); 3.8 GiB RAM; 26 GB disk; no dedicated GPU.	Kafka broker 2, Kerberos KDC, NameNode, DataNode 2, Hive Metastore and metastore database, Spark master, submit helper, worker 2, Thrift Server and History Server, Trino coordinator, HTTPS gateway, cAdvisor	Data-plane coordination node for storage, processing, metadata, querying, and protected cluster services.
VM3	10.10.252.13	Ubuntu 22.04.5 LTS; QEMU virtual CPU (2 sockets $\times$ 2 cores); 3.8 GiB RAM; 26 GB disk; no dedicated GPU.	Kafka broker 3, HDFS DataNode 3, Spark worker 3, Trino worker 2, cAdvisor	Additional worker node reinforcing distributed storage, compute, and query execution.

Table IV.5: Physical node distribution of the final deployment

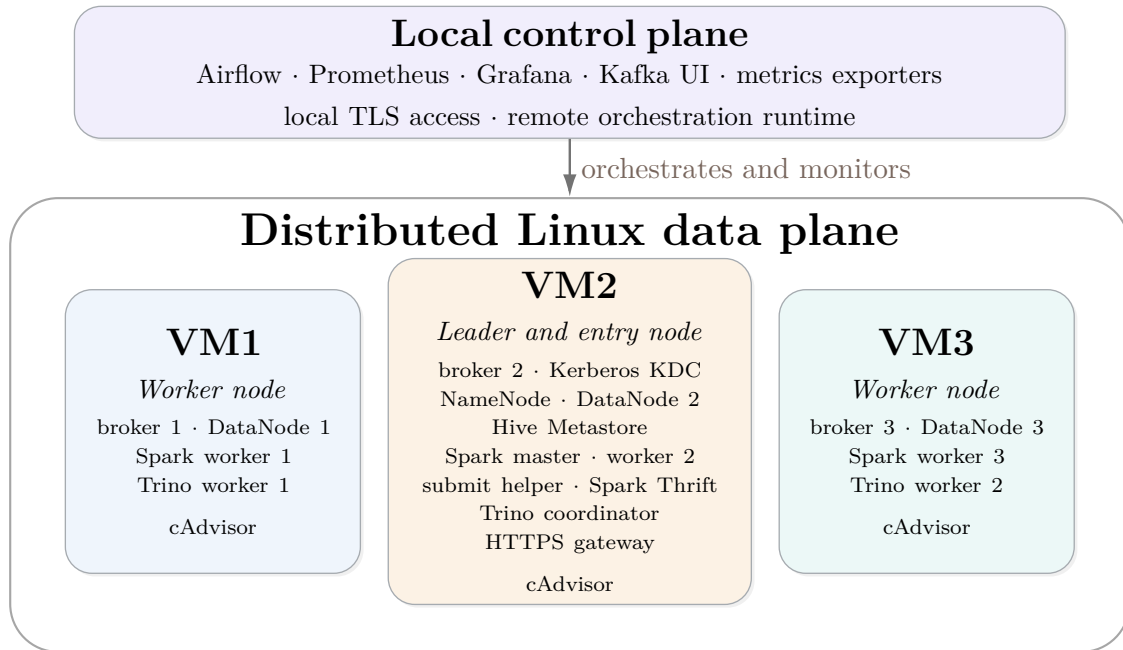


Figure IV.3: Physical topology of the local control plane and three-node distributed data plane.

IV.9 End-to-End Data Flow

The end-to-end operational flow of the platform can be summarized as follows:

1. Airflow initiates the five implemented workflows for environment reset, lakehouse setup, service readiness, integrated raw ingestion/loading and validation, and dbt transformation;
2. source datasets are prepared and published through the configured ingestion path;
3. Kafka receives the records and distributes topic partitions across the broker cluster;
4. bronze consumers persist replayable raw artifacts in HDFS through NameNode/DataNode coordination;
5. Spark reads the bronze zone and loads Iceberg raw tables through distributed execution, with Hive Metastore registering the shared table definitions;
6. within the same Kafka–HDFS–Spark workflow, Great Expectations assets are bootstrapped and the raw lakehouse tables are validated, producing JSON results and HTML Data Docs;
7. dbt-spark creates staging, intermediate, and analytics models and applies its declared transformation tests;

8. Trino exposes distributed SQL access to the governed raw and curated lakehouse tables;
9. Prometheus and Grafana continuously expose infrastructure and pipeline monitoring signals across the execution lifecycle.

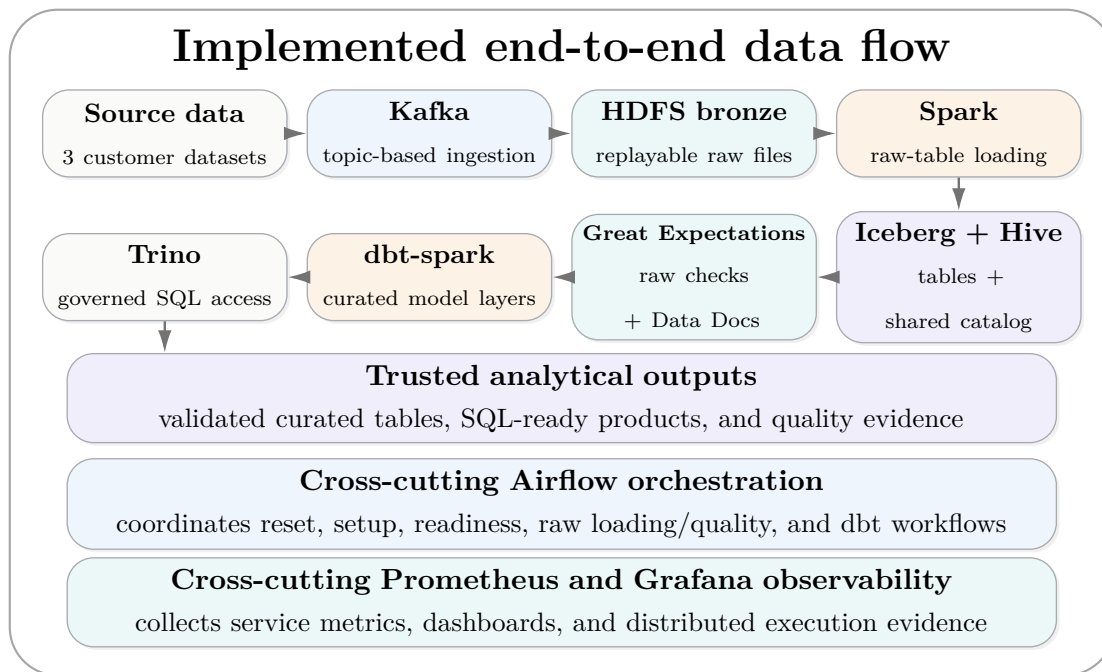


Figure IV.4: Implemented data flow from source ingestion to validated analytical outputs.

This ordering distinguishes the primary data path from the cross-cutting orchestration and observability controls. It also establishes the reference sequence used by the implementation and validation chapters.

IV.10 Security-by-Design Integration

Security is not positioned as an external afterthought. It is integrated into the design of the platform through explicit controls attached to core layers. This follows the same engineering rationale as the rest of the architecture: identity, traffic protection, and protected storage boundaries are treated as part of the system design itself, not as a late operational patch [?, ?, ?]. Table ?? summarizes this logic.

Security control	Protected scope	Design contribution
Kerberos	Hadoop secure mode and service authentication	Ensures that protected cluster services rely on explicit identity rather than implicit trust.
SPNEGO	Secured Hadoop web interfaces	Extends authenticated access to browser-facing Hadoop endpoints.
TLS	Sensitive exposed service endpoints	Protects data movement and browser-facing access paths.
LUKS	Runtime storage volumes on the VMs	Provides encrypted-at-rest support for persisted service state.
SSH-based remote orchestration	Control-plane to data-plane submission path	Keeps remote execution structured and controlled from the orchestration side.
Authenticated gateways	Query and access entry points	Reduces uncontrolled direct exposure of critical services.

Table IV.6: Security controls integrated into the system design

IV.11 Design Rationale and Trade-offs

The final design reflects deliberate engineering trade-offs rather than maximum-complexity choices:

- **Distributed execution within constrained resources:** real multi-node cooperation is retained within the available VM resource limits.
- **Layered responsibilities:** explicit service boundaries improve maintainability and fault isolation.
- **Integrated hardening:** deployed paths are protected without implying an enterprise-complete security program.
- **Operational transparency:** checks, metrics, logs, and execution artifacts keep platform behavior inspectable and diagnosable.
- **Future-ready scope:** curated outputs support later Customer 360 and AI services without overstating current functionality.

These trade-offs align the architecture with the implemented workload, available infrastructure, and operational requirements.

IV.12 Conclusion

This chapter formalized the stakeholders and requirements of AdOptimizer CDP, mapped them to the logical and physical architectures, and defined the end-to-end flow and integrated security baseline. These design choices provide the reference model for the concrete platform implementation presented in Chapter V.

Chapter V

Implementation of the Distributed Lakehouse Platform

V.1 Introduction

This chapter presents the concrete implementation of AdOptimizer CDP as an operational distributed lakehouse platform. While Chapter IV defined the design baseline, the present chapter explains how that design was translated into working services, executable pipelines, governed data layers, and secured access paths. The objective is not to describe isolated tools separately, but to show how ingestion, storage, processing, metadata, querying, transformation, quality validation, orchestration, and observability were assembled into one coherent engineering system.

V.2 Data Sources and Dataset Modeling

The implementation is based on a compact but representative three-source bundle selected to support the Customer 360 orientation of the platform while remaining compatible with the available infrastructure. Rather than using one oversized monolithic source, the project retains complementary datasets, each contributing a different view of customer behavior.

Each dataset keeps its source identity during ingestion and bronze persistence. This choice preserves replayability and traceability before any normalization or business transformation is applied. In practice, the implementation therefore distinguishes between source-specific raw inputs and later converged analytical models, maintaining schema fidelity at the entry point while enabling business-oriented integration in the upper layers.

Dataset and source file	Analytical role	Customer signal covered	Implementation contribution
Bank Marketing bank-additional-full.csv	campaign and response analysis	contact history, subscription outcome, profile attributes	provides customer profile and campaign-oriented raw inputs
Online Retail II online_retail_2.xlsx	transactional and commercial behavior analysis	invoices, products, quantities, revenue, countries	provides purchase history and product interaction facts
Online Shoppers Purchasing Intention online_shoppers_intention.csv	digital behavioral analysis	session behavior, navigation, purchase intention	provides pre-purchase intent and online behavior indicators

Table V.1: Active datasets retained in the implemented platform

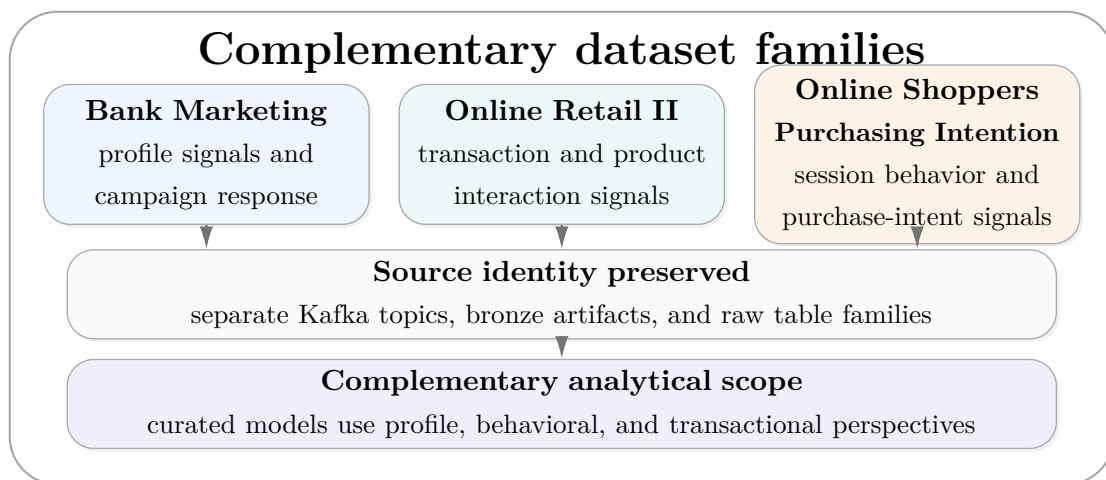


Figure V.1: Complementary dataset families and their later use in curated analytical models.

V.3 Kafka-Based Ingestion Layer

Kafka is used as the transport backbone of the ingestion layer. The implementation does not load datasets directly into storage or processing services; instead, each source is first published through a topic-oriented flow that decouples dataset reading from downstream persistence. This approach improves control, replayability, and operational clarity, which is consistent with both the Kafka event-log model and broader stream-oriented architectural reasoning [?, ?].

The Kafka cluster is distributed across the three virtual machines:

- broker 1 on VM1;
- broker 2 on VM2;
- broker 3 on VM3.

At the implementation level, each source dataset is associated with its own topic family. This prevents unrelated sources from being merged too early and makes reset and replay operations more explicit. The control plane triggers the ingestion logic, producers publish the records, and downstream consumers persist the bronze output in HDFS. This means Kafka acts as the controlled transport boundary between local source preparation and distributed storage.

The platform also implements topic reset logic within the orchestration flow. This supports reproducible reruns by allowing the environment-reset pipeline to remove previous ingestion state before a fresh execution begins.

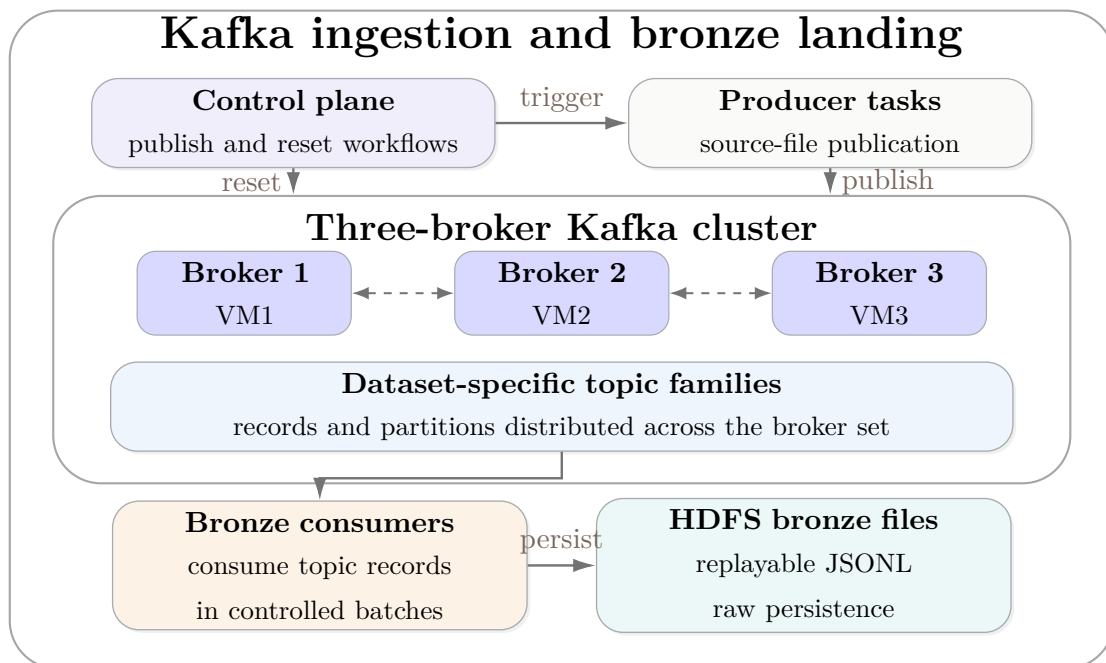


Figure V.2: Kafka publication, topic distribution, and HDFS bronze landing flow.

V.4 HDFS Bronze Storage Layer

The bronze layer is implemented on HDFS as the replayable raw persistence zone of the platform. After Kafka-based transport, source records are landed into HDFS in batch-oriented bronze files before any lakehouse structuring step is executed. This choice gives the project an explicit raw persistence boundary, which is essential for reproducibility, debugging, and future reprocessing.

The HDFS cluster is implemented with:

- the NameNode on VM2;
- one DataNode on each VM: VM1, VM2, and VM3.

In operational terms, this means the namespace authority remains centralized on VM2, while block storage is distributed across the three nodes. The implemented bronze organization is client-oriented and source-aware: each dataset is persisted under a controlled bronze structure rather than being mixed into one generic folder. This reinforces traceability and keeps the raw layer easy to inspect.

The bronze layer also plays a strategic role in the whole project: it is the stable checkpoint between ingestion and compute. If a downstream Spark stage fails while valid bronze artifacts remain available, the affected processing stages can be rerun from HDFS without rereading the original local files or republishing the datasets through Kafka.

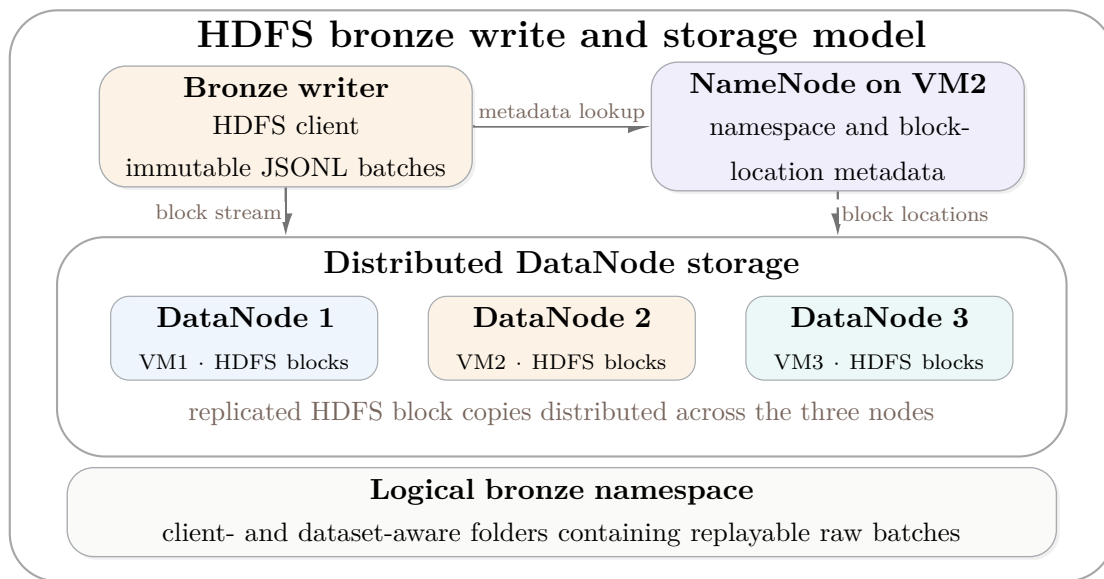


Figure V.3: HDFS bronze storage with NameNode control and distributed DataNodes.

V.5 Spark Processing and Iceberg Loading

Spark is the main distributed processing engine of the platform. Its implementation is aligned with the physical cluster design and with the lakehouse objective of turning replayable distributed files into governed analytical tables [?, ?, ?]:

- the Spark master runs on VM2;
- workers are distributed across VM1, VM2, and VM3;
- the control plane submits jobs remotely to the Spark environment on VM2.

This execution path is important. The final deployment does not rely on a heavy local Spark driver on the workstation. Instead, the control plane submits work to the distributed cluster, maintaining a clear separation between orchestration and compute.

From a data perspective, Spark reads the bronze files from HDFS, interprets their schemas, normalizes the raw structures, and writes the outputs into Iceberg-backed raw lakehouse tables. The implementation therefore transforms replayable raw artifacts into governed table objects while preserving distributed execution. The raw lakehouse layer is not yet the final curated business layer; it is the structured foundation that is validated within the same raw workflow before the following dbt workflow builds the curated models.

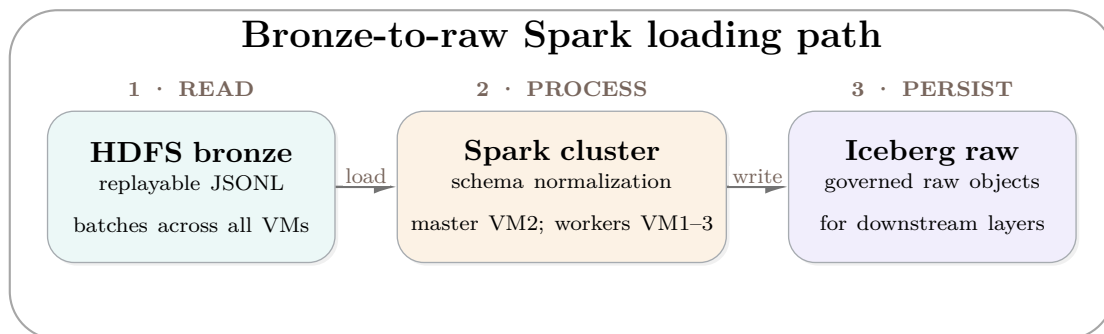


Figure V.4: Spark loading path from HDFS bronze files to Iceberg raw tables.

V.6 Hive Metastore and Catalog Governance

Hive Metastore is implemented as the shared metadata authority of the lakehouse platform. It is deployed on VM2 together with its PostgreSQL backing store and acts as the central catalog reference for upper services such as Spark and Trino [?, ?].

Its role in the implementation is essential: once Spark writes the raw Iceberg tables, their definitions must be discoverable through a common metadata layer. Without this shared governance service, each engine would have to manage table visibility independently, weakening consistency across the platform.

In practical terms, Hive Metastore allows the platform to:

- register the lakehouse tables created after Spark loading;
- expose the same logical objects to different access engines;
- preserve a governed catalog boundary between raw persistence and analytical consumption.

This makes the metadata layer a real implementation component, not a theoretical addition. It is the reason the lakehouse can behave like a coordinated data platform rather than a simple collection of files.

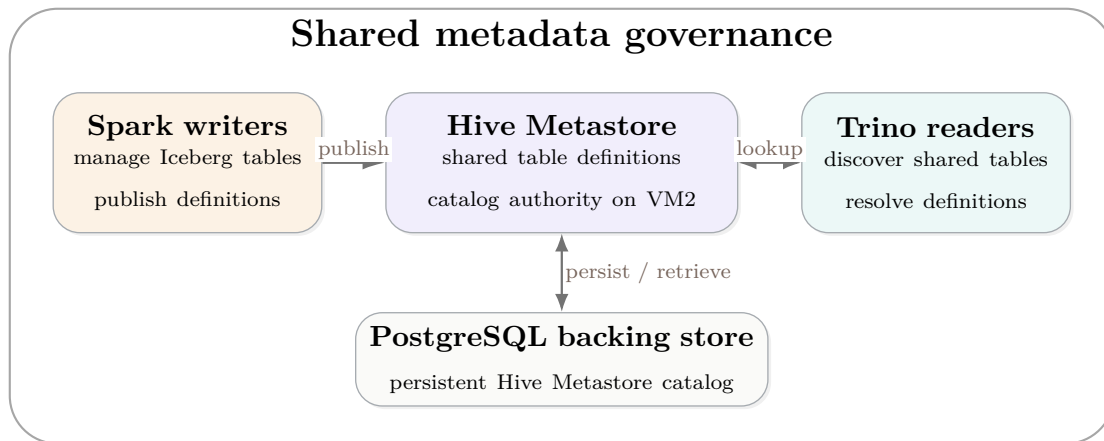


Figure V.5: Shared metadata governance through Hive Metastore.

V.7 Trino Query Layer

Trino provides the analytical SQL access layer of the platform. Its implementation follows a coordinator/worker separation consistent with distributed query-engine design [?]:

- the coordinator is hosted on VM2;
- worker 1 is hosted on VM1;
- worker 2 is hosted on VM3.

This arrangement is important because it demonstrates that the query layer is also distributed, not only the storage or compute layer. Trino consumes the shared catalog definitions and exposes interactive SQL access over the Iceberg tables. In the project, this layer is used for readiness checks, lakehouse verification, dataset inspection, and analytical exploration.

The query access path is also protected by an authenticated gateway. Consequently, the Trino browser interface is not exposed as an uncontrolled plain endpoint, and analytical access remains aligned with the security posture described in Chapter VII.

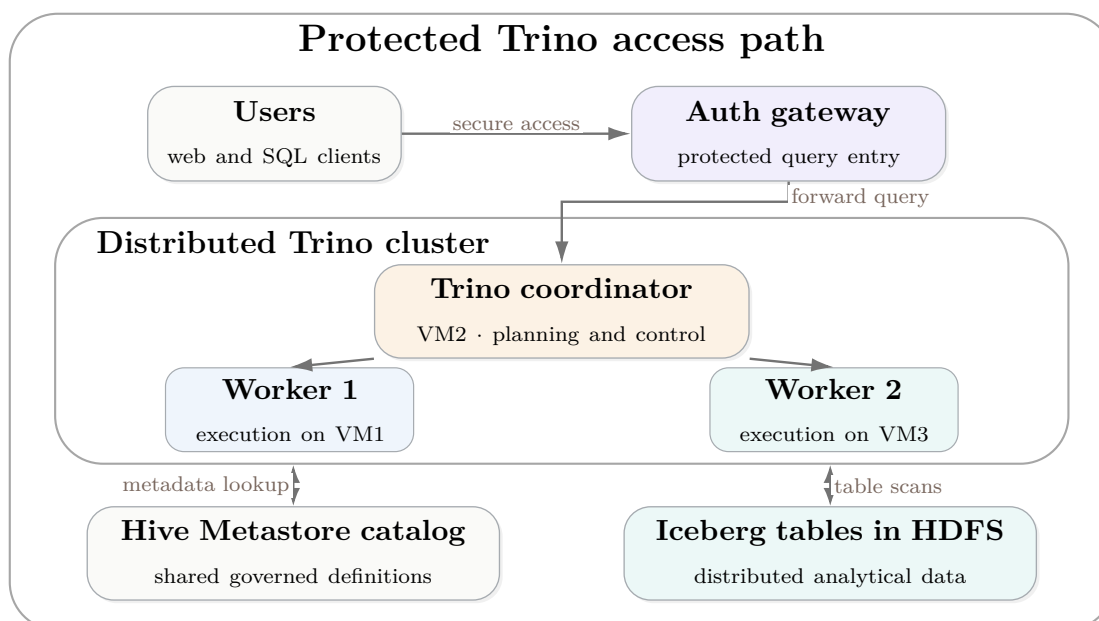


Figure V.6: Protected Trino access through the gateway and distributed query cluster.

V.8 dbt-spark Transformation Layer

dbt-spark is used to organize transformations as declarative models rather than as ad hoc processing scripts. This implementation choice is important because it introduces structure, dependency control, and documentation logic into the transformation stage [?].

The implemented transformation flow follows a layered modeling philosophy:

- raw lakehouse tables remain the direct output of the Spark loading stage;
- staging models standardize the three source-specific raw structures;
- intermediate models prepare reusable behavioral, commercial, campaign, and Customer 360 features;
- analytical models expose business-oriented outputs ready for exploration.

The implemented model inventory is summarized in Table ??.

Layer	Implemented models	Purpose
Staging	stg_bank_marketing, stg_online_shoppers_intention, stg_online_retail_2	standardize source-specific schemas and field semantics
Intermediate	int_bank_marketing_contacts, int_online_shopper_sessions, int_online_retail_customer_sales, int_customer_360_feature_store	build reusable subject-level and cross-source features
Analytics	analytics_customer360_overview, analytics_conversion_performance, analytics_sales_performance	expose governed analytical outputs for consumption

Table V.2: Implemented dbt-spark model inventory

In this design, dbt does not replace Spark. Instead, it builds on top of Spark Thrift access and uses the distributed lakehouse environment as its execution target. This distinction matters in the report because it shows that transformation governance is separated from raw compute mechanics: Spark provides the engine, while dbt provides the modeling discipline.

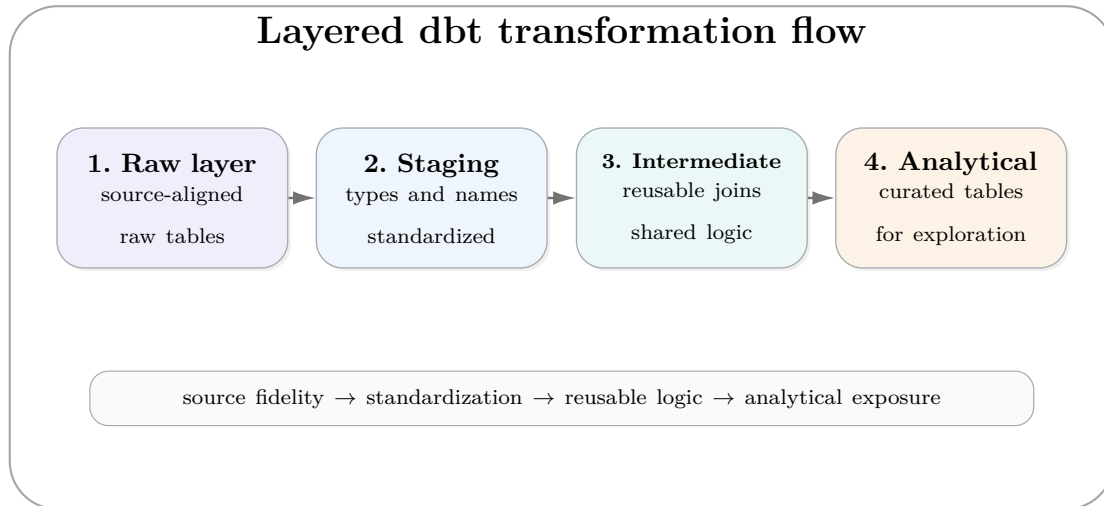


Figure V.7: Layered dbt transformation flow from raw to analytical models.

V.9 Great Expectations Validation Layer

Great Expectations is implemented as the validation layer of the platform. Its role is not to perform generic visual checking, but to formalize trust conditions over the produced data and retain evidence of each validation run [?].

The active raw-lakehouse validation checkpoints are:

- `raw_lakehouse_quality_checkpoint__bank_marketing.json`;
- `raw_lakehouse_quality_checkpoint__online_shoppers_intention.json`;
- `raw_lakehouse_quality_checkpoint__online_retail_2.json`.

These checkpoints evaluate structural and semantic expectations for the three implemented raw-lakehouse source families. They are executed inside `customerdna_client1_kafka_hdfs_spark_lakehouse_pipeline`, after Spark has materialized the raw Iceberg tables. In that DAG, `bootstrap_raw_lakehouse_quality_assets` prepares the validation assets and `validate_raw_lakehouse_quality` runs the checks. Their results are persisted as validation JSON and rendered through HTML Data Docs, providing traceable evidence that can be inspected independently of the Airflow task status. Curated-model validation remains the responsibility of the declared dbt tests in the following transformation DAG.

This layer is therefore central to the governance dimension of the project. It strengthens the transition from “data processed successfully” to “data satisfies the defined conditions for analytical use.”

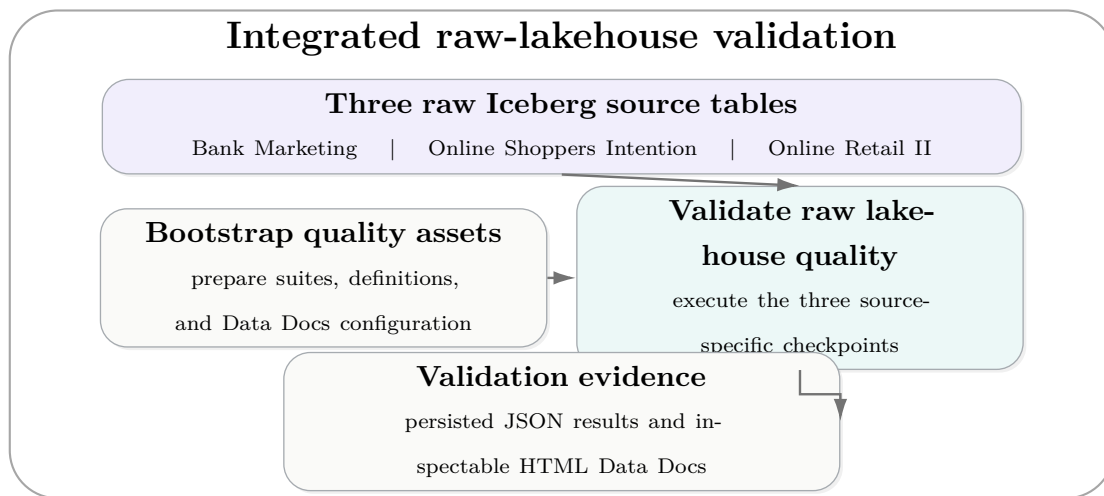


Figure V.8: Raw-lakehouse validation tasks integrated into the Kafka-HDFS-Spark workflow, with persisted evidence.

V.10 Airflow Orchestration Layer

Airflow orchestrates the platform from the local control plane. In the final implementation, orchestration is not reduced to one oversized DAG. Instead, the project is structured around a small set of operational DAGs, each with a clear responsibility in the lifecycle of the environment [?].

Order and implemented DAG	Operational role	Main output or effect
1. <code>customerdna_client1_environment_reset_pipeline</code>	clears previous Kafka, HDFS, Hive, and Trino state	controlled baseline for reproducible reruns
2. <code>customerdna_client1_lakehouse_setup_pipeline</code>	initializes the bronze namespace and required lakehouse structures	prepared technical foundation for later checks and loading
3. <code>customerdna_client1_lakehouse_readiness_pipeline</code>	verifies HDFS, Hive, Spark, Spark Thrift, and Trino availability	positive readiness state before data execution
4. <code>customerdna_client1_kafka_hdfs_spark_lakehouse_pipeline</code>	publishes source records, lands bronze files, loads raw Iceberg tables, and executes the integrated raw-quality tasks	distributed raw tables, validation JSON, and Data Docs
5. <code>customerdna_client1_dbt_spark_lakehouse_pipeline</code>	executes staging, intermediate, and analytical transformations with dbt tests	governed curated models and transformation-test results

Table V.3: Implemented orchestration flow of the platform

This orchestration strategy matches the implemented operational logic of the project. The platform can be reset, initialized, checked, loaded and raw-validated, and then transformed through five controlled workflow boundaries. Great Expectations is not represented by a sixth or independent DAG: its asset-bootstrap and validation tasks belong to the

fourth workflow. Failures can therefore be localized either to a workflow boundary or, for raw-quality validation, to its explicit task boundary inside the raw pipeline.

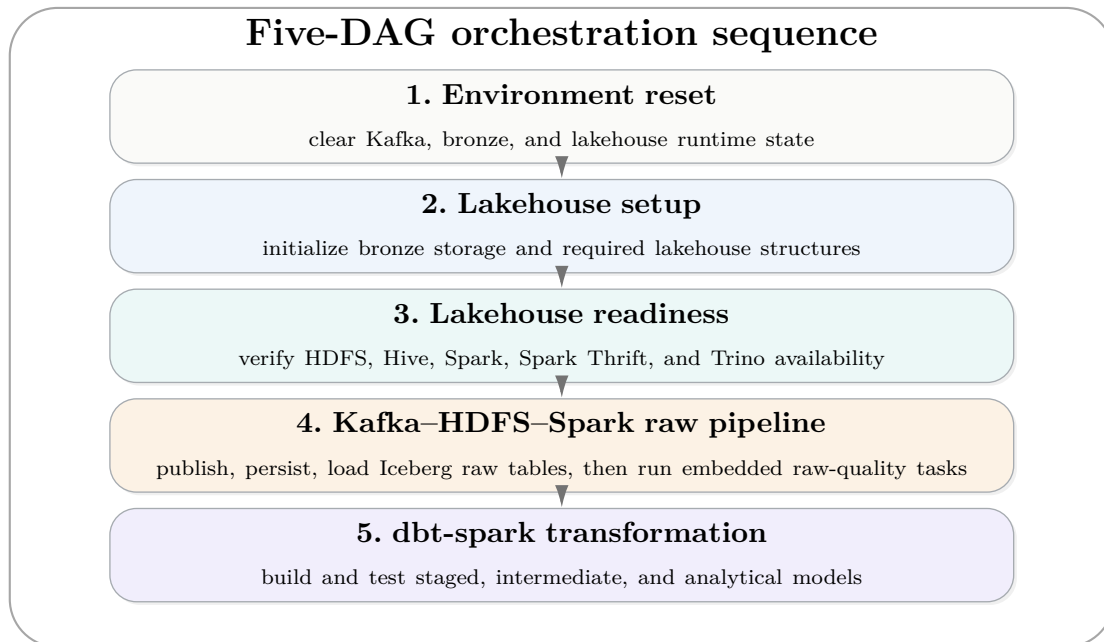


Figure V.9: Five-DAG orchestration sequence from reset and setup to readiness, integrated raw loading and validation, and curated transformation.

V.11 Observability Integration

Operational visibility is integrated from the local control plane. Prometheus collects service, exporter, container, and pipeline metrics, while Grafana presents infrastructure and execution dashboards. Kafka UI, the Spark user interface and history server, and Airflow logs complement these centralized views. Together, they relate workflow status to service health, node activity, and processing behavior; the resulting validation evidence is examined in Chapter VIII.

V.12 Conclusion

This chapter demonstrated the implemented engineering chain: Kafka publication, HDFS bronze persistence, distributed Spark processing, Iceberg tables governed through Hive Metastore, integrated Great Expectations raw validation, dbt-spark transformations and tests, Trino access, Airflow orchestration, and Prometheus/Grafana supervision. These integrated layers form a distributed lakehouse with explicit operational and data-movement boundaries. Chapter VI examines its physical deployment and operation across the final multi-node environment.

Chapter VI

Deployment Strategy and Distributed Execution

VI.1 Introduction

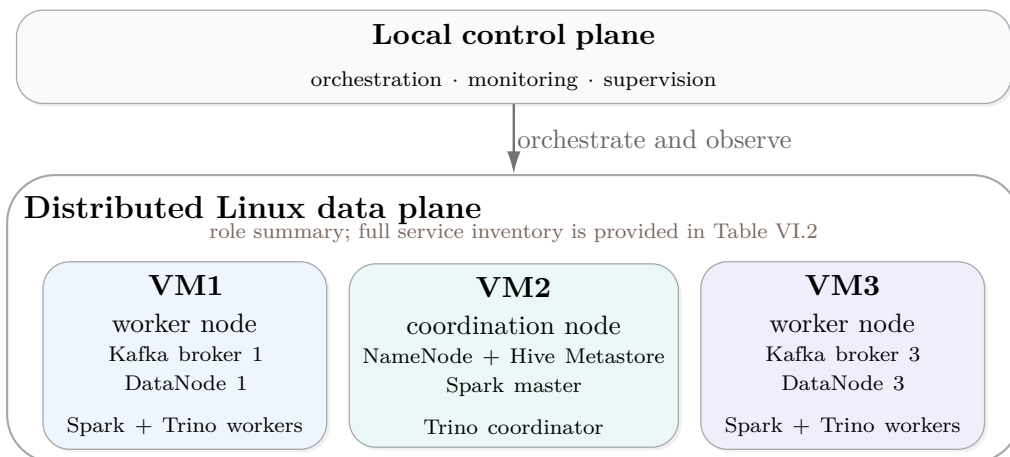
This chapter explains how AdOptimizer CDP is physically deployed and how its distributed execution is realized in practice. It clarifies the separation of services, the movement of data and control across the environment, and the operational mechanisms used to reproduce the deployment. The approach combines distributed-service reasoning, containerized packaging, dependency-aware startup logic, and remote execution paths [?, ?, ?].

VI.2 Control Plane Responsibilities

The project adopts a deployment strategy based on a clear separation between a **local control plane** and a **distributed data plane**. The control plane is hosted on the local workstation, while the heavy data services are distributed across the three Linux virtual machines.

This separation prevents the virtual machines from being loaded with orchestration and monitoring services that do not participate directly in distributed storage or processing. It also establishes a clear responsibility boundary: the control plane orchestrates, supervises, and initiates remote work, while the data plane transports, stores, processes, governs, and serves data.

Control-plane service	Operational role	Reason for local placement
Airflow	orchestration of reset, setup, readiness, integrated raw loading/quality, and dbt transformation DAGs	avoids consuming VM resources for workflow control and keeps execution coordination centralized
Prometheus	collection of infrastructure and pipeline metrics	centralizes observability and simplifies metric management
Grafana	dashboard-based monitoring of infrastructure and pipeline state	provides one visual supervision point for the whole platform
Kafka UI	broker and topic inspection	keeps operational visibility outside the distributed data services themselves
Pipeline metrics exporter	exposure of pipeline-specific operational metrics	complements infrastructure metrics with workflow-level signals
Local TLS gateway	protected browser access to control-plane interfaces	centralizes HTTPS exposure for Airflow, Grafana, Kafka UI, and Prometheus
SSH submission path	remote Spark submission to the entry point on VM2	avoids local-driver networking constraints and centralizes cluster entry

Table VI.1: Main responsibilities of the local control plane**Figure VI.1:** Control-plane supervision of the distributed Linux data plane.

VI.3 VM-Level Service Separation

The data plane is not deployed as three identical generic machines. Instead, the services are deliberately separated according to their coordination role or worker role.

This service distribution supports clear operational responsibility. VM2 concentrates services that require a central authority or coordination function, including Kerberos, the HDFS namespace, the shared catalog, Spark scheduling, application history, and distributed-query coordination. VM1 and VM3 provide horizontal extension through storage blocks, Spark execution resources, Kafka brokers, and Trino worker capacity.

Node	Main hosted services	Architectural logic
VM1	Kafka broker 1, HDFS DataNode 1, Spark worker 1, Trino worker 1, cAdvisor	extends storage, compute, and query-worker capacity without carrying cluster leadership responsibilities
VM2	Kafka broker 2, Kerberos KDC, NameNode, DataNode 2, Hive Metastore and PostgreSQL store, Spark master, Spark worker 2, Spark submit client, Spark Thrift Server, Spark History Server, Trino coordinator, Trino gateway, cAdvisor	acts as the leader node and integration point for coordination-heavy services
VM3	Kafka broker 3, HDFS DataNode 3, Spark worker 3, Trino worker 2, cAdvisor	reinforces the cluster with additional worker capacity and distributed persistence

Table VI.2: VM-level service allocation in the distributed deployment

Unlike a pseudo-distributed installation, the principal storage, processing, transport, and query responsibilities are materially distributed across separate machines. The deployment topology is therefore part of the implemented engineering contribution.

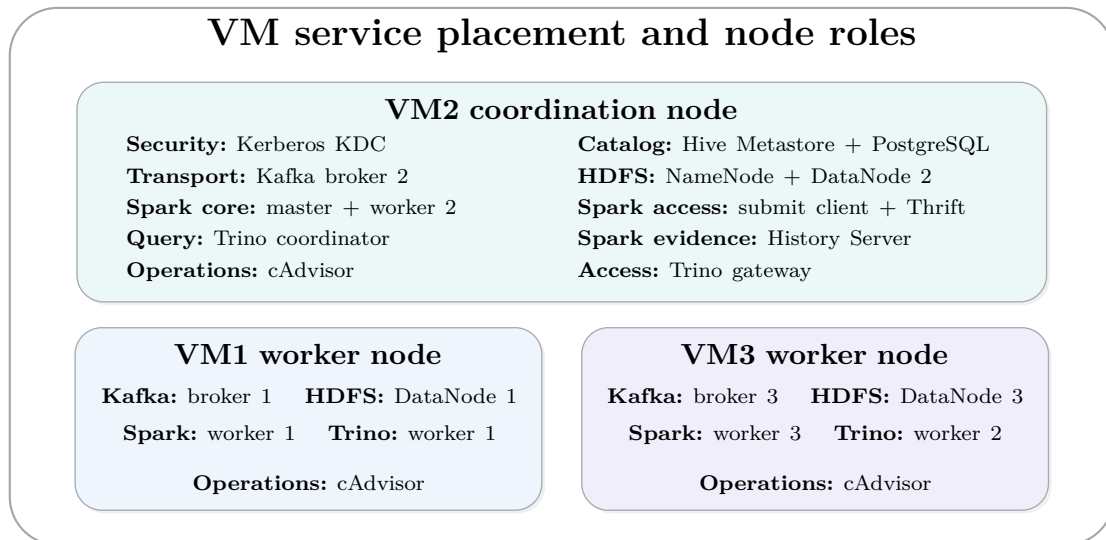


Figure VI.2: Service placement and node roles across VM1, VM2, and VM3.

VI.4 Data Flow Across Machines

The implemented machine-to-machine flow follows an explicit sequence:

1. Airflow initiates the relevant workflow from the local control plane;
2. local producers read the source datasets and publish records to Kafka;
3. topic partitions are hosted across the brokers on VM1, VM2, and VM3;
4. bronze consumers read the topics and write source-specific `.jsonl` batches to HDFS;
5. the NameNode on VM2 coordinates block persistence across the three DataNodes;

6. Spark jobs enter through the remote submission path on VM2, with executors allocated across registered workers;
7. Spark reads bronze data, writes raw Iceberg tables, and registers their metadata through Hive Metastore on VM2;
8. the embedded Great Expectations tasks validate the raw lakehouse tables and persist validation results and Data Docs on the control plane;
9. dbt-spark is initiated from the control plane while its transformation work executes through the Spark cluster;
10. the Trino coordinator on VM2 plans analytical queries whose fragments are executed by workers on VM1 and VM3;
11. Prometheus collects distributed service metrics and Grafana exposes centralized monitoring views.

The resulting path crosses explicit machine and service boundaries for transport, storage, processing, metadata governance, querying, transformation, validation, and supervision. The platform therefore does not collapse its data path into a single-machine ETL process.

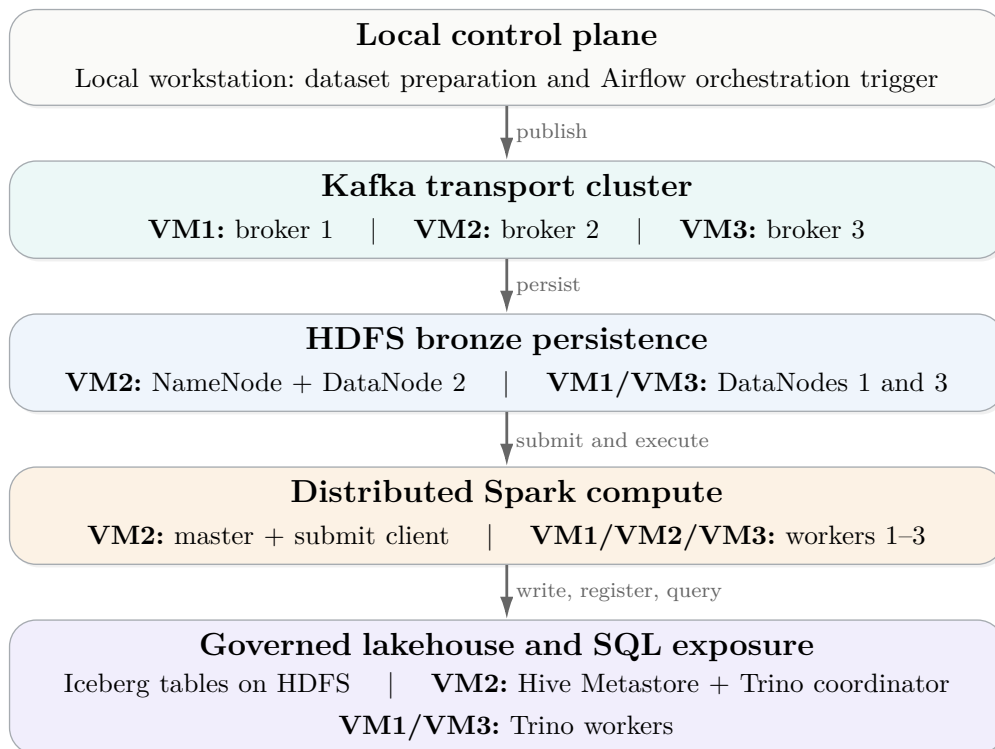


Figure VI.3: Machine-level flow across the local control and three-node data planes.

VI.5 Distributed Spark Execution

Spark provides the principal distributed-processing mechanism of the platform. The Spark master is hosted on VM2, while executor capacity is registered through workers on VM1, VM2, and VM3. Processing capacity is therefore not bound to the coordination node.

The Spark master interface exposes the registered worker pool, and applications allocate executors according to workload requirements and available capacity. Processing tasks can consequently be scheduled beyond the coordinator node itself; the corresponding runtime evidence is examined in Chapter VIII.

The deployment also includes a Spark submit client, Spark Thrift Server, and Spark History Server on VM2. These components preserve the separation between cluster entry, SQL access, execution, and historical inspection [?]:

- Spark jobs are launched from the control plane through an SSH-based remote submission path to VM2;
- transformation-oriented SQL access targets the distributed Spark environment through the Thrift interface;
- application history is retained through the History Server for later inspection;
- execution is coordinated centrally while tasks are carried out by workload-selected workers.

Executor allocation is workload-dependent. A registered worker may receive no tasks during a light application, because distributed execution does not imply equal simultaneous utilization of every node. The relevant criterion is that the cluster can allocate work beyond VM2 when the workload and available resources require it.

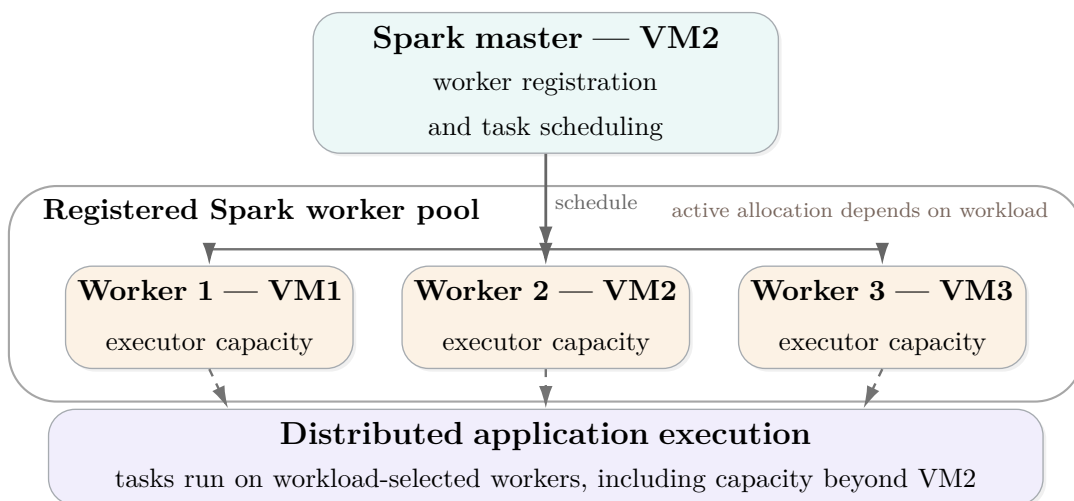


Figure VI.4: Workload-dependent Spark execution across VM1, VM2, and VM3.

VI.6 Operational Packaging and Deployment Bundles

The deployment is supported by explicit operational packaging rather than undocumented manual steps. Node-specific bundles, startup logic, reset logic, and preflight checks capture the operational dependencies of the platform and support reproducible execution [?].

Deployment artifact type	Role in operations	Operational benefit
Bundle management scripts	prepare and synchronize node-specific deployment content	maintain consistent configuration across repeated deployments
Preflight checks	verify ports, dependencies, peer endpoints, and critical services	identify blocking conditions before service startup
Reset scripts	clean previous runtime state and support reproducible reruns	restore a controlled baseline for validation cycles
Startup scripts	launch services in the required sequence	formalize the operational lifecycle of the platform
Access-point documentation	record URLs, ports, and secured entry paths	preserves the traceability of operational interfaces

Table VI.3: Operational deployment artifacts used in the project

The deployment bundles translate the architectural service map into machine-specific operational units. Each node receives the configuration, compose definitions, mounted service files, and startup logic required for its assigned role. Keeping these elements together reduces configuration drift between repeated deployments and makes the relationship between a service, its node, and its exposed interfaces explicit. Bundle preparation is therefore a reproducibility mechanism rather than a simple file-transfer step.

Preflight verification acts as a gate before any distributed workflow is launched. It checks the conditions on which later services depend, including required ports, peer reachability, expected endpoints, deployment content, and critical prerequisites. Detecting these conditions before startup avoids partial cluster states in which a worker is running but cannot reach its coordinator, or an orchestration task begins while the storage, catalog, or execution layer is still unavailable. The checks also provide a clearer failure boundary because deployment problems can be separated from data-pipeline failures.

The startup order is intentional. VM2 is started first because it hosts the coordination-heavy services: the Kerberos authority, HDFS NameNode, Hive Metastore, Spark master, and Trino coordinator establish identities, namespaces, metadata, scheduling, and query coordination for the rest of the platform. VM1 and VM3 are then started to register their storage, transport, compute, and query-worker capacity against those shared services. Finally, the local control plane is activated only after data-plane readiness has been confirmed, preventing Airflow and the monitoring stack from interpreting normal initialization delays as operational failures. This sequence follows service dependencies rather than an arbitrary launch order.

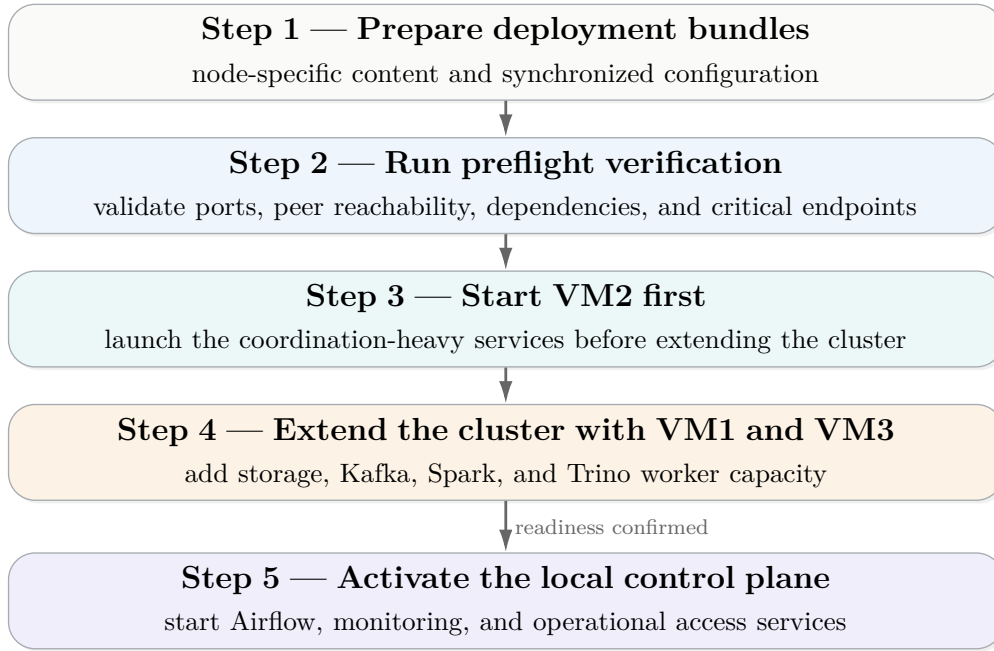


Figure VI.5: Dependency-aware deployment lifecycle and startup order.

VI.7 Conclusion

This chapter clarified that the deployment of AdOptimizer CDP is intentionally distributed at both the service level and the machine level. The local control plane supervises the environment without collapsing it into a single node, VM2 plays the role of coordination leader, and VM1 and VM3 extend storage, compute, and worker capacity. Data movement, processing, and analytical access therefore take place across explicit multi-node boundaries.

The deployment is reproducible through node-specific bundles, reset logic, startup scripts, and preflight checks. It therefore constitutes an implemented operational topology rather than only a conceptual design. Chapter VII examines the security controls that harden this deployment and protect its critical access paths.

Chapter VII

Security Architecture and Hardening

VII.1 Introduction

This chapter presents the security architecture integrated into AdOptimizer CDP. The goal is not to claim a complete enterprise identity-management or zero-trust program, but to provide an implemented hardening baseline for service identity, browser access, exposed interfaces, remote operations, credential separation, and persisted runtime data. The controls are grounded in established secure-service mechanisms [?, ?, ?, ?].

VII.2 Security Objectives

The security work of the project follows six main objectives:

- authenticate critical distributed services rather than relying on implicit trust;
- secure browser-facing administrative or analytical interfaces;
- protect exposed endpoints through encryption in transit;
- reduce the operational risk of remote orchestration actions;
- separate service, administration, and demonstration identities and credentials;
- strengthen the protection of service data persisted on the virtual machines.

Security objective	Implemented mechanism	Protected scope
Distributed identity control	Kerberos-secured Hadoop runtime	Hadoop services and related secure interactions
Protected web access	SPNEGO + HTTPS for HDFS browser endpoints	NameNode and DataNode administrative web paths
Secured analytical exposure	authenticated Trino gateway over TLS	browser-based SQL access path
Protected orchestration path	SSH-based remote Spark submission	control-plane to VM2 operational execution
Operational identity separation	distinct service, administration, and demonstration identities with environment-scoped credentials	non-interactive services and human-operated access paths
Encrypted persisted runtime data	optional LUKS-backed runtime storage support	service data at rest on the Linux nodes when enabled

Table VII.1: Main security objectives and corresponding implementation controls

VII.3 Kerberos-Secured Hadoop Services

Kerberos is the core identity mechanism used to secure the Hadoop side of the platform. In a distributed environment, services such as the NameNode and DataNodes should not trust requests only because they originate from another cluster component. They must validate the identity of the caller explicitly [?, ?].

In the implementation, the Kerberos KDC and administration service are hosted on VM2, which is already the coordination-heavy node of the data plane. This is consistent with the overall architecture: the node that centralizes namespace and metadata authority also centralizes identity management for the secure Hadoop runtime.

The practical contribution of Kerberos in this project is threefold:

- service principals and keytabs provide authenticated identities for non-interactive Hadoop services;
- it allows secure service-to-service interactions in the distributed cluster;
- it provides the foundation for browser-based negotiated authentication through SP-NEGO.

The Hadoop runtime therefore uses explicit authentication rather than relying only on open internal trust between cluster components.

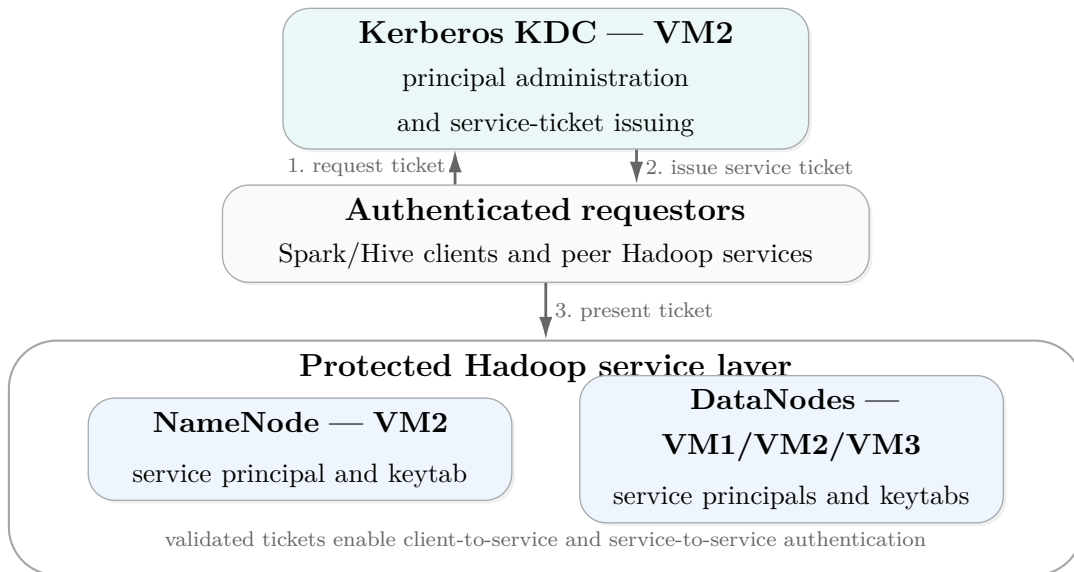


Figure VII.1: Kerberos ticket flow for authenticated Hadoop clients and services.

VII.4 SPNEGO-Protected HDFS Browser Access

The HDFS browser path is protected through SPNEGO layered on top of the Kerberos-based identity model. This means that access to the secured NameNode and DataNode web interfaces is not treated as a public or weakly protected administrative convenience. It becomes part of the authenticated architecture [?, ?].

This protection extends the Hadoop security posture to the human-facing HDFS administration and browsing path instead of securing only back-end service interactions.

The implementation required both server-side and client-side alignment:

- server-side Kerberos-aware Hadoop configuration;
- SPNEGO-protected web endpoints;
- HTTPS exposure of the HDFS browser interfaces;
- MIT Kerberos client configuration and a valid ticket on the Windows workstation;
- Firefox SPNEGO configuration aligned with the documented project hostname aliases rather than raw IP navigation.

This browser workflow is integration-sensitive because successful negotiation depends on the client ticket cache, hostname resolution, and Firefox configuration. A client-side SPNEGO failure must therefore be distinguished from the health of the Kerberos-secured Hadoop services themselves.

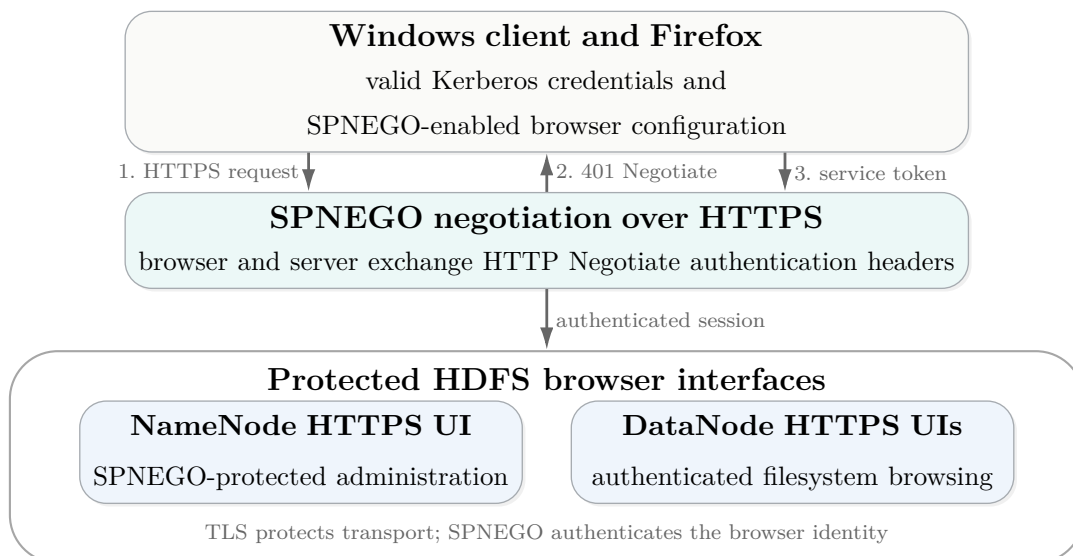


Figure VII.2: SPNEGO-authenticated HTTPS access to HDFS browser interfaces.

VII.5 TLS-Protected Interfaces

TLS is used to protect the main browser-facing interfaces of the platform. The objective is not to wrap every internal port indiscriminately, but to secure the access paths used for administration, monitoring, and analytical interaction. Transport protection complements identity protection rather than replacing it [?, ?].

Two main exposure zones are protected:

- the HDFS browser interfaces through HTTPS;
- the Trino analytical access gateway through HTTPS.

The local control-plane interfaces are also exposed through a local TLS gateway, establishing a consistent protected entry path for Airflow, Grafana, Prometheus, and Kafka UI.

The deployment uses internal or self-signed trust paths rather than a public enterprise certificate chain. Browser warnings may therefore appear until the local certificate authority is trusted on the client machine. This limitation affects certificate trust management, not the presence of encrypted transport.

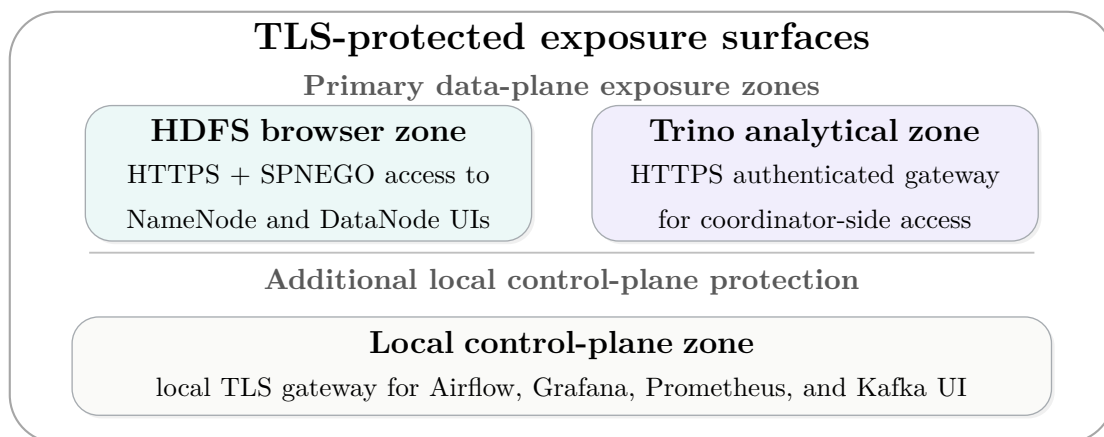


Figure VII.3: TLS-protected exposure zones for data-plane and control-plane interfaces.

VII.6 Authenticated Trino Access

Trino is not exposed as an open analytical endpoint. Instead, the project implements an authenticated gateway in front of the coordinator-side browser access path. This has two important effects [?]:

- it reduces uncontrolled direct access to the analytical query service;
- it applies an explicit authentication boundary to user-facing SQL access.

The gateway is hosted on VM2 together with the Trino coordinator, which is consistent with the node's role as the entry point for coordination-heavy services. Through this design, the browser-facing access path is separated from the internal worker topology. Users connect to the protected entry point, while the internal distributed Trino execution still takes place through the coordinator and worker nodes.

Authentication is enforced at the gateway level; this control should not be interpreted as a complete enterprise IAM or fine-grained authorization framework.

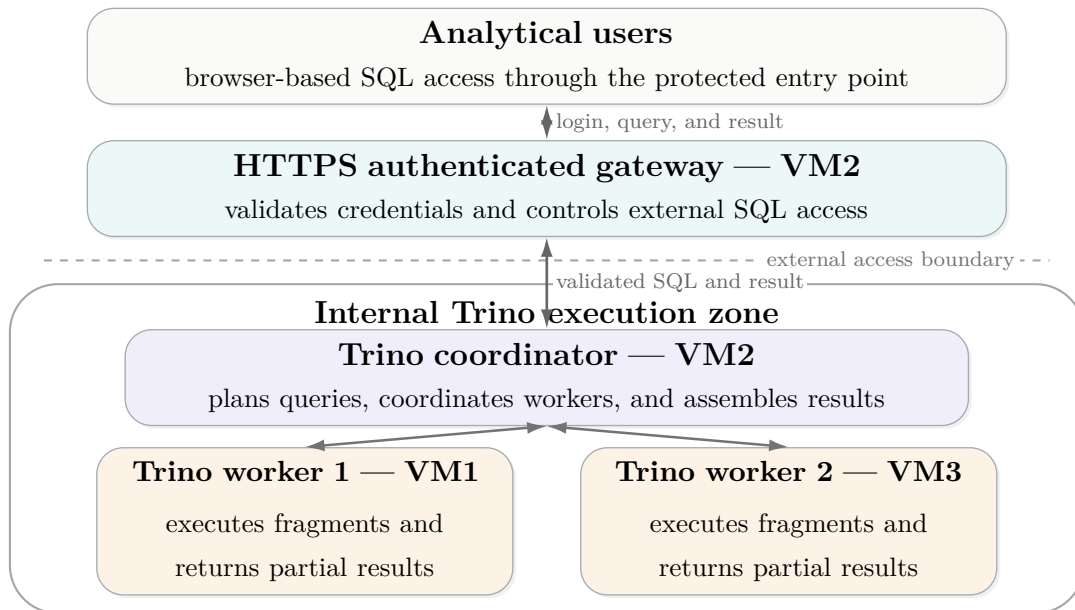


Figure VII.4: Authenticated Trino path from the VM2 gateway to distributed workers.

VII.7 SSH-Based Remote Spark Submission

The orchestration model relies on a remote Spark submission path from the control plane to the Spark submit client on VM2. This avoids fragile local-driver networking assumptions and keeps the operational launch path aligned with the distributed cluster environment.

From a security perspective, this means the orchestration node must not submit jobs through uncontrolled or manually repeated remote actions. The platform therefore uses an SSH-based remote execution path with dedicated key material and documented access preparation [?].

This design improves the platform in three ways:

- it keeps the control plane separate from the heavy Spark runtime;
- it avoids exposing the orchestration flow through weaker manual execution habits;
- it provides a consistent operational bridge between local Airflow logic and the distributed cluster leader on VM2.

This SSH path is therefore the authenticated operational bridge through which orchestration reaches distributed compute.

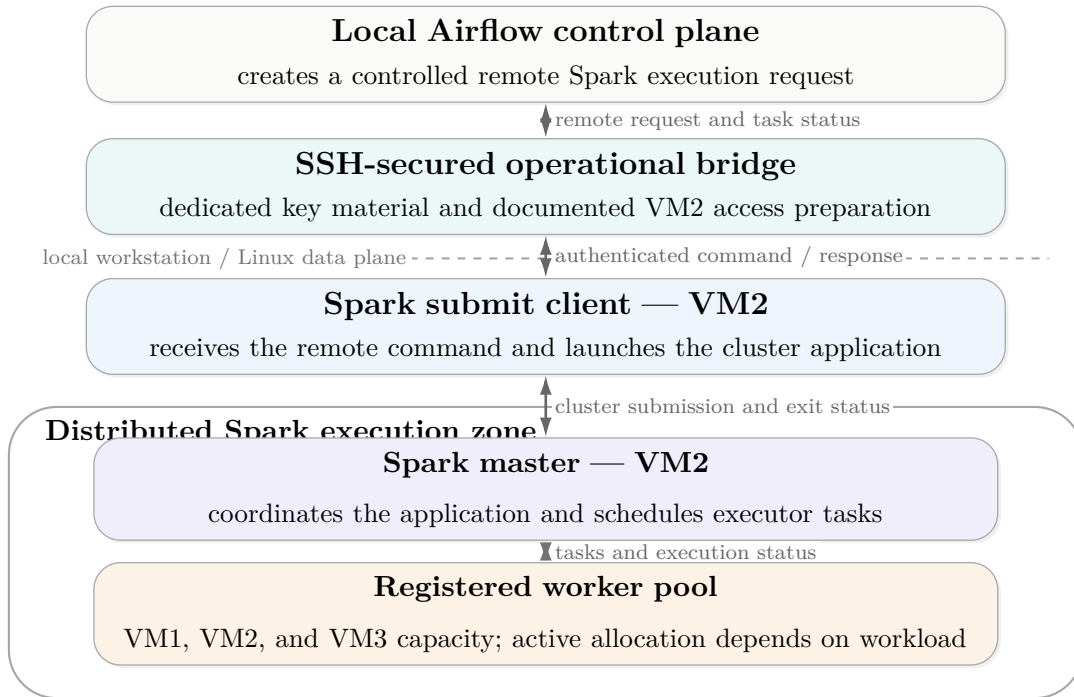


Figure VII.5: SSH-secured Spark submission from the local control plane to VM2.

Operational Identity and Credential Separation

The deployment separates non-interactive service identities from human-operated administration and demonstration identities. Kerberos principals and keytabs are assigned to protected Hadoop services, while browser credentials, gateway credentials, and SSH key material are scoped to their operational environments. This reduces uncontrolled credential sharing and clarifies which identity is used at each access boundary.

The implemented separation is a baseline control rather than a centralized secret-management system. Automated secret rotation, enterprise vault integration, and organization-wide identity federation remain outside the current scope.

VII.8 Encrypted Runtime Storage Support

The project also implements optional support for encrypted runtime storage through LUKS on the Linux virtual machines. This measure targets **data at rest** rather than service authentication or traffic encryption [?].

When enabled, this control protects runtime directories used by Kafka, HDFS, Hive, Spark, and Trino on VM1, VM2, and VM3. It complements the controls applied to service identity, browser access, and network transport.

The implemented logic therefore complements the other controls:

- Kerberos protects identity,
- SPNEGO protects browser negotiation,
- TLS protects exposed traffic,
- the authenticated Trino gateway protects analytical access,
- SSH protects remote Spark submission,
- LUKS-backed storage protects persisted runtime data when enabled.

This layered view helps the report explain that security was approached as a set of complementary boundaries rather than a single technology checkbox.

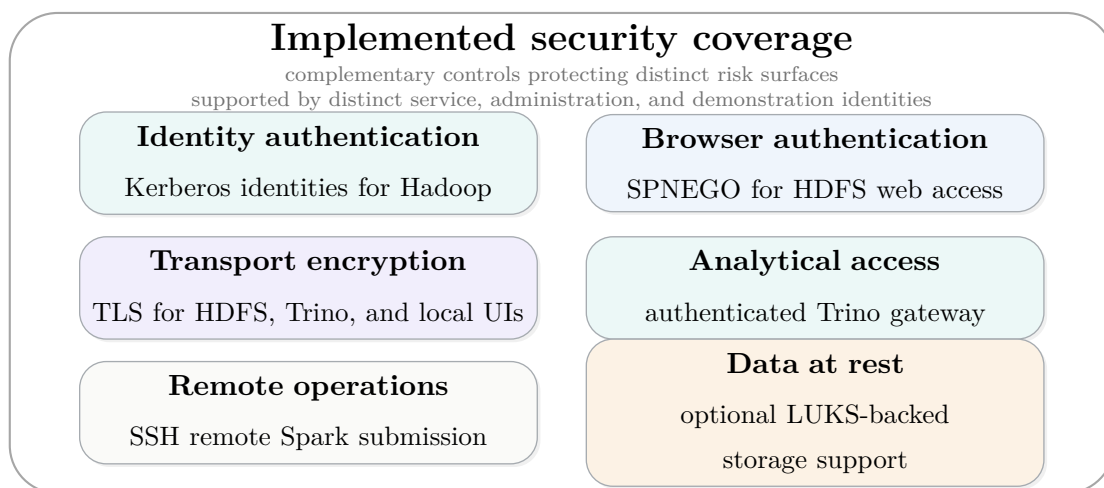


Figure VII.6: Implemented security coverage across complementary protection dimensions.

VII.9 Security Boundaries and Remaining Limits

The implemented security layer provides a meaningful distributed-platform hardening baseline; it does not constitute a finished enterprise security program.

The current implementation can validly claim:

- Kerberos-secured Hadoop runtime;
- SPNEGO-protected HDFS browser access;
- TLS-protected browser-facing endpoints;
- authenticated Trino access;
- SSH-based remote Spark submission;

- distinct service, administration, and demonstration identities;
- environment-scoped credential handling;
- optional encrypted runtime storage support on the Linux nodes.

At the same time, some limits remain outside the strict PFE scope:

- enterprise-scale PKI and certificate lifecycle automation;
- full centralized secret management;
- fine-grained authorization frameworks across all analytical consumers;
- large-scale security auditing, SIEM integration, or intrusion detection;
- organization-wide identity federation.

These boundaries distinguish the controls implemented in the project from future enterprise hardening work.

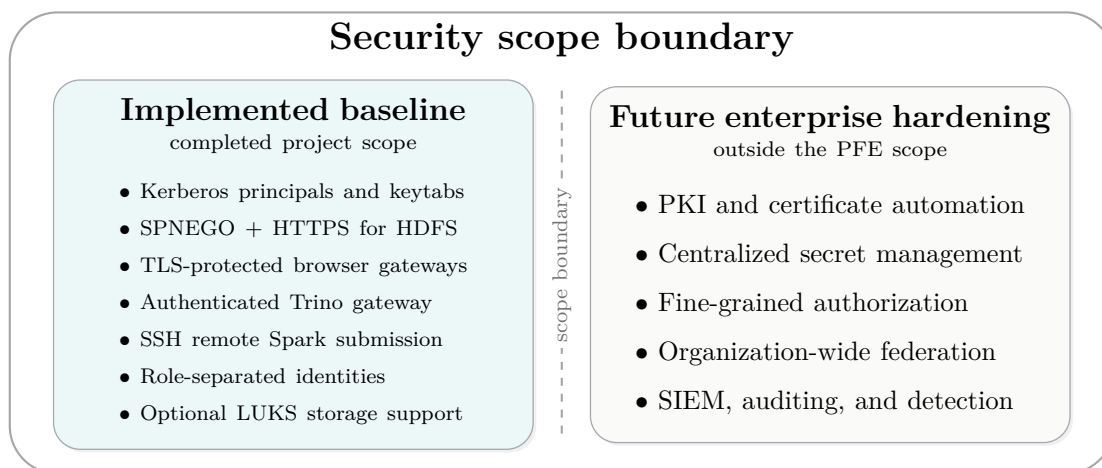


Figure VII.7: Implemented security baseline and enterprise controls outside the PFE scope.

VII.10 Conclusion

This chapter demonstrated a layered security baseline comprising Kerberos principals and keytabs, SPNEGO/HTTPS access to HDFS, TLS-protected browser gateways, authenticated Trino entry, SSH-based Spark submission, separated operational identities, and optional LUKS-backed storage. These controls harden the distributed deployment without being presented as a complete enterprise IAM program. Chapter VIII examines the resulting readiness, pipeline, observability, and security evidence.

Chapter VIII

Validation, Testing, and Results

VIII.1 Introduction

This chapter presents the validation methodology and the main technical results of AdOptimizer CDP. Validation was designed to determine whether the implemented platform was operational across its complete execution path: service readiness, orchestration, distributed storage and processing, governed transformation, data-quality control, analytical access, observability, and secured interaction. The assessment therefore relies on retained runtime evidence rather than architectural intent alone [?, ?, ?].

The quantitative values reported in this chapter correspond to the retained validation capture of August 18, 2026. They represent the observed state of that execution cycle rather than permanent capacity or performance guarantees.

VIII.2 Validation Strategy

The validation campaign covered six complementary axes, summarized in Table ??.

The strategy followed a gated progression. Infrastructure readiness was assessed first because later results are not reliable when storage, metadata, compute, query, identity, or monitoring services are unavailable. The five Airflow workflows were then checked in their implemented order. Multi-node participation was evaluated separately from workflow completion so that a successful task could not be mistaken for proof of distributed execution.

Acceptance combined Airflow histories for execution state, service interfaces for cluster roles, persisted artifacts for durable outputs, and external Trino queries for analytical usability. Dashboard values were treated as time-bound observations, and conclusions were limited to capabilities documented in Chapter IV and Appendix ??.

Validation axis	What was checked	Acceptance evidence
Infrastructure	service endpoints, cluster membership, readiness, and storage state	reachable services and healthy distributed members
Orchestration	five-DAG execution order, task completion, and rerun behavior	successful workflow histories and controlled lifecycle transitions
Distributed execution	Spark workers, HDFS DataNodes, Kafka brokers, and Trino roles	runtime participation across separate virtual machines
Transformation and quality	dbt models, tests, Great Expectations checkpoints, and retained artifacts	curated tables, validation JSON, and HTML Data Docs
Security	Kerberos, SPNEGO/HTTPS, gateways, SSH, and identity boundaries	functioning protected access paths with documented scope limits
Observability and access	metrics, dashboards, run history, and external SQL querying	measurable service state and successful analytical consumption

Table VIII.1: Validation strategy adopted for the deployed platform

VIII.3 Infrastructure Validation

Infrastructure validation established the operational baseline before pipeline results were interpreted. The campaign covered Kafka, HDFS, Hive Metastore, Spark, Trino, Kerberos, the browser-facing gateways, and the monitoring stack [?, ?, ?, ?, ?, ?, ?, ?].

The retained checks confirmed that:

- Kafka brokers were reachable on VM1, VM2, and VM3;
- HDFS exposed an active NameNode and three live DataNodes;
- Hive Metastore was available as the shared metadata authority;
- the Spark master, three workers, Thrift Server, and History Server were reachable;
- the Trino coordinator and its two dedicated workers were available through the protected entry path;
- Kerberos and the secured HDFS endpoints were operational;
- Prometheus, Grafana, and the configured exporters were reachable from the control plane.

The HDFS namespace contained two principal branches: **/bronze** for replayable raw ingestion and **/warehouse** for governed analytical storage. The client-specific bronze branch contained the three retained datasets, while the Iceberg warehouse contained the raw, staging, intermediate, and analytics namespaces.

Observed HDFS indicator	Retained runtime result
DataNode state	3 live nodes, 0 dead nodes
Configured storage capacity	62.97 GB
DFS used	918.83 MB (1.42%)
DFS remaining	7.22 GB (11.46%)
Filesystem objects	338 objects: 196 files/directories and 142 replicated blocks
DataNode usage distribution	minimum 1.19%, median 1.48%, maximum 1.61%, standard deviation 0.18%
Replication health	0 under-replicated blocks and 0 pending deletions
Storage type	DISK across 3 nodes in service

Table VIII.2: HDFS runtime indicators retained from the distributed deployment

Configured capacity also includes host storage classified outside DFS. Consequently, DFS used and DFS remaining should not be interpreted as a direct arithmetic decomposition of the configured total.

Observed path	Observed objects	Validation meaning
/	/bronze, /warehouse	separates replayable persistence from governed analytical storage
/bronze/client_1	bank_marketing, online_retail_2, online_shoppers_intention	confirms materialized bronze data for the three retained sources
/warehouse/iceberg	raw_data.db, staging.db, intermediate.db, analytics.db	confirms the physical lakehouse-layer progression

Table VIII.3: HDFS namespace evidence retained from the validated deployment state

Together, the service and namespace observations establish that the storage layer was active, populated, distributed, and free from immediate replication anomalies during the retained capture.

VIII.4 Pipeline, Transformation, and Quality Validation

Pipeline validation followed the five prioritized Airflow workflows implemented for the final platform:

1. `customerdna_client1_environment_reset_pipeline;`
2. `customerdna_client1_lakehouse_setup_pipeline;`
3. `customerdna_client1_lakehouse_readiness_pipeline;`
4. `customerdna_client1_kafka_hdfs_spark_lakehouse_pipeline;`
5. `customerdna_client1_dbt_spark_lakehouse_pipeline.`

This decomposition validates lifecycle control without relying on one monolithic workflow. Reset returns the platform to a controlled baseline, setup initializes the required storage and lakehouse structures, readiness prevents deeper execution against unavailable services, the Kafka–HDFS–Spark workflow materializes the distributed raw foundation and executes its embedded Great Expectations tasks, and the dbt workflow builds and tests the governed models [?].

Validated workflow	Retained result	Evidence retained
customerdna_client1_environment_reset_pipeline	successful controlled reset	Airflow task history and clean-state checks for Kafka, bronze storage, and lakehouse state
customerdna_client1_lakehouse_setup_pipeline	successful lakehouse initialization	creation and verification of the bronze namespace and required lakehouse structures
customerdna_client1_lakehouse_readiness_pipeline	successful readiness validation	positive checks for HDFS, Hive, Spark, Spark Thrift, and Trino
customerdna_client1_kafka_hdfs_spark_lakehouse_pipeline	successful raw loading and integrated quality validation	Kafka activity, bronze <code>.jsonl</code> files, Spark applications, raw Iceberg tables, validation JSON, and Data Docs
customerdna_client1_dbt_spark_lakehouse_pipeline	successful model execution	dbt build/test traces and materialized staging, intermediate, and analytics tables

Table VIII.4: Retained validation evidence for the five operational workflows

The retained Airflow history demonstrates repeatable, dependency-aware execution rather than a one-time manual sequence. Each lifecycle stage can be diagnosed and rerun at its workflow boundary, while the raw-quality checks remain separately inspectable as named tasks within the fourth workflow.

Transformation and Data-Quality Evidence

Transformation validation confirmed that the raw Iceberg tables could be converted into the implemented staging, intermediate, and analytics model layers through dbt-spark. The retained execution traces included model builds and dbt tests, while Trino inspection confirmed that the resulting governed tables were queryable.

Great Expectations validated the three raw-lakehouse source families through the active checkpoint definitions shown in Table ?? . The fourth DAG exposes two explicit quality-task boundaries:

- `bootstrap_raw_lakehouse_quality_assets;`
- `validate_raw_lakehouse_quality.`

Each execution produced machine-readable validation JSON and HTML Data Docs, separating quality evidence from Airflow task status alone [?, ?].

The retained dashboard state also reported three refreshed raw outputs with no skipped or partial incremental outputs. Across the three datasets, approximately 1.12 million raw

Source family	Active checkpoint definition	Retained evidence
Bank Marketing	<code>raw_lakehouse_quality_checkpoint__bank_marketing.json</code>	validation JSON and Data Docs
Online Shoppers Intention	<code>raw_lakehouse_quality_checkpoint__online_shoppers_intention.json</code>	validation JSON and Data Docs
Online Retail II	<code>raw_lakehouse_quality_checkpoint__online_retail_2.json</code>	validation JSON and Data Docs

Table VIII.5: Source-specific Great Expectations evidence retained by the integrated raw-pipeline validation tasks

records were visible: about 41.2K bank-marketing rows, 1.07M online-retail rows, and 12.3K online-shopping-session rows.

VIII.5 Distributed Execution Validation

Distributed execution was assessed through the runtime roles of Spark, HDFS, Kafka, and Trino. The Spark master registered workers on VM1, VM2, and VM3; HDFS stored blocks through DataNodes on all three machines; Kafka brokers were distributed across the same nodes; and Trino separated coordination on VM2 from query workers on VM1 and VM3.

The retained Spark capture recorded three live workers, three total cores, 3.0 GiB of aggregate worker memory, an active Thrift service, and twelve completed applications associated with client-data loading and serving. Workload allocation was not identical across every worker, which is expected for light jobs; the relevant result is that executor capacity existed beyond the coordinator node [?].

Distributed indicator	Observed runtime result
Registered Spark workers	3 alive workers on VM1, VM2, and VM3; 0 dead and 0 decommissioned
Spark capacity at capture time	3 total cores, 1 active core; 3.0 GiB total worker memory, 512 MiB used
Spark SQL service	Thrift JDBC/ODBC Server active on the cluster
Completed Spark applications	12 completed applications visible in the retained master/history views
Raw-to-Iceberg jobs	successful applications for <code>bank_marketing</code> , <code>online_retail_2</code> , and <code>online_shoppers_intention</code>
HDFS participation	3 live DataNodes with replicated blocks across the cluster
Kafka participation	one broker on each of VM1, VM2, and VM3
Trino participation	coordinator on VM2 and two dedicated workers on VM1 and VM3

Table VIII.6: Runtime indicators of multi-node storage, transport, processing, and querying

VIII.6 Security Validation

Security validation checked whether the hardened paths described in Chapter VII were usable in the deployed environment [?, ?, ?, ?, ?].

Security path	Validated result	Scope interpretation
Kerberos-secured Hadoop	service principals, keytabs, and authenticated Hadoop interactions were operational	implemented service-identity baseline
HDFS browser access	Windows Kerberos ticket and Firefox SPNEGO reached HTTPS-only HDFS interfaces	client-sensitive path requiring documented hostnames and browser configuration
TLS browser exposure	HDFS, Trino, and control-plane interfaces used protected browser entry paths	internal/self-signed trust rather than enterprise PKI
Trino analytical access	authenticated gateway allowed controlled access to the coordinator path	gateway authentication, not full enterprise IAM
Spark submission	Airflow reached the Spark submit client on VM2 through SSH	authenticated control-plane/data-plane bridge
Operational identities	service, administration, and demonstration identities were separated	baseline role separation without centralized secret management
Runtime storage	LUKS-backed deployment support was available for VM runtime directories	optional data-at-rest hardening when enabled

Table VIII.7: Validated security paths and their implementation boundaries

The Windows/Firefox SPNEGO workflow depends on the client ticket cache, hostname aliases, and browser negotiation settings. Client-side integration failures must therefore be distinguished from the health of the Kerberos-secured Hadoop back end.

VIII.7 Observability and Analytical Access Validation

Observability validation confirmed that platform diagnosis was not limited to manual container-log inspection. Prometheus collected infrastructure, exporter, container, and pipeline metrics; Grafana exposed the retained dashboards; cAdvisor supplied node/container observations from all three VMs; and Airflow retained task histories and logs [?, ?, ?].

Analytical access was validated independently from the transformation engines through an external SQL client. The retained query targeted the Online Retail II raw table through Trino and completed successfully.

Monitoring indicator	Observed metric or state
Prometheus target health	6 healthy targets
Observed host capacity	12 CPU cores and 11.46 GiB memory
Hive Metastore PostgreSQL	available, 11.4 MiB database size, 44 active connections at capture time
Pipeline status	retained workflow and quality indicators reported successful completion
Latest visible raw volume	approximately 1.12 million records across the three retained datasets
Raw load outcomes	3 refreshed outputs, 0 skipped, and 0 partial incremental outputs
Great Expectations evidence	successful source-specific checkpoint artifacts retained for the raw lakehouse

Table VIII.8: Observed monitoring evidence supporting operational visibility

Analytical-access indicator	Observed runtime result
Trino version and environment	version 435, environment <code>DISTRIBUTED_DEMO</code>
Coordinator uptime	6.74 days at capture time
Distributed query roles	coordinator on VM2; dedicated workers on VM1 and VM3
Query state	external SQL query completed with state <i>Finished</i>
Query throughput snapshot	89.0 rows/s and 242K bytes/s
Query target	<code>lakehouse.raw_data.online_retail_2</code> , limited to 200 rows
Consumption result	governed lakehouse tables were browsable from an external SQL client

Table VIII.9: Runtime evidence that the distributed analytical-access layer was usable

VIII.8 Main Results

The validation campaign confirmed the delivery of a distributed, orchestrated, governed, observable, and security-aware customer-data lakehouse platform under the AdOptimizer CDP scope.

The principal result is the demonstrated continuity of the implemented data path. The retained datasets were transported through Kafka, persisted in the HDFS bronze zone, materialized as governed Iceberg tables, transformed through the dbt model layers, and consumed externally through Trino. Consequently, the analytical-access result was not produced by an isolated demonstration dataset; it depended on the same storage, metadata, processing, and orchestration chain validated throughout the chapter.

Completion was also established through complementary forms of evidence. Airflow histories confirmed workflow order and task state, cluster interfaces exposed the participating distributed services, persisted files and tables demonstrated durable outputs, Great Expectations retained independently inspectable validation artifacts, and Trino supplied a consumer-side query result. This triangulation reduces the risk of interpreting a successful task status or an available endpoint as sufficient proof of end-to-end platform behavior.

Result dimension	Observed completion evidence
Orchestration lifecycle	the five workflows covered reset, setup, readiness, integrated raw loading and quality validation, and governed transformation
Distributed storage	3 live DataNodes, populated bronze and warehouse namespaces, 62.97 GB configured capacity, and no immediate replication anomalies
Distributed compute	the Spark master registered workers on all three VMs and retained 12 completed applications
Governed transformation	dbt materialized staging, intermediate, and analytics models and executed associated tests
Data quality	three source-specific Great Expectations checkpoints produced validation JSON and HTML Data Docs
Analytical access	Trino served a completed external SQL query through its coordinator and two dedicated workers
Observability	Prometheus and Grafana exposed healthy targets, infrastructure capacity, metastore state, pipeline status, and quality indicators
Data volume	approximately 1.12 million raw records were visible across the three retained datasets
Security posture	Kerberos, SPNEGO/HTTPS, authenticated gateways, SSH submission, role separation, and optional LUKS support hardened the deployed baseline

Table VIII.10: Consolidated validation results for the implemented platform

The validated platform is a **Customer 360-ready analytical foundation**, not a finished enterprise AI product or customer master. The retained results show that heterogeneous data could enter through Kafka, persist in distributed bronze storage, become governed lakehouse tables, pass modeled transformations and quality controls, remain queryable through Trino, and expose operational evidence through the monitoring stack. These outcomes establish the completed Data Engineering and Big Data contribution while preserving the boundary between the implemented platform and future AdOptimizer AI capabilities.

VIII.9 Conclusion

This chapter confirmed that AdOptimizer CDP was tested as an integrated operational platform rather than as a collection of independently reachable services. Readiness, orchestration, multi-node participation, transformation, quality, analytical access, security, and observability were assessed through retained runtime evidence.

Chapter IX builds on these results to examine the architectural strengths, trade-offs, limitations, and industrial relevance of the implemented solution.

Chapter IX

Discussion and Critical Analysis

IX.1 Introduction

Chapter VIII established that AdOptimizer CDP operated as an integrated distributed customer-data platform. The purpose of the present chapter is not to repeat those validation results, but to interpret their significance. The discussion examines the coherence of the architectural choices, the trade-offs introduced by the selected stack and deployment model, the lessons derived from implementation, and the limits within which the observed results remain valid.

The analysis follows the principles commonly associated with distributed data-intensive systems and lakehouse architectures: explicit responsibility boundaries, durable data, independent compute and storage concerns, shared metadata, reproducible processing, and controlled operational exposure [?, ?]. The conclusions are deliberately restricted to the implemented three-VM data plane, the local control plane, and the retained evidence presented in the preceding chapter.

IX.2 Critical Assessment of the Architectural Decisions

The platform was designed as a chain of cooperating capabilities rather than as a single processing application. This separation made it possible to assign ingestion, persistence, computation, cataloging, transformation, validation, orchestration, observation, and protection to explicit architectural layers. The resulting modularity is valuable because a change in one concern does not necessarily require redesigning the entire platform. However, modularity also creates more interfaces, configuration contracts, and runtime dependencies that must remain mutually consistent.

Architectural decision	Value obtained	Associated trade-off
Kafka transport and HDFS bronze persistence	decoupled source publication from durable, replayable raw history	duplicated operational responsibilities and additional storage and retention management
Spark with Iceberg and Hive Metastore	distributed processing over governed tables shared by transformation and query services	compatibility and availability depend on coordinated engine, catalog, and storage configuration
dbt-spark and Great Expectations as separate controls	distinguished modeled transformation from explicit, retained data-quality evidence	introduced separate projects, runtimes, artifacts, and failure paths
Local control plane and Linux data plane	isolated orchestration and supervision from distributed data services	made network resolution, SSH access, credentials, and remote readiness part of every execution path
Multi-node service placement	demonstrated genuine participation in storage, processing, transport, and querying	retained several central coordinator roles and did not constitute a high-availability topology
Security and observability as cross-cutting layers	made protected access and runtime evidence part of normal platform operation	increased certificate, ticket, gateway, exporter, and configuration-management overhead

Table IX.1: Benefits and trade-offs of the principal architectural decisions

The strongest result of these decisions is the creation of an end-to-end engineering contract. A source is not considered usable merely because it reaches Kafka; it must also be durably landed, transformed into governed tables, validated, exposed analytically, and accompanied by operational evidence. This contract gives the platform more value than a collection of independently functioning services.

IX.3 Strengths Supported by the Implementation

IX.3.1 End-to-End Coherence

The implementation connects the complete data lifecycle. The same three source perspectives—campaign response, digital behavior, and retail transactions—move through transport, bronze persistence, raw lakehouse loading, raw-data validation, modeled transformation, and analytical access. The five Airflow workflows make the lifecycle explicit through environment reset, lakehouse setup, service readiness, a Kafka–HDFS–Spark workflow with integrated raw-quality tasks, and governed dbt transformation. This decomposition supports repeatability and makes failures easier to localize than in a single opaque workflow.

IX.3.2 Distributed Execution with Shared Governance

The deployment demonstrates distribution in the areas where it is technically meaningful: Kafka brokers span the three virtual machines, HDFS stores data through three DataNodes, Spark registers workers on all three machines, and Trino separates its coordinator from two dedicated workers. At the same time, Hive Metastore and Iceberg provide a shared table interpretation across processing and query services. This combination is important because distribution without consistent metadata would only move computation across nodes; it would not create a governed analytical platform [?, ?].

IX.3.3 Reproducibility and Evidence

Readiness checks, ordered orchestration, dbt tests, Great Expectations artifacts, Airflow histories, and monitoring dashboards make execution outcomes inspectable. The platform therefore preserves evidence not only of resulting data, but also of the operational path that produced it. This strengthens reproducibility and separates a demonstrable engineering result from an implementation that can only be explained from source code or manual observation.

IX.3.4 Security-Aware Operational Design

Security was integrated into deployed paths rather than left as a conceptual recommendation. Kerberos-secured Hadoop interactions, SPNEGO/HTTPS browser access, authenticated gateways, SSH-based submission, and separated operational identities reduce the openness of the baseline cluster. Their value lies in demonstrating how security affects service communication and deployment practice. The result remains infrastructure hardening rather than a complete enterprise identity-governance system, but it is nevertheless part of the functioning architecture.

IX.4 Trade-Offs and Operational Constraints

IX.4.1 Resource Capacity versus Architectural Breadth

The three virtual machines provide sufficient separation to demonstrate distributed behavior, but their limited CPU, memory, and storage constrain concurrency and workload scale. The breadth of the stack also means that infrastructure resources are shared by storage, processing, query, metadata, security, and monitoring services. Consequently,

the deployment proves architectural feasibility and integration; it does not establish the maximum throughput or capacity of the design.

IX.4.2 Distribution versus Centralization

Several worker-oriented capabilities are distributed, while coordination remains intentionally centralized. VM2 hosts central roles including the HDFS NameNode, Spark master, Hive Metastore, and Trino coordinator, and the local machine hosts Airflow and the principal monitoring services. This arrangement is appropriate for a resource-constrained implementation, but the loss of a coordinator host or the control plane would interrupt important platform capabilities. The deployment must therefore be described as multi-node and distributed, not as fully redundant or highly available.

IX.4.3 Specialized Components versus Integration Cost

Each selected component has a clear responsibility, yet every boundary creates an integration obligation. File paths, schemas, catalog identifiers, Kerberos principals, certificates, hostnames, ports, and service readiness must agree across independently configured systems. Much of the engineering difficulty consequently lies between tools rather than inside them. This is a characteristic trade-off of distributed data platforms: specialized services improve separation of concerns while increasing the cost of maintaining reliable interfaces [?].

IX.4.4 Controlled Batch Processing versus Real-Time Semantics

Kafka provides event transport, but the validated lifecycle is an orchestrated sequence in which the raw-loading workflow includes explicit quality tasks and is followed by a governed transformation workflow. The implementation should therefore not be interpreted as a continuously updated real-time customer profile or an event-driven personalization engine. Its current strength is controlled, replayable, and observable batch-oriented processing over heterogeneous customer datasets.

IX.4.5 Hardening versus Operational Simplicity

Kerberos tickets, keytabs, SPNEGO client settings, TLS trust, gateways, and optional encrypted runtime storage improve the security posture while making deployment and troubleshooting more sensitive. Internal or self-signed trust paths require deliberate client configuration, gateway authentication is not equivalent to enterprise IAM, and LUKS

support only protects runtime storage when enabled. These boundaries prevent the security layer from being overstated while preserving its implemented value.

IX.5 Engineering Lessons Learned

Several lessons emerged from constructing and validating the platform.

1. **Interfaces are as important as individual services.** Successful component startup is insufficient when schemas, metadata, identities, or network assumptions disagree across service boundaries.
2. **Readiness must precede data movement.** Separating environment preparation and lakehouse readiness from data-processing workflows reduces ambiguous failures and makes execution order reproducible.
3. **Raw durability and analytical modeling serve different purposes.** HDFS bronze artifacts preserve replayability, while Iceberg, dbt-spark, and the curated layers provide governed analytical structure. Combining these responsibilities would weaken both.
4. **Validation evidence is a platform output.** Great Expectations results, dbt tests, workflow histories, and monitoring metrics are not auxiliary screenshots; they are retained proof that the platform executed its contracts.
5. **Security changes architecture.** Authentication, protected browser paths, remote submission, and credential separation affect hostnames, protocols, startup order, and operational procedures. They are most effective when designed with the deployment rather than added after it.
6. **A distributed demonstration still requires explicit claim boundaries.** Multi-node participation proves that work crosses machine boundaries, but it does not by itself prove high availability, production capacity, or indefinite fault tolerance.

IX.6 Academic and Industrial Contribution

Academically, the contribution lies in the integration and validation of concepts that are often examined separately. The work combines distributed ingestion, storage, computation, and querying with lakehouse table management, workflow orchestration, data-quality evidence, observability, and infrastructure security. It demonstrates how abstract principles

such as replayability, separation of concerns, shared metadata, and controlled service exposure become concrete deployment decisions [?, ?, ?].

Industrially, AdOptimizer CDP provides SMART AUTOMATION TECHNOLOGIES with an implemented reference foundation for the broader AdOptimizer AI vision. The value is not a completed AI product, but a repeatable path for turning heterogeneous customer-related sources into governed and queryable data products. The use of explicit service boundaries also makes the architecture transferable: components can be scaled, replaced, or managed differently without changing the central lifecycle from ingestion to trusted analytical output.

This transferability has an operational cost. A wider deployment would require ownership of service upgrades, compatibility testing, incident response, access governance, certificate and secret lifecycles, capacity planning, and recovery procedures. The implementation therefore supplies an architectural foundation, while industrialization would require additional operational processes around it.

IX.7 Limitations and Validity Boundaries

The results must be interpreted within the conditions under which they were produced. The following limitations define what the implementation demonstrates and what remains outside its validated scope.

Boundary	Current validated scope	Claim that cannot yet be made
Workload representativeness	three retained customer-oriented datasets and approximately 1.12 million visible raw records	sustained enterprise ingestion capacity or production-scale performance
Availability	successful coordinated operation across a three-VM data plane and local control plane	high availability under coordinator, node, or control-plane failure
Performance evaluation	runtime and resource snapshots from the implemented environment	statistically repeatable speedup or superiority over an equivalent local baseline
Customer unification	complementary campaign, behavioral, and transactional analytical perspectives	production-grade cross-source identity resolution or an enterprise customer master
Processing mode	replayable, orchestrated loading and transformation workflows	continuously updated real-time profiles or low-latency decision serving
Security maturity	implemented Kerberos, protected browser paths, authenticated gateways, SSH submission, identity separation, and optional LUKS support	enterprise IAM, centralized secrets management, public-key infrastructure automation, or universal data-at-rest encryption

Table IX.2: Validated scope and limits of interpretation

These limitations do not negate the completed implementation. They delimit the evidence: the project establishes a functioning, distributed, governed, observable, quality-aware, and security-aware customer-data lakehouse foundation. It does not establish that every production-scale, availability, identity-resolution, or enterprise-governance requirement has already been satisfied.

IX.8 Conclusion

The critical analysis shows that the principal value of AdOptimizer CDP lies in architectural coherence. Heterogeneous source handling, distributed data services, governed transformation, quality evidence, orchestration, observability, and protected access were connected into one operational lifecycle. The same choices also create real costs in resource consumption, integration effort, centralized coordination, and security administration.

The project should therefore be understood as a completed Data Engineering and Big Data platform implementation with clearly bounded evidence. It is a credible Customer 360-ready analytical foundation within the broader AdOptimizer AI direction, but not a production-scale customer master or finished AI system. Chapter X concludes the report and converts the identified boundaries into focused perspectives for the platform's continued evolution.

Chapter X

General Conclusion and Perspectives

X.1 General Conclusion

Customer-related information is frequently fragmented across campaign systems, digital interaction channels, and transactional applications. This fragmentation weakens replayability, governance, analytical trust, and the ability to prepare reliable inputs for advanced customer-oriented services. The central problem addressed by this project was therefore to determine how heterogeneous customer data could be integrated through a distributed, governed, observable, and protected engineering platform rather than through isolated scripts or a single local database.

To answer this problem, the project designed, implemented, deployed, and validated AdOptimizer CDP within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES. The resulting platform connects Kafka ingestion, HDFS bronze persistence, Spark processing, Iceberg table management, Hive Metastore governance, Trino analytical access, dbt-spark transformations, Great Expectations validation, Airflow orchestration, and Prometheus/Grafana observability. These capabilities operate across a local control plane and a three-VM Linux data plane, with security controls integrated into the deployed access and execution paths.

The implementation demonstrates that campaign-response, digital-behavior, and retail-transaction datasets can follow one reproducible lifecycle: source publication, replayable raw landing, distributed loading, integrated raw-data validation, governed transformation and testing, and external SQL consumption. The five-workflow orchestration sequence—reset, setup, readiness, raw loading with embedded quality tasks, and dbt transformation—together with retained validation artifacts and monitoring evidence shows that the result is an operational platform rather than a collection of independently demonstrated technologies.

The original problem is therefore answered by a coherent Data Engineering and Big Data architecture that turns heterogeneous source data into governed and reusable analytical products. Its contribution is consistent with the principles of distributed data-intensive and lakehouse systems: durable raw history, separation of responsibilities, shared table metadata, independent processing and query services, and evidence-driven operation [?, ?, ?].

The final positioning must nevertheless remain precise. AdOptimizer CDP is a completed distributed customer-data lakehouse implementation and a Customer 360-ready analytical foundation. It is not yet a production-scale customer master, a highly available enterprise platform, or a finished AI product. Those boundaries do not reduce the delivered contribution; they define the point from which the platform can evolve responsibly.

X.2 Main Contributions

The project contributes at several complementary levels.

Contribution dimension	Implemented contribution	Resulting value
Customer-data integration	one governed lifecycle for three heterogeneous campaign, behavioral, and transactional perspectives	a credible analytical basis for broader Customer 360 development
Distributed architecture	multi-node transport, storage, processing, and querying across the three-VM data plane	practical demonstration that the main data workloads cross machine boundaries
Lakehouse governance	replayable HDFS bronze artifacts, Iceberg tables, shared Hive metadata, and layered dbt models	separation between durable raw history and reusable analytical structures
Quality and reproducibility	dbt tests, source-specific Great Expectations checkpoints, validation JSON, and Data Docs	retained evidence that transformations and data contracts were checked
Operational control	five ordered Airflow workflows covering reset, setup, readiness, integrated raw loading/quality, and dbt transformation, complemented by metrics and Grafana dashboards	repeatable execution, diagnosable failures, and runtime visibility
Deployment hardening	Kerberos-secured Hadoop services, protected browser paths, authenticated gateways, SSH submission, identity separation, and optional encrypted runtime storage	a security-aware baseline integrated into normal platform operation

Table X.1: Synthesis of the principal project contributions

Taken together, these contributions establish the project primarily as a platform-engineering achievement. The value does not come from any individual framework in isolation, but from the consistency of the complete path between source data, distributed infrastructure,

governed analytical outputs, validation evidence, and controlled access.

X.3 Future Perspectives

The next developments should preserve the current architecture's separation of concerns while increasing its availability, data coverage, governance maturity, and downstream business value. The perspectives can be organized into three successive directions.

X.3.1 Platform Industrialization

The first priority is to strengthen the operating foundation before expanding business functionality. Central coordinator and catalog responsibilities should be made more resilient, and the local control plane should evolve toward a managed deployment environment. Backup, restoration, failover, and node-loss scenarios should be tested explicitly rather than inferred from successful nominal execution.

Deployment automation can also be extended through versioned environment configuration, infrastructure provisioning, compatibility checks, and repeatable certificate and service-identity initialization. Centralized secret management, automated certificate rotation, finer-grained access control, and encryption enabled consistently across persistent runtime storage would move the current security baseline closer to enterprise operation.

Performance evaluation should become systematic. Repeated benchmarks over identical datasets and workflows can compare local and distributed execution, measure end-to-end duration and service-level throughput, and identify CPU, memory, storage, and network bottlenecks. Such measurements would guide capacity planning without replacing the functional evidence already established by the project.

X.3.2 Customer 360 Enrichment

The second direction is to broaden the customer-data foundation. Additional operational sources can extend the current campaign, behavioral, and transactional perspectives with support interactions, consent and preference data, product usage, and other company-relevant channels. Each new source should retain the same ingestion, bronze, transformation, quality, and evidence contracts.

A mature Customer 360 layer would also require canonical identifiers, controlled cross-source identity resolution, survivorship rules, and explicit handling of uncertain matches. Richer semantic marts, documented data contracts, lineage, retention rules, and access policies would turn the current analytical foundation into a more complete governed

customer domain. Incremental and streaming transformations could then reduce data freshness where business requirements justify lower latency.

X.3.3 AdOptimizer AI Capabilities

The third direction concerns the intelligent services of the broader AdOptimizer AI vision. Curated customer features can support segmentation, persona generation, churn-risk estimation, customer-lifetime-value modeling, conversion analysis, campaign optimization, recommendation, and personalization. These services should consume governed outputs rather than bypass the platform and rebuild source integrations independently.

Moving from analytical features to operational AI would require a dedicated lifecycle for training, evaluation, versioning, deployment, monitoring, and retraining. Feature consistency, model drift, explainability, privacy, and decision traceability would become additional requirements. In this evolution, AdOptimizer CDP remains the trusted data foundation, while machine-learning and decision services form downstream products with their own validation responsibilities.

Evolution direction	Priority objectives	Expected outcome
Platform industrialization	resilience, automated deployment, secrets and certificate lifecycles, recovery testing, and repeatable benchmarking	a more scalable, recoverable, and operationally controlled platform
Customer 360 enrichment	additional sources, identity resolution, semantic models, lineage, policies, and lower-latency processing where required	a governed and increasingly unified customer-information domain
AdOptimizer AI capabilities	governed features, model lifecycle management, monitoring, and decision-service integration	trustworthy intelligent marketing services consuming validated platform outputs

Table X.2: Prioritized evolution path from the implemented platform to downstream intelligent services

X.4 Closing Perspective

AdOptimizer CDP demonstrates that trustworthy customer analytics depend on a durable, governed, observable, and protected data foundation. By integrating distributed execution, controlled transformation, explicit validation, and secured access, the completed platform gives SMART AUTOMATION TECHNOLOGIES a credible basis for extending AdOptimizer AI toward Customer 360 and future AI services while preserving reproducibility, governance, evidence, security, and scope discipline.

Bibliography

- [1] Apache Software Foundation. Apache Airflow documentation.
<https://airflow.apache.org/docs/>. Accessed August 23, 2026.
- [2] Apache Software Foundation. Apache Flink documentation.
<https://nightlies.apache.org/flink/flink-docs-stable/>. Accessed August 23, 2026; used for distributed-processing comparison.
- [3] Apache Software Foundation. Apache Hadoop documentation.
<https://hadoop.apache.org/docs/>. Accessed August 23, 2026.
- [4] Apache Software Foundation. Apache Hive documentation.
<https://hive.apache.org/docs/latest/>. Accessed August 23, 2026.
- [5] Apache Software Foundation. Apache Hudi documentation.
<https://hudi.apache.org/docs/>. Accessed August 23, 2026; used for lakehouse-table-format comparison.
- [6] Apache Software Foundation. Apache Iceberg documentation.
<https://iceberg.apache.org/docs/latest/>. Accessed August 23, 2026.
- [7] Apache Software Foundation. Apache Kafka documentation.
<https://kafka.apache.org/documentation/>. Accessed August 23, 2026.
- [8] Apache Software Foundation. Apache Parquet documentation.
<https://parquet.apache.org/docs/>. Accessed August 23, 2026.
- [9] Apache Software Foundation. Apache Pulsar documentation.
<https://pulsar.apache.org/docs/>. Accessed August 23, 2026; used for messaging-technology comparison.
- [10] Apache Software Foundation. Apache Spark documentation.
<https://spark.apache.org/documentation.html>. Accessed August 23, 2026.
- [11] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In

- 11th Annual Conference on Innovative Data Systems Research (CIDR '21)*, Online, January 2021.
https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf.
- [12] Broadcom. RabbitMQ documentation. <https://www.rabbitmq.com/docs>. Accessed August 23, 2026; used for messaging-technology comparison.
- [13] cryptsetup Project. cryptsetup and LUKS documentation. <https://gitlab.com/cryptsetup/cryptsetup/-/wikis/home>. Accessed August 23, 2026.
- [14] Dagster Labs. Dagster documentation. <https://docs.dagster.io/>. Accessed August 23, 2026; used for workflow-orchestration comparison.
- [15] dbt Labs. dbt developer hub documentation. <https://docs.getdbt.com/>. Accessed August 23, 2026.
- [16] Delta Lake Project. Delta Lake documentation. <https://docs.delta.io/>. Accessed August 23, 2026; used for lakehouse-table-format comparison.
- [17] Docker, Inc. Docker documentation. <https://docs.docker.com/>. Accessed August 23, 2026.
- [18] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, June 2002. <https://doi.org/10.1145/564585.564601>.
- [19] Google. cAdvisor project documentation. <https://github.com/google/cadvisor>. Accessed August 23, 2026.
- [20] Grafana Labs. Grafana documentation. <https://grafana.com/docs/grafana/latest/>. Accessed August 23, 2026.
- [21] Great Expectations. Great Expectations documentation. <https://docs.greatexpectations.io/>. Accessed August 23, 2026.
- [22] Martin Kleppmann and Chris Riccomini. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, Inc., 2nd edition, February 2026. ISBN 978-1-0981-1905-8.
- [23] Jay Kreps. Questioning the lambda architecture. O’Reilly Radar, <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>, July 2014. Accessed August 23, 2026.

- [24] Doug Laney. 3D data management: Controlling data volume, velocity, and variety. Application Delivery Strategies 949, META Group, February 2001.
- [25] Nathan Marz and James Warren. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, April 2015. ISBN 978-1-61729-034-3.
- [26] Massachusetts Institute of Technology. MIT Kerberos documentation.
<https://web.mit.edu/kerberos/krb5-latest/doc/>. Accessed August 23, 2026.
- [27] OpenSSH Project. OpenSSH manual pages.
<https://www.openssh.org/manual.html>. Accessed August 23, 2026.
- [28] Prefect Technologies, Inc. Prefect documentation. <https://docs.prefect.io/>. Accessed August 23, 2026; used for workflow-orchestration comparison.
- [29] Prometheus Authors. Prometheus documentation.
<https://prometheus.io/docs/introduction/overview/>. Accessed August 23, 2026.
- [30] Trino Software Foundation. Trino documentation.
<https://trino.io/docs/current/>. Accessed August 23, 2026.

Appendix A

Requirements-to-Evidence Traceability

This appendix connects the requirements defined in Chapter IV with the implemented platform components and the retained validation material. It is intentionally concise: the detailed requirement definitions remain in Tables IV.2, IV.3, and IV.4, while the validation interpretation remains in Chapter VIII.

The traceability analysis is organized around the functional and non-functional requirement identifiers established in Chapter IV. For each requirement, the appendix identifies the implemented technical response and its principal verification source. Evidence includes runtime interfaces, retained validation results, and deployment information extracted from the implemented service definitions. Each artifact is interpreted only within its observable scope, while the detailed technical discussion and validation boundaries remain in the main report.

A.1 Functional Requirements

ID	Required capability	Implemented response	Principal retained evidence
FR-1	Structured ingestion of heterogeneous customer data	One dataset-aware ingestion path handles Bank Marketing, Online Shoppers Intention, and Online Retail II inputs.	Dataset-specific Spark applications in Figure ?? and the raw-pipeline history in Figure ??.
FR-2	Reproducible topic-based transport	A three-broker Kafka deployment provides the transport boundary used by the controlled ingestion workflow.	Broker placement in Table ?? and orchestration history in Figure ??.
FR-3	Distributed, replayable bronze persistence	HDFS retains raw artifacts before governed lakehouse loading, with one DataNode on each Linux VM.	Three live DataNodes and retained capacity in Figure ??.
FR-4	Governed raw lakehouse loading	Spark converts bronze inputs into Iceberg/Parquet raw tables registered through the shared metadata layer.	Completed raw-to-Iceberg jobs in Figure ??.
FR-5	Centralized analytical metadata	Hive Metastore and its PostgreSQL repository provide shared table discovery for Spark and Trino.	Metastore availability in Figure ?? and service placement in Table ??.
FR-6	Interactive analytical SQL access	Trino exposes governed lakehouse tables independently from the transformation runtime.	Completed Online Retail II query in Figure ??.
FR-7	Layered governed transformations	dbt-spark coordinates dependency-aware transformations after the raw lakehouse load.	Successful dbt workflow state in Figures ?? and ??.
FR-8	Persistent data-quality validation	Great Expectations runs as explicit bootstrap and validation tasks inside the Kafka-HDFS-Spark raw pipeline, recording validation results and HTML Data Docs.	Three successful validation records in Figure ??.
FR-9	Observable workflow orchestration	Airflow separates reset, setup, readiness, integrated raw loading/quality, and dbt transformation into five controlled workflows.	Successful run history in Figure ??; the authoritative final workflow set is documented in Chapter VIII.
FR-10	Infrastructure and pipeline observability	Prometheus collects platform signals while Grafana exposes infrastructure, metastore, and pipeline views.	Figures ??–??.

Table A.1: Traceability from functional requirements to implemented and retained evidence

A.2 Non-Functional Requirements

ID	Quality objective	Implemented response	Verification reference
NFR-1	Horizontal evolution	Kafka brokers, HDFS DataNodes, Spark workers, and Trino execution roles are distributed across three VMs.	Table ?? and Figures ??, ??, and ??.
NFR-2	Modularity and separation of concerns	Orchestration, monitoring, transport, storage, metadata, compute, validation, and query access remain separate service groups.	Compose-derived topology in Appendix ??.
NFR-3	Reproducibility	Replayable bronze storage, explicit workflow boundaries, persistent Spark events, and retained validation artifacts support controlled reruns.	Figures ??, ??, and ??.
NFR-4	Observability	Health checks, cAdvisor, Prometheus, Grafana, exporters, and Airflow histories expose service and pipeline state.	Figures ??–??.
NFR-5	Baseline security	Kerberos configuration and role-specific keytabs protect Hadoop-facing identities; Caddy gateways provide protected service exposure.	Security-related Compose controls in Table ?? and the validation boundaries stated in Chapter VIII.
NFR-6	Inspectability and evidence retention	Runtime interfaces, task histories, validation documents, dashboards, and persistent event data retain evidence beyond console output.	Evidence set in Appendix ??.
NFR-7	Resource feasibility	Central coordination services are concentrated on VM2 while worker roles are distributed and the control plane remains local.	Table ?? and the capacity view in Figure ??.

Table A.2: Traceability from non-functional requirements to architectural verification

The traceability confirms that the appendices support the completed implementation without replacing the argument developed in the main chapters. A screenshot is treated as evidence only for the indicators explicitly identified in its caption.

Appendix B

Implemented Deployment Reference

This appendix summarizes the four Docker Compose definitions supplied with the implemented platform. The objective is to document deployment responsibility and service placement, not to reproduce operational configuration verbatim. Passwords, key material, secret values, host-specific paths, and complete environment-variable contents are therefore intentionally excluded.

B.1 Compose-to-Node Allocation

Source definition	Execution zone	Architectural responsibility
<code>docker-compose.yml</code>	Local control plane	Airflow and its PostgreSQL repository, Prometheus, Grafana, Kafka UI, monitoring exporters, and the TLS gateway used to expose control-plane interfaces.
<code>docker-compose_2.yml</code>	VM1 data-plane node	Kafka broker 1, HDFS DataNode 1, Spark worker 1, Trino worker 1, and cAdvisor.
<code>docker-compose_3.yml</code>	VM2 coordination node	Kerberos KDC, Kafka broker 2, HDFS NameNode and DataNode 2, HDFS administration client, Hive Metastore and PostgreSQL, Spark coordination and serving services, Trino coordinator and gateway, and cAdvisor.
<code>docker-compose_4.yml</code>	VM3 data-plane node	Kafka broker 3, HDFS DataNode 3, Spark worker 3, Trino worker 2, and cAdvisor.

Table B.1: Allocation of the supplied Compose definitions to the deployed execution zones

B.2 Service Placement

Node	Principal Compose services	Resulting role
Local control plane	airflow-postgres, airflow-init, airflow-api-server, airflow-scheduler, airflow-dag-processor, airflow-triggerer	Workflow definition, scheduling, execution-state persistence, task processing, and operator access.
Local control plane	postgres-exporter, pipeline-metrics-exporter, prometheus, grafana, kafka-ui, tls-gateway	Central monitoring, dashboarding, transport inspection, and protected browser-facing entry points.
VM1	broker-1, datanode-1, spark-worker-1, trino-worker-1, cadvisor	First distributed worker node for transport, storage, processing, querying, and container metrics.
VM2: identity and storage	kerberos-kdc, broker-2, namenode, datanode-2, hdfs-admin-client	Central identity authority, HDFS namespace coordination, administrative access, and local participation in replicated storage.
VM2: metadata and compute	hive-metastore-db, hive-metastore, spark-master, spark-submit-client, spark-worker-2, spark-history-server, spark-thrift-server	Shared lakehouse metadata plus Spark coordination, submission, worker capacity, execution history, and SQL-serving support.
VM2: analytical access	trino-coordinator, trino-gateway, cadvisor	Distributed-query coordination, protected Trino entry, and node-level monitoring.
VM3	broker-3, datanode-3, spark-worker-3, trino-worker-2, cadvisor	Third transport/storage/compute node and second dedicated Trino worker.

Table B.2: Service placement extracted from the four Docker Compose definitions

B.3 Cross-Cutting Deployment Properties

Property	Compose-derived implementation
Network organization	Data-plane services use host networking on the Linux VMs. The local control plane uses its Compose network, while a Caddy gateway publishes the selected Airflow, Prometheus, Grafana, and Kafka UI interfaces.
Persistent state	Named volumes retain control-plane PostgreSQL, Prometheus, Grafana, and gateway state. VM definitions mount dedicated runtime directories for Kafka, HDFS, Spark events, Hive PostgreSQL, Trino, Kerberos, and audit data.
Identity and protected access	VM definitions mount the Kerberos client configuration and service-specific keytab locations. HDFS mounts TLS configuration, while Caddy gateways protect the Trino and control-plane browser paths.
Startup dependencies and health	Compose dependencies establish key startup relationships around PostgreSQL, Kerberos, HDFS, Hive, Spark, Trino, Prometheus, and Airflow. Health checks are declared for critical stateful and entry-point services.
Observability	cAdvisor runs on VM1, VM2, and VM3. The local Prometheus and Grafana services consume infrastructure, PostgreSQL, and pipeline-exporter signals.
Controlled configuration	Images, versions, addresses, ports, runtime directories, and credentials are parameterized through environment variables. The report retains the architecture and role mapping while excluding secret values.

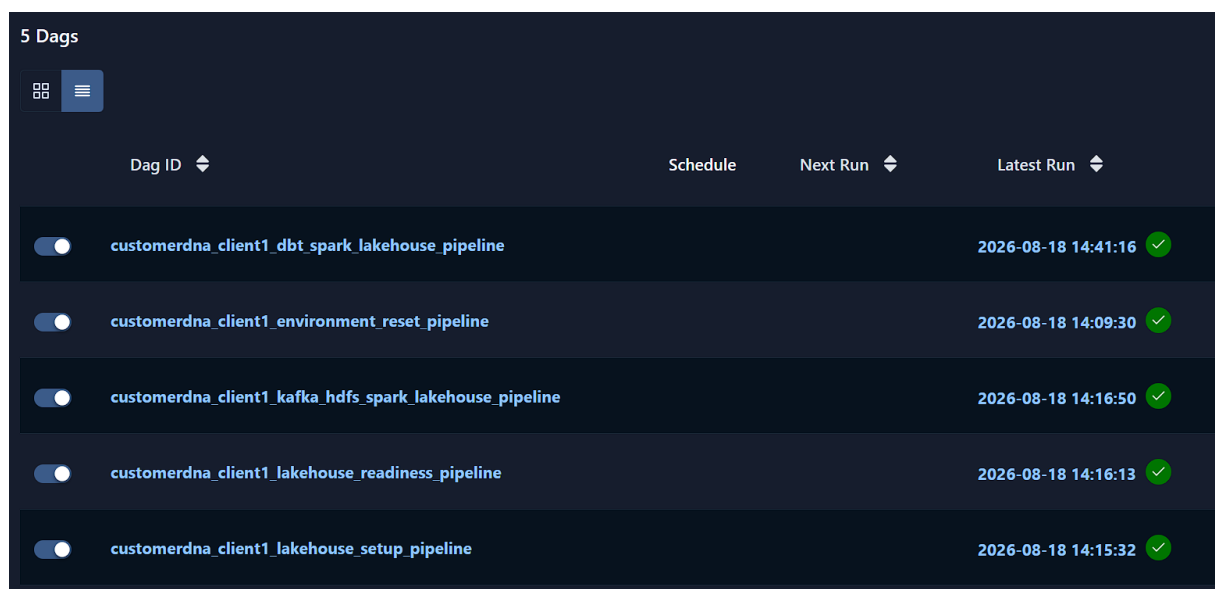
Table B.3: Cross-cutting implementation properties extracted from the Compose definitions

Appendix C

Selected Validation Evidence

This appendix presents the eight retained interface captures supplied for the report. They were collected at different moments of the validation campaign rather than as one synchronized snapshot. Each figure supports only the indicators stated in its caption; the final architecture and validation boundaries remain those documented in Chapters VI–VIII.

C.1 Orchestration and Quality Evidence



Dag ID	Schedule	Next Run	Latest Run
☒ customerdna_client1_dbt_spark_lakehouse_pipeline			2026-08-18 14:41:16 ✓
☒ customerdna_client1_environment_reset_pipeline			2026-08-18 14:09:30 ✓
☒ customerdna_client1_kafka_hdfs_spark_lakehouse_pipeline			2026-08-18 14:16:50 ✓
☒ customerdna_client1_lakehouse_readiness_pipeline			2026-08-18 14:16:13 ✓
☒ customerdna_client1_lakehouse_setup_pipeline			2026-08-18 14:15:32 ✓

Figure C.1: Airflow grid captured on August 18, 2026, showing successful latest runs for the five implemented workflows: environment reset, lakehouse setup, lakehouse readiness, Kafka–HDFS–Spark raw loading with integrated raw-quality tasks, and dbt-spark transformation.

Data Docs | local_site

Validation Results Expectation Suites

Search  Clear Filters

Status	Run Time	Run Name	Asset Name	Batch ID	Expectation Suite
▼			▼		▼
✓	2026-08-03 22:54:57 UTC	__none__	raw_lakehouse_metrics	client1_quality_metrics-raw_lakehouse_metrics	raw_lakehouse_metrics_suite
✓	2026-08-03 22:54:53 UTC	__none__	raw_lakehouse_metrics	client1_quality_metrics-raw_lakehouse_metrics	raw_lakehouse_metrics_suite
✓	2026-08-03 22:54:50 UTC	__none__	raw_lakehouse_metrics	client1_quality_metrics-raw_lakehouse_metrics	raw_lakehouse_metrics_suite

Showing 1 to 3 of 3 rows

Figure C.2: Great Expectations Data Docs showing three successful raw-lakehouse validation records retained on August 3, 2026.

C.2 Observability Evidence



Figure C.3: Grafana infrastructure overview showing six healthy monitoring targets, twelve observed CPU cores, 11.46 GiB of host memory, and the associated availability and utilization panels.

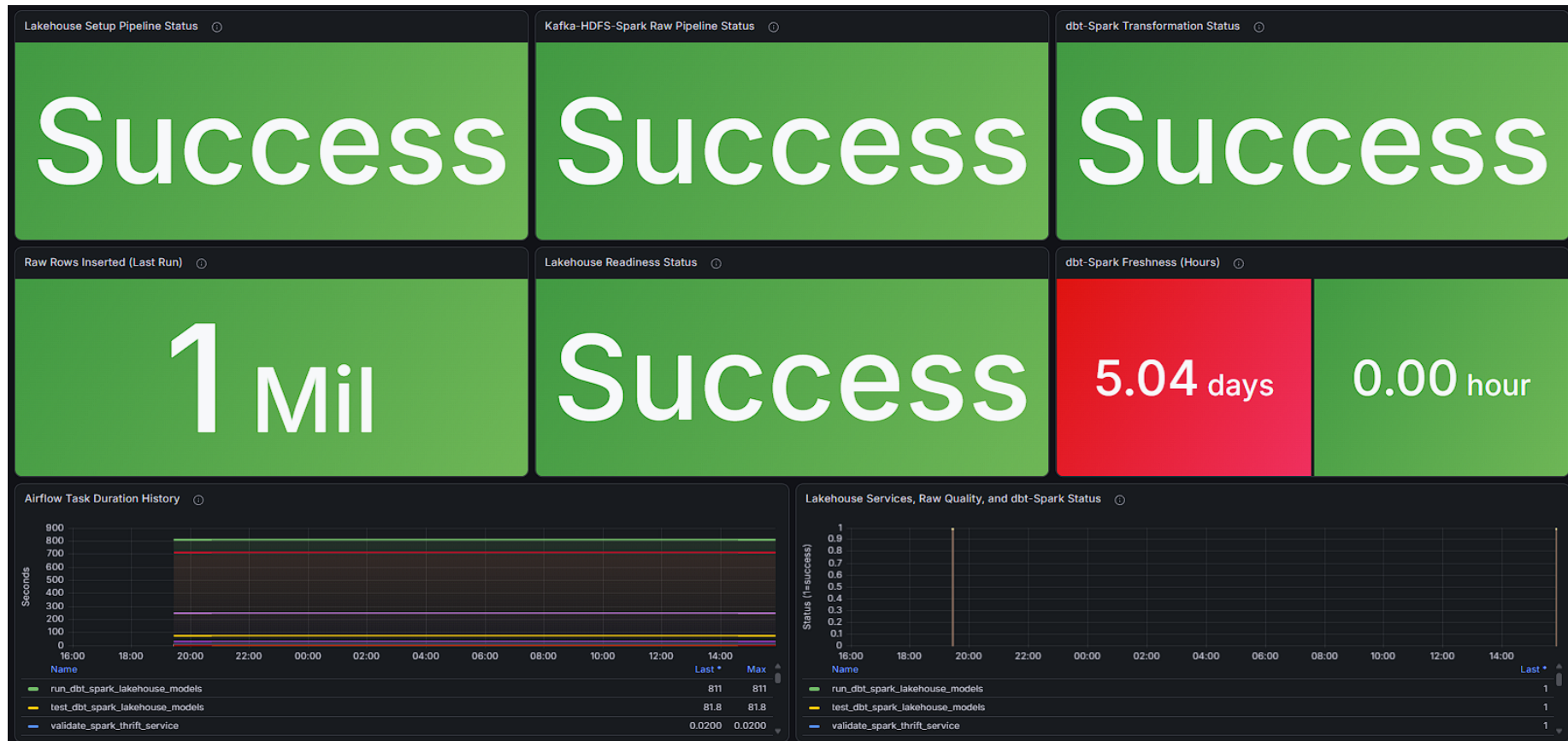


Figure C.4: Grafana pipeline-health view retaining workflow status, raw-row volume, readiness, and freshness indicators as they appeared at the dashboard capture time.

C.3 Storage and Metadata Evidence

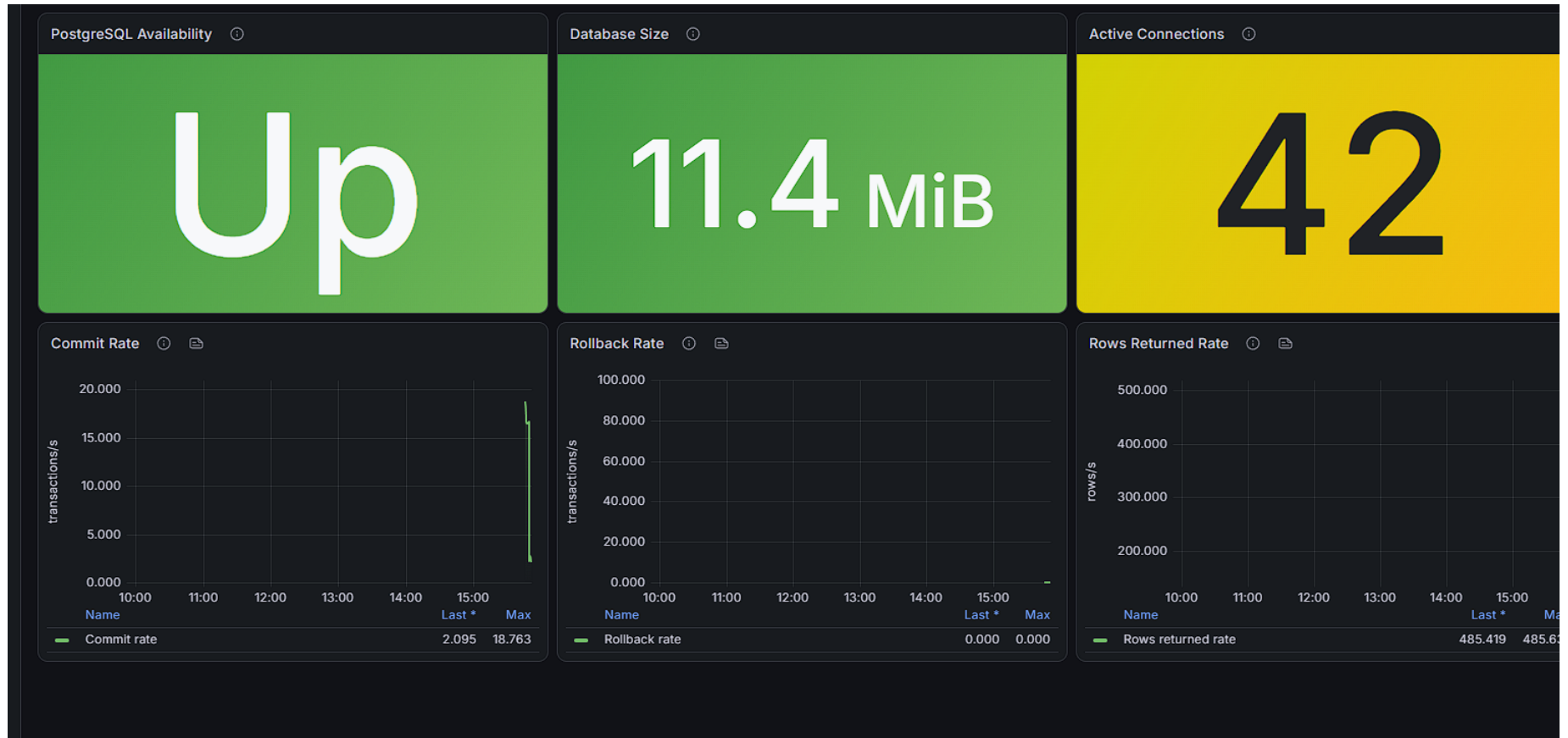


Figure C.5: Grafana Hive Metastore dashboard showing PostgreSQL availability, a database size of 11.4 MiB, 42 active connections in this time-specific capture, and transaction and row-return activity.

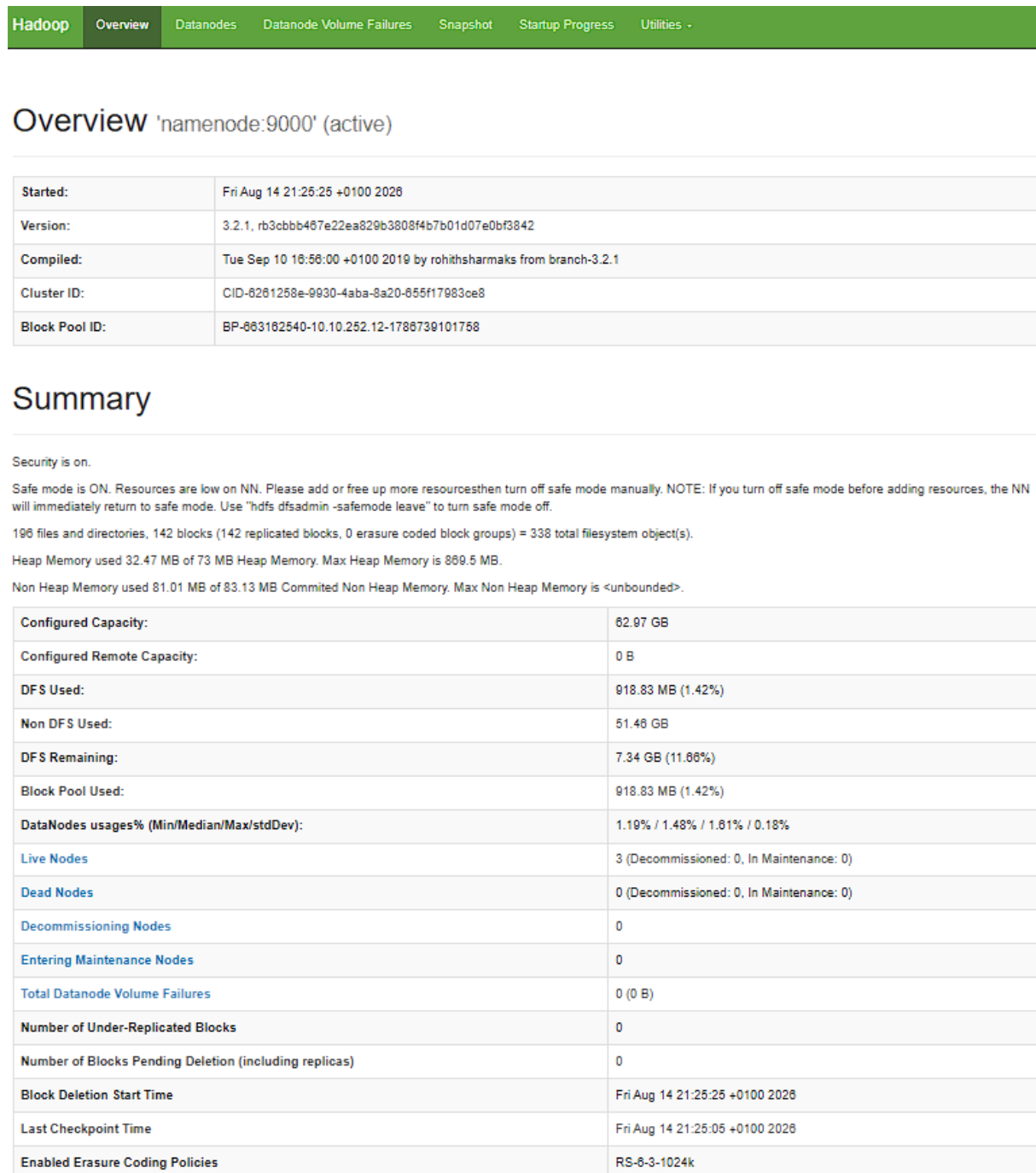


Figure C.6: HDFS administrative overview showing 62.97 GB of configured capacity, three live DataNodes, no dead nodes, and no under-replicated blocks. The visible security and safe-mode notices are not used as evidence for the final hardened access path.

C.4 Distributed Processing and Analytical Access

Spark Master at spark://10.10.252.12:7077

URL: spark://10.10.252.12:7077

REST URL: spark://10.10.252.12:6066 (cluster mode)

Workers: 3 Alive, 0 Dead, 0 Decommissioned, 0 Unknown

Cores in use: 3 Total, 1 Used

Memory in use: 3.0 GiB Total, 512.0 MiB Used

Resources in use:

Applications: 1 Running, 12 Completed

Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)

Status: ALIVE (Environment, Log)

Workers (3)

Worker Id	Address	State	Cores	Memory
worker-20260814202200-10.10.252.11-34787	10.10.252.11:34787	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)
worker-20260814202329-10.10.252.13-33877	10.10.252.13:33877	ALIVE	1 (1 Used)	1024.0 MiB (512.0 MiB Used)
worker-20260814202556-10.10.252.12-45845	10.10.252.12:45845	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260814202731-0002	(kill) Thrift JDBC/ODBC Server	1	512.0 MiB		2026/08/14 20:27:31	spark	RUNNING	210.3 h

Completed Applications (12)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260818133517-0012	customerdna-client1-online_retail_2-bronze-to-iceberg	2	512.0 MiB		2026/08/18 13:35:17	spark	FINISHED	50 s
app-20260818132700-0011	customerdna-client1-online_shoppers_intention-bronze-to-iceberg	2	512.0 MiB		2026/08/18 13:27:00	spark	FINISHED	29 s
app-20260818132514-0010	customerdna-client1-bank_marketing-bronze-to-iceberg	2	512.0 MiB		2026/08/18 13:25:14	spark	FINISHED	1.2 min
app-20260818131917-0009	customerdna-client1-bank_marketing-bronze-to-iceberg	2	512.0 MiB		2026/08/18 13:19:17	spark	FINISHED	2.0 min
app-20260815171309-0008	customerdna-client1-online_retail_2-bronze-to-iceberg	2	512.0 MiB		2026/08/15 17:13:09	spark	FINISHED	1.4 min
app-20260815170334-0007	customerdna-client1-online_shoppers_intention-bronze-to-iceberg	2	512.0 MiB		2026/08/15 17:03:34	spark	FINISHED	34 s
app-20260815170032-0006	customerdna-client1-bank_marketing-bronze-to-iceberg	2	512.0 MiB		2026/08/15 17:00:32	spark	FINISHED	1.9 min
app-20260815113847-0005	customerdna-client1-online_retail_2-bronze-to-iceberg	2	512.0 MiB		2026/08/15 11:38:47	spark	FINISHED	53 s
app-20260815113226-0004	customerdna-client1-online_shoppers_intention-bronze-to-iceberg	2	512.0 MiB		2026/08/15 11:32:26	spark	FINISHED	41 s
app-20260815112705-0003	customerdna-client1-bank_marketing-bronze-to-iceberg	2	512.0 MiB		2026/08/15 11:27:05	spark	FINISHED	4.3 min
app-20260814202710-0001	Thrift JDBC/ODBC Server	1	512.0 MiB		2026/08/14 20:27:10	spark	FINISHED	13 s
app-20260814202609-0000	Thrift JDBC/ODBC Server	1	512.0 MiB		2026/08/14 20:26:09	spark	FINISHED	34 s

Figure C.7: Spark master interface showing three alive workers, three aggregate cores, 3.0 GiB of worker memory, an active Thrift JDBC/ODBC service, and twelve completed applications.

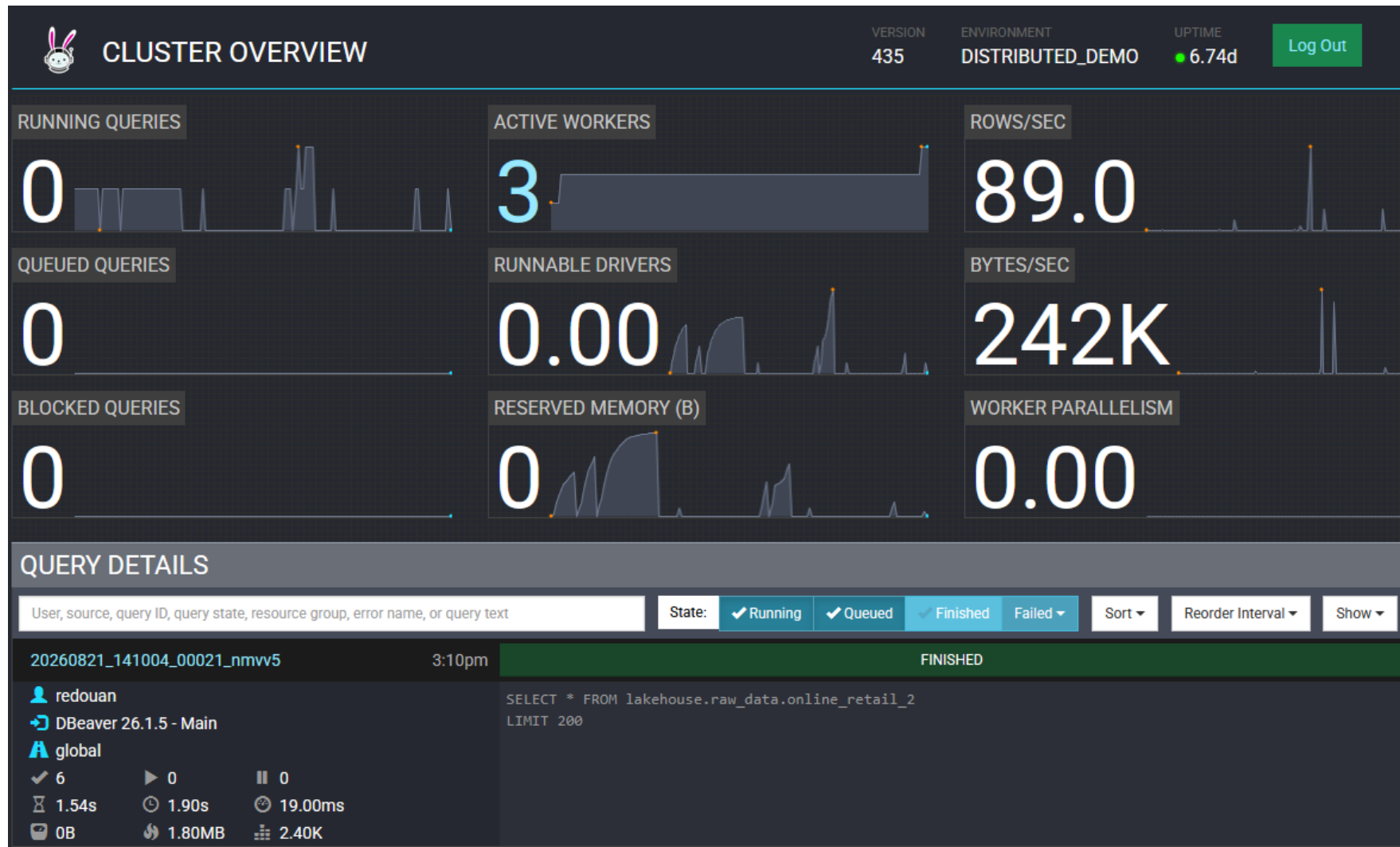


Figure C.8: Trino 435 cluster view showing environment DISTRIBUTED_DEMO, 6.74 days of uptime, three active execution nodes, and a completed query over `lakehouse.raw_data.online_retail_2` with the displayed throughput snapshot.